



МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ

ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ
УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«МОСКОВСКИЙ АВИАЦИОННЫЙ ИНСТИТУТ
(национальный исследовательский университет)»

Институт № 3 «Системы управления, информатика и электроэнергетика»
Кафедра 301 Группа МЗО-201СВ-24
Направление подготовки 27.04.04 «Управление в технических системах»
Программа 27.04.04.СМ1 Управление и информационные технологии в технических системах
Уровень образования специализированное высшее образование - магистратура
Квалификация инженер-исследователь

ВЫПУСКНАЯ КВАЛИФИКАЦИОННАЯ РАБОТА

ПЛАН РАБОТЫ

над выпускной квалификационной работой

Обучающийся Воеводова Надежда Андреевна
(фамилия, имя, отчество полностью)
Руководитель Кананадзе Сергей Сергеевич
(фамилия, имя, отчество полностью)
к.т.н., доцент, доцент кафедры 301 МАИ
(ученая степень, ученое звание, должность и место работы)

1. Наименование предварительной темы (тематика) Разработка системы идентификации подвижных объектов в зоне пешеходных переходов
2. Срок сдачи обучающимся законченной работы 24.05.2026
3. Цель Разработать автомобильную систему автоматической идентификации по видеопотоку с фронтальной камеры автомобиля световых дорожных знаков, информирующих о пешеходах, а также подвижных объектов в пешеходной зоне с целью минимизации риска дорожно-транспортных происшествий за счёт своевременного принятия решений о снижении скорости или экстренном торможении автомобиля.

Перечень иллюстративно-графических материалов: *при наличии

№ п/п	Наименование	Количество листов
1	2	3
1	Структурная схема системы автоматической идентификации подвижных объектов в зоне пешеходных переходов	1
2	Структурно-функциональная схема взаимодействия подсистем	1
3	Диаграмма потока данных при обработке видеопотока	1
4	Перечень анализируемых дорожных знаков и элементов разметки	1
5	Алгоритм контекстно-зависимой обработки видеопотока в зоне пешеходного перехода	1

6	Алгоритм принятия решения о необходимости снижения скорости или торможения автомобиля	1
7	Модель расчёта тормозного пути транспортного средства	1
8	Результаты оценки качества детекции объектов	2
9	Зависимость качества детекции от порога уверенности	1
10	Оценка вычислительной эффективности системы	1
11	Примеры работы системы в различных условиях наблюдения	2
12	Моделирование работы системы и идентификации на тестовых видеоданных	2

4. Перечень задач, решаемых для достижения поставленной цели

№ п/п	Наименование задачи	Срок выполнения	Примечание
1	2	3	4
1	Обзор и анализ нейросетевых методов распознавания дорожных знаков, пешеходов и других участников движения	10.12.2024	
2	Сбор и подготовка видеоданных, преобразование и разметка данных, формирование обучающей и тестовой выборки	31.01.2025	
3	Алгоритм распознавания выбранных объектов на видеопотоке и первичная оценка точности	28.02.2025	
4	Разработка алгоритма отслеживания движения для определения траекторий пешеходов и других подвижных объектов	31.03.2025	
5	Выбор метода вычисления расстояния от автомобиля до пешеходного перехода и подвижных объектов на основе видеоданных	30.04.2025	
6	Разработка алгоритма вычисления времени до потенциального пересечения траекторий автомобиля и объекта	31.08.2025	
7	Разработка логики принятия решения о необходимости торможения на основе данных о расстоянии и времени до столкновения	30.11.2025	
8	Моделирование работы системы в различных погодных и временных условиях	28.02.2026	
9	Анализ результатов работы системы	24.05.2026	

5. Исходные материалы и пособия 1) Цехановский В.В., Бутырский Е.Ю., Жукова Н.А., Мельников В.Б. Искусственные нейронные сети: учебник для вузов. — Москва: КноРус, 2023. — 352 с.

2) Браунли Дж. Глубокое обучение с Python: руководство для разработчиков. — Москва: ДМК Пресс, 2022. — 416 с.

3) Никоненко С., Кадурин А., Архангельская Е. Глубокое обучение. — СПб.: Питер, 2018 — 480 с.

6. Дата составления плана 19.11.2024 



**ЗАКЛЮЧЕНИЕ
РЕЦЕНЗЕНТА
О ВЫПУСКНОЙ КВАЛИФИКАЦИОННОЙ РАБОТЕ**

Обучающийся Воеводова Надежда Андреевна
Институт № 3 «Системы управления, информатика и электроэнергетика»
Кафедра 301 Группа МЗО-201СВ-24
Направление подготовки 27.04.04 Управление в технических системах
Программа 27.04.04.СМ1 Управление и информационные технологии в технических системах
Уровень образования специализированное высшее образование - магистратура
Квалификация инженер-исследователь
Наименование темы Разработка системы идентификации подвижных объектов в зоне пешеходных переходов
Рецензент Тупиков Максим Михайлович
(фамилия, имя, отчество полностью)
к.т.н., заведующий лабораторией №30 НИИОСП им. Н.М. Герсевича АО «НИЦ «Строительство»
(ученая степень, ученое звание, должность и место работы)

Отмеченные достоинства: работа Воеводовой Н.А. посвящена актуальной задаче разработки системы автоматической идентификации подвижных объектов в зоне пешеходных переходов на основе анализа видеопотока с фронтальной камеры автомобиля. Актуальность подтверждается статистическими данными о дорожно-транспортных происшествиях с участием пешеходов. К достоинствам работы следует отнести комплексный подход к решению поставленной задачи: обзор методов и формализации математической модели до программной реализации, экспериментального исследования. Разработанная математическая модель включает кинематическое описание движения объектов, алгоритм фильтрации Калмана, расчёт тормозного пути и критерий времени до столкновения, что свидетельствует о глубине проработки теоретической части. Заслуживает внимания предложенный алгоритм контекстно-зависимой обработки видеопотока с переключением между режимами наблюдения и активной идентификации, позволяющий снизить вычислительную нагрузку.

Отмеченные недостатки:

- в работе отсутствуют натурные испытания разработанной системы на реальном транспортном средстве;
- качество детекции в ночных условиях существенно ниже дневных показателей, что ограничивает применимость системы в тёмное время суток.

Заключение: несмотря на отмеченные недостатки, выпускная квалификационная работа Воеводовой Н.А. выполнена на высоком научно-техническом уровне, соответствует требованиям предъявляемым к выпускным квалификационным работам по направлению подготовки 27.04.0 «Управление в технических системах», и заслуживает оценки «отлично», а её автор — присвоены квалификации «инженер-исследователь».

Handwritten signature



**ОТЗЫВ
НАУЧНОГО РУКОВОДИТЕЛЯ**

Обучающийся Воеводова Надежда Андреевна
(фамилия, имя, отчество полностью)
Институт № 3 «Системы управления, информатика и электроэнергетика»
Кафедра 301 Группа МЗО-201СВ-24
Направление подготовки 27.04.04 «Управление в технических системах»
Программа 27.04.04.СМ1 «Управление и информационные технологии в технических системах»
Уровень образования специализированное высшее образование - магистратура
Квалификация инженер-исследователь
Наименование темы Разработка системы идентификации подвижных объектов в зоне пешеходных переходов
Руководитель Капанадзе Сергей Сергеевич, к.т.н., доцент, доцент каф. 301 МАИ
(фамилия, имя, отчество полностью, ученая степень, ученое звание, должность и место работы)

Выпускная квалификационная работа Воеводовой Н.А. посвящена актуальной проблеме обеспечения безопасности пешеходов в зонах пешеходных переходов с использованием методов компьютерного зрения и глубокого обучения. Тема работы обусловлена значительным числом дорожно-транспортных происшествий с участием пешеходов как в Российской Федерации, так и в мире, что подтверждается статистическими данными, приведёнными во введении.

В работе выполнен обстоятельный обзор современных методов детекции и трекинга объектов, обоснован выбор архитектуры YOLOv8 и алгоритма многообъектного трекинга на основе IoU-ассоциации с фильтром Калмана. Разработана математическая модель системы, включающая кинематическое описание движения объектов, алгоритм фильтрации Калмана, формулы расчёта тормозного пути и критерий времени до столкновения (TTC). Предложена модульная архитектура программного обеспечения в виде замкнутого контура управления из шести подсистем.

Следует отметить предложенный автором алгоритм контекстно-зависимой обработки видеопотока с двумя режимами работы (наблюдение и активная идентификация), позволяющий снизить вычислительную нагрузку за счёт активации полного цикла анализа только при наличии признаков пешеходного перехода. Данный подход свидетельствует о понимании автором инженерных ограничений встраиваемых систем.

Воеводова Н.А. продемонстрировала способность к систематическому анализу научной литературы, формализации инженерных задач и экспериментальному исследованию. Структура работы логична, изложение последовательно, выводы обоснованы результатами экспериментов.

Работа проверена на объём заимствования. Процент заимствования – 5,84%.

Заключение: в целом, представленная работа выполнена на высоком научно-техническом уровне, а студент Воеводова Н.А. заслуживает присвоения квалификации «инженер-исследователь» по направлению подготовки 27.04.04 «Управление в технических системах».

Воеводова Надежда Андреевна МЗО-201СВ-24 29 мая 2026.pdf
Надежда Андреевна
Темы идентификации подвижных объектов в зоне



Научные статьи 2.0, Википедия, Англоязычная
Энциклопедий, Библиотека Либрусек, Университетская
КФУ, ВКР Российского университета кооперации,
коллекция ПГУТИ, Репозиторий открытого доступа СПб
тронная библиотека "КиберЛенинка", ЦНМБ Сеченова,
хсертации ВАК, Диссертации РГБ, Авторефераты РГБ,
ТС. Изобретения, ФИПС. Полезные модели, ФИПС.
ы, Коллекция Руконт, Библиотека им. Ушинского,
ть 2), Открытые научные источники, НЭБ,
ые документы I, Правовые документы II, Правовые
законодательства Российской Федерации

рки

документа: 91%

90,64%

ания 5,84%

91%

644%

ОПРЕДЕЛЕНИЯ, ОБОЗНАЧЕНИЯ И СОКРАЩЕНИЯ

В настоящей работе применяют следующие определения, сокращения и обозначения.

Определения и сокращения:

ABS (Anti-lock Braking System) – антиблокировочная система тормозов – система, предотвращающая блокировку колёс при экстренном торможении, что позволяет сохранить управляемость автомобиля.

ADAS (Advanced Driver Assistance Systems) – усовершенствованные системы помощи водителю – комплекс аппаратно-программных средств, автоматизирующих отдельные функции восприятия дорожной обстановки и управления транспортным средством.

AEB (Autonomous Emergency Braking) – автономное экстренное торможение – функция ADAS, способная самостоятельно инициировать торможение при обнаружении угрозы столкновения.

Anchor-free подход – метод детекции без заранее заданных якорных рамок (anchor boxes). В таком подходе сеть напрямую предсказывает положение объекта, например смещения от центра ячейки до границ объекта. Это упрощает настройку детектора и делает модель более гибкой при обнаружении объектов разных размеров.

Backbone – базовая сеть-экстрактор признаков – начальная и наиболее крупная часть архитектуры нейронной сети, которая преобразует входное изображение в набор карт признаков. Backbone выделяет из кадра важную информацию: контуры, текстуры, формы и более сложные визуальные структуры. Эти признаки затем используются следующими частями сети для обнаружения пешеходов, дорожных знаков и разметки перехода.

Benchmark (бенчмарк) – стандартизированный тест производительности, представляющий собой набор эталонных задач, данных и условий, на которых сравниваются различные модели или алгоритмы. Результатом бенчмарка является набор количественных метрик, позволяющих объективно оценить преимущества и недостатки каждого подхода. Бенчмарки

обеспечивают воспроизводимость сравнительных экспериментов и являются общепринятым инструментом оценки в области машинного обучения и компьютерного зрения.

BN (Batch Normalization) – нормализация по мини-батчу – метод стабилизации процесса обучения нейронной сети путём нормализации значений промежуточных слоёв.

CIoU (Complete IoU – полная IoU-метрика) – улучшенная функция оценки качества ограничивающих прямоугольников. В отличие от обычного IoU, CIoU дополнительно учитывает расстояние между центрами прямоугольников и сходство их соотношений сторон. Оценивает не только площадь перекрытия, но и то, насколько правильно предсказанная рамка расположена и насколько она похожа по форме на истинную рамку.

CLAHE (Contrast Limited Adaptive Histogram Equalization – адаптивное выравнивание гистограммы с ограничением контраста) – метод предобработки изображений для улучшения видимости в условиях низкой освещённости. Изображение делится на небольшие области, в каждой из которых локально выравнивается распределение яркости. Ограничение контраста предотвращает чрезмерное усиление шумов, бликов и случайных артефактов. Таким образом, CLAHE делает тёмные или слабоконтрастные участки кадра более различимыми, но не допускает чрезмерного «перекручивания» контраста.

CNN (Convolutional Neural Network) – свёрточная нейронная сеть – класс нейросетевых архитектур для обработки изображений. CNN использует обучаемые фильтры свёртки: небольшие матрицы коэффициентов, которые последовательно перемещаются по изображению и выделяют локальные признаки. На нижних уровнях такие фильтры обычно реагируют на границы, углы, контрастные переходы и текстуры, а на более глубоких уровнях – на сложные формы объектов, например силуэт пешехода, разметку пешеходного перехода или дорожный знак.

COCO (Common Objects in Context) – набор данных общего назначения для задач компьютерного зрения, содержащий более 200 000 аннотированных изображений с 80 классами объектов.

DeepSORT – расширение алгоритма SORT, дополненное нейросетевой моделью извлечения визуальных признаков Re-ID. Re-ID (Re-Identification – повторная идентификация) позволяет повторно узнавать один и тот же объект после кратковременной потери, перекрытия другим объектом или выхода из поля зрения. В рамках данной работы DeepSORT рассматривается как перспективное направление повышения устойчивости трекинга, тогда как базовая реализация использует IoU-ассоциацию с фильтром Калмана.

DFD (Data Flow Diagram) – диаграмма потоков данных – графическая нотация для моделирования информационных потоков между процессами системы.

ESP (Electronic Stability Program) – электронная система курсовой устойчивости – система активной безопасности, помогающая сохранять траекторию движения автомобиля при заносе или сносе.

Euro NCAP (European New Car Assessment Programme – Европейская программа оценки безопасности новых автомобилей) – независимая международная организация, основанная в 1997 году, которая проводит стандартизированные испытания (краш-тесты) и оценку систем безопасности. В контексте настоящей работы протоколы Euro NCAP используются как эталонные сценарии для оценки работы системы обнаружения пешеходов.

F1-score – F1-мера – среднее гармоническое точности и полноты: $F1 = 2 \cdot \text{Precision} \cdot \text{Recall} / (\text{Precision} + \text{Recall})$. Эта метрика применяется, когда важно одновременно учитывать и ложные срабатывания, и пропуски объектов.

FLOPs (Floating Point Operations – операции с плавающей точкой) – показатель вычислительной сложности модели, отражающий количество математических операций, необходимых для обработки входного изображения. Чем больше FLOPs, тем выше требования к вычислительным ресурсам и тем сложнее обеспечить работу в реальном времени. Для

автомобильной системы этот показатель важен, поскольку задержка обработки кадра напрямую влияет на возможность своевременного предупреждения или торможения.

FN (False Negative – ложноотрицательное срабатывание) – объект, который реально присутствовал в кадре, но не был обнаружен моделью. Пропуск объекта является наиболее критичной ошибкой в системах безопасности, поскольку необнаруженный пешеход представляет прямую угрозу.

FP (False Positive – ложноположительное срабатывание) – объект, который модель ошибочно обнаружила в том месте кадра, где реального объекта данного класса не существует. Примером является столб, ошибочно классифицированный как пешеход.

FPS (Frames Per Second) – частота кадров – число кадров, обрабатываемых системой за одну секунду.

GPU (Graphics Processing Unit) – графический процессор – специализированное вычислительное устройство, используемое для ускорения обучения и инференса нейронных сетей.

Head – детекционная головка – финальная часть архитектуры, которая формирует итоговые предсказания: координаты ограничивающих прямоугольников, класс объекта и уверенность модели. Простыми словами, Head выдаёт ответ системы: где находится объект, к какому классу он относится и насколько модель уверена в результате.

HMI (Human-Machine Interface) – интерфейс человек-машина – совокупность средств взаимодействия водителя с бортовыми системами автомобиля.

IDF1 (Identification F1-score – F1-мера идентификации) – метрика качества сохранения идентификаторов объектов во времени. Она показывает, насколько хорошо трекер понимает, что один и тот же пешеход на разных кадрах остаётся тем же объектом. Если трекер часто меняет ID одного пешехода или путает его с другим объектом, значение IDF1 снижается.

IoU (Intersection over Union) – пересечение по объединению – метрика перекрытия двух прямоугольных областей. Она вычисляется как отношение площади пересечения двух прямоугольников к площади их объединения. Если предсказанный прямоугольник полностью совпадает с реальным положением объекта, IoU равен 1. Если прямоугольники не пересекаются, IoU равен 0. В простом виде IoU показывает, насколько точно модель «обвела» объект на изображении.

mAP (mean Average Precision) – средняя Average Precision, или средняя точность детекции. Метрика показывает интегральное качество детектора с учётом правильных обнаружений, ложных срабатываний и пропусков объектов при разных порогах уверенности. Average Precision соответствует площади под PR-кривой (Precision–Recall), а mAP получается усреднением AP по всем анализируемым классам. В обозначении mAP@0.5 число 0,5 означает порог IoU: обнаружение считается корректным, если предсказанная рамка перекрывает истинную рамку с IoU не менее 0,5.

MOT (Multi-Object Tracking) – многообъектный трекинг – задача одновременного отслеживания множества объектов на видеопоследовательности с присвоением каждому уникального идентификатора.

MOTA (Multiple Object Tracking Accuracy) – точность многообъектного трекинга – интегральная метрика, учитывающая пропуски, ложные срабатывания и переключения идентификаторов.

Neck (PANet – Path Aggregation Network) – агрегатор многомасштабных признаков, расположенный между Backbone и Head. Эта часть сети объединяет карты признаков разных уровней: одни карты лучше описывают крупные объекты вблизи автомобиля, другие – мелкие объекты на дальнем расстоянии. За счёт такого объединения модель может одновременно обнаруживать как близкого пешехода, так и небольшой дорожный знак или разметку перехода вдали.

NMS (Non-Maximum Suppression) – немаксимальное подавление – алгоритм постобработки, удаляющий дублирующиеся детекции одного объекта путём отбора предсказания с наивысшей уверенностью.

Precision (точность) – доля корректных обнаружений среди всех обнаружений, сделанных моделью. Вычисляется как отношение числа правильно найденных объектов к сумме правильно найденных и ошибочно обнаруженных объектов.

Recall (полнота) – доля обнаруженных объектов среди всех реально присутствующих объектов. Вычисляется как отношение числа правильно найденных объектов к сумме правильно найденных объектов и пропущенных объектов, которые присутствовали в кадре, но не были обнаружены моделью.

RTSD (Russian Traffic Sign Dataset) – набор данных российских дорожных знаков, содержащий 104 358 кадров с аннотациями 156 классов дорожных знаков.

SORT (Simple Online and Realtime Tracking) – простой онлайн-алгоритм трекинга в реальном времени, использующий фильтр Калмана и венгерский алгоритм для ассоциации детекций с треками. Ассоциация детекций с треками означает сопоставление объектов, найденных на текущем кадре, с уже существующими траекториями объектов на предыдущих кадрах. Иными словами, система определяет, является ли найденный прямоугольник новым объектом или продолжением движения уже отслеживаемого пешехода.

TensorRT – библиотека компании NVIDIA, предназначенная для оптимизации и ускорения инференса (прямого прохода) нейронных сетей на графических процессорах (GPU). Название TensorRT образовано от слов Tensor (тензор – многомерный массив данных, являющийся основной структурой представления информации в нейронных сетях) и RT (сокращение от Runtime – среда выполнения). В контексте настоящей работы TensorRT рассматривается как инструмент подготовки модели YOLOv8 к развёртыванию на платформе NVIDIA Jetson.

TP (True Positive – истинно положительное срабатывание) – объект, который был правильно обнаружен моделью и действительно присутствует в кадре. Количество истинно положительных срабатываний используется при вычислении метрик точности и полноты детекции.

TTC (Time-To-Collision) – время до столкновения – ключевой критерий оценки опасности, вычисляемый как отношение текущей дистанции до объекта к относительной скорости сближения.

YOLO (You Only Look Once) – семейство нейросетевых архитектур для одностадийной детекции объектов, выполняющих классификацию и локализацию за один проход сети.

Обозначения:

b – обучаемый параметр смещения (bias)

c – диагональ наименьшего охватывающего прямоугольника

D – расстояние от автомобиля до объекта, м

$d(t)$ – расстояние до объекта в момент времени t , м

$d_{\text{запас}}$ – дополнительная дистанция безопасности, м

d_0 – текущая дистанция до объекта, м

F – матрица перехода состояния фильтра Калмана

f – фокусное расстояние камеры, пикс

FN – число ложноотрицательных срабатываний (False Negative)

FP – число ложноположительных срабатываний (False Positive)

g – ускорение свободного падения (9,81 м/с²)

GT – общее число эталонных объектов

H – матрица наблюдения фильтра Калмана

$h_{\text{пикс}}$ – высота ограничивающего прямоугольника объекта, пикс

$H_{\text{реал}}$ – реальная (известная) высота объекта, м

I – единичная матрица

K – матрица усиления Калмана (Kalman gain)

L_{CIoU} – функция потерь CIoU

P – ковариационная матрица ошибки оценки

Q – ковариационная матрица шума процесса
 R – ковариационная матрица шума измерений
 $S_{ост}$ – полный остановочный путь автомобиля, м
 $S_{торм}$ – тормозной путь автомобиля, м
 $t_{реакц}$ – время реакции водителя или системы, с
 TP – число истинно положительных срабатываний (True Positive)
 TTC – время до столкновения (Time-To-Collision), с
 TTC_x, TTC_y – компоненты времени до столкновения по продольной и поперечной осям, с
 $TTC_{крит}$ – пороговое значение времени до столкновения, с
 v – скорость автомобиля, м/с
 $v_{авто}$ – скорость автомобиля, м/с
 $v_{объекта}$ – скорость объекта, м/с
 $v_{реал}$ – относительная скорость сближения, м/с
 W – ядро свёртки
 X – входная карта признаков или изображение
 $\hat{x}(k|k-1)$ – предсказанный вектор состояния на шаге k по данным до шага $k-1$
 $Y[i,j]$ – значение выходной карты признаков в позиции i, j
 $z(k)$ – вектор измерений (координаты ограничивающего прямоугольника от детектора)
 α, ν – параметры, учитывающие соотношение сторон в CIoU
 Δt – интервал дискретизации (период кадра), с
 ε – пороговое значение рассогласования компонент TTC, с
 μ – коэффициент сцепления колёс с дорожным покрытием
 $\rho(b, b_{gt})$ – евклидово расстояние между центрами предсказанного и истинного прямоугольников

СОДЕРЖАНИЕ

ВВЕДЕНИЕ.....	13
1 Обзор и анализ методов обнаружения и отслеживания объектов в контексте систем помощи водителю.....	18
1.1 Задачи компьютерного зрения в системах помощи водителю.....	18
1.2 Свёрточные нейронные сети и их применение в задачах распознавания	20
1.3 Архитектуры нейронных сетей для детекции объектов.....	25
1.4 Методы многообъектного трекинга.....	29
1.5 Оценка расстояния по данным монокулярной камеры.....	32
1.6 Анализ наборов данных для обучения и тестирования.....	34
1.7 Выводы по разделу 1.....	35
2 Математическая модель системы идентификации подвижных объектов....	36
2.1 Кинематическая модель движения объекта.....	36
2.2 Фильтр Калмана для оценки состояния объекта.....	37
2.3 Расчёт расстояния до объекта и тормозного пути автомобиля.....	39
2.4 Критерий времени до столкновения.....	41
2.5 Модель принятия решений.....	43
3 Разработка программного обеспечения системы идентификации.....	45
3.1 Принцип действия системы.....	45
3.1.1 Сенсорный уровень.....	45
3.1.2 Уровень детекции объектов.....	45
3.1.3 Контекстный анализатор.....	46
3.1.4 Уровень трекинга.....	46
3.1.5 Уровень оценки расстояния.....	46
3.1.6 Уровень принятия решений.....	47
3.2 Общая архитектура системы и структурная схема подсистем.....	47
3.3 Диаграмма потока данных при обработке видеопотока.....	49
3.4 Выбор инструментов и технологий.....	51
3.5 Алгоритм контекстно-зависимой обработки видеопотока.....	51
3.6 Архитектура двухмодельной детекции.....	54
3.7 Реализация модуля трекинга объектов.....	55
3.8 Алгоритм принятия решения о торможении.....	56
3.9 Описание реализованного программного конвейера.....	58
3.10 Сбор и подготовка данных.....	58

3.11 Дообучение моделей.....	58
3.12 Модуль визуализации результатов.....	60
3.13 Развёртывание на встраиваемых платформах.....	60
3.14 Выводы по разделу 3.....	62
4 Экспериментальные исследования и анализ результатов.....	63
4.1 Методика проведения экспериментов.....	63
4.2 Оценка качества детекции объектов.....	63
4.3 Зависимость качества от порога уверенности.....	66
4.4 Оценка качества трекинга.....	67
4.5 Влияние условий наблюдения.....	68
4.6 Оценка вычислительной эффективности.....	69
4.7 Моделирование на тестовых видеоданных.....	70
4.8 Анализ работы системы принятия решений.....	76
4.9 Итоговая сводка результатов.....	77
ЗАКЛЮЧЕНИЕ.....	79
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ.....	82
ПРИЛОЖЕНИЕ.....	85
Приложение А Полный листинг модуля обработки видеопотока.....	85
Приложение Б Иллюстративно-графические материалы.....	102

ВВЕДЕНИЕ

Безопасность пешеходов в зоне пешеходных переходов остаётся одной из наиболее значимых задач современной транспортной инженерии. Несмотря на развитие дорожной инфраструктуры, систем видеонаблюдения и электронных помощников водителя, наезды на пешеходов продолжают составлять существенную долю дорожно-транспортных происшествий (ДТП). Согласно докладу Всемирной организации здравоохранения «Global Status Report on Road Safety» (2023), ежегодно в мире в результате дорожно-транспортных происшествий погибают около 1,19 миллиона человек [1]. При этом 53 % всех погибших составляют уязвимые участники дорожного движения: пешеходы (23 %), мотоциклисты (21 %), велосипедисты (6 %) и пользователи средств микромобильности (3 %) [1, 2].

Для Российской Федерации проблема особенно актуальна в условиях плотного городского движения. По данным Министерства внутренних дел (МВД) России, за 2025 год в стране зафиксировано 128 тысяч ДТП с пострадавшими, в которых погибло 13,9 тысячи человек и 158 тысяч получили ранения [3, 4]. Несмотря на снижение аварийности на 3,1 % по сравнению с 2024 годом, наезды на пешеходов по-прежнему остаются вторым по частоте видом ДТП после столкновений транспортных средств [5]. В 2024 году таких наездов было зафиксировано 34,3 тысячи случаев. Три четверти всех ДТП в стране происходят в населённых пунктах, где плотность пешеходного потока максимальна [4]. Поэтому задача автоматического анализа дорожной сцены именно в зоне пешеходного перехода имеет не только техническое, но и социальное значение.

Современные методы компьютерного зрения и глубокого обучения позволяют автоматически обнаруживать объекты различных классов, отслеживать их перемещение и оценивать потенциальную опасность дорожной ситуации [6]. Архитектуры семейства YOLO (You Only Look Once – «смотришь только один раз») обеспечивают высокоскоростную детекцию объектов за один проход нейронной сети. Однако для практического

применения в автомобильной системе одной детекции недостаточно: необходимо учитывать контекст дорожной сцены, определять наличие пешеходного перехода, отслеживать траекторию пешехода и оценивать возможность пересечения траекторий пешехода и автомобиля.

В рамках настоящей работы рассматривается ограниченный, но практически значимый класс сценариев – движение автомобиля вблизи пешеходного перехода. Такой подход позволяет не анализировать всю дорожную сцену одинаково, а активировать полный цикл обработки только при наличии признаков пешеходного перехода: дорожной разметки, соответствующих знаков или светофора. Это снижает вычислительную нагрузку и уменьшает вероятность ложных решений в ситуациях, не связанных с переходом. Сценарии внезапного появления пешехода в данной работе также рассматриваются только применительно к зоне пешеходного перехода. Протоколы Европейской программы оценки безопасности новых автомобилей (European New Car Assessment Programme, Euro NCAP) используются в работе как ориентир для анализа требований к системам Autonomous Emergency Braking (АЕВ – автономное экстренное торможение), поскольку они включают сценарии обнаружения пешеходов и других уязвимых участников движения [7]. Вместе с тем разрабатываемая система не претендует на полное воспроизведение всех стандартных сценариев Euro NCAP. В работе адаптируется только та часть подобных сценариев, которая соответствует зоне пешеходного перехода и может быть реализована на основе фронтальной камеры автомобиля.

Традиционно ответственность за обнаружение пешеходов и начало торможения возлагается на водителя. Среднее время реакции водителя на внезапно возникающую опасность составляет 1,0–1,5 с [8], что при городской скорости 50 км/ч соответствует значительному расстоянию, пройденному автомобилем до начала торможения. Поэтому система, способная заранее обнаружить пешехода в зоне перехода, оценить его движение и сформировать

предупреждение или команду на снижение скорости, может повысить безопасность движения.

Актуальность данной работы обусловлена несколькими факторами. Во-первых, несмотря на развитие усовершенствованных систем помощи водителю (Advanced Driver Assistance Systems, ADAS), многие серийные решения ориентированы на типовые дорожные условия и не всегда учитывают специфику локальной инфраструктуры: особенности дорожных знаков, типы разметки пешеходных переходов и характерные погодные условия. Во-вторых, значительная часть серийных систем использует комбинацию камеры и радара, тогда как задача создания эффективного прототипа на основе единственной фронтальной камеры остаётся актуальной с точки зрения снижения стоимости и упрощения аппаратной части. В-третьих, использование контекстно-зависимой логики позволяет сосредоточить вычислительные ресурсы именно на участках дороги, где риск взаимодействия автомобиля с пешеходом наиболее высок.

Целью настоящей работы является разработка и исследование системы автоматической идентификации подвижных объектов в зоне пешеходных переходов на основе анализа видеопотока с фронтальной камеры автомобиля.

Система предназначена для обнаружения признаков пешеходного перехода, выявления пешеходов и других подвижных объектов в зоне перехода, отслеживания их перемещения, оценки степени опасности и формирования решения о необходимости предупреждения, снижения скорости или экстренного торможения автомобиля.

Для достижения поставленной цели сформулированы следующие задачи:

- провести обзор и анализ современных нейросетевых методов распознавания дорожных знаков, пешеходов и других участников движения;
- осуществить сбор, подготовку и разметку видеоданных, формирование обучающей и тестовой выборок;

- разработать алгоритм распознавания выбранных объектов на видеопотоке и выполнить первичную оценку точности;
- разработать алгоритм отслеживания движения для определения траекторий пешеходов и других подвижных объектов;
- выбрать метод вычисления расстояния от автомобиля до пешеходного перехода и подвижных объектов на основе видеоданных;
- разработать алгоритм вычисления времени до потенциального пересечения траекторий автомобиля и объекта;
- разработать логику принятия решения о необходимости торможения на основе данных о расстоянии и времени до столкновения;
- провести моделирование работы системы в различных погодных и временных условиях;
- выполнить анализ результатов работы системы.

Объектом исследования является процесс автоматической идентификации и отслеживания подвижных объектов в зоне пешеходных переходов с использованием данных фронтальной видеокамеры автомобиля.

Предметом исследования выступают алгоритмы и методы компьютерного зрения, многообъектного трекинга и принятия решений, применяемые для создания автомобильной системы помощи водителю в зоне пешеходного перехода.

Научная новизна работы заключается в разработке комплексного подхода к идентификации подвижных объектов в зоне пешеходных переходов, интегрирующего контекстно-зависимую обработку видеопотока с детектором на базе YOLOv8, алгоритм многообъектного трекинга и многоуровневую систему принятия решений на основе критерия времени до столкновения (Time-To-Collision, TTC). Предложен алгоритм контекстно-зависимой активации модулей системы, снижающий вычислительную нагрузку за счёт перехода от общего режима наблюдения к активной идентификации только при наличии признаков пешеходного перехода.

Практическая значимость работы заключается в создании прототипа системы, способной в режиме, близком к реальному времени, обнаруживать и отслеживать подвижные объекты в зоне пешеходного перехода, оценивать степень опасности дорожной ситуации и формировать предупреждающие или управляющие воздействия. Результаты работы могут быть использованы при разработке перспективных систем ADAS, ориентированных на предупреждение наездов на пешеходов в зоне регулируемых и нерегулируемых переходов.

Методы исследования. В работе использованы методы теории нейронных сетей (свёрточные нейронные сети, архитектура YOLO), теории оценивания (фильтр Калмана), математического моделирования (кинематические модели движения), теории принятия решений (пороговые правила, критерий TTC), а также методы экспериментального исследования с использованием видеоданных.

Работа состоит из введения, четырёх разделов, заключения, списка использованных источников и приложения. В первом разделе выполнен обзор современных методов детекции, классификации и трекинга объектов, обоснован выбор архитектуры YOLOv8 и алгоритма трекинга на основе метрики пересечения по объединению (Intersection over Union, IoU) с фильтром Калмана. Во втором разделе представлена математическая модель системы, включающая кинематическое описание движения объектов, фильтрацию Калмана, расчёт тормозного пути и критерий TTC. В третьем разделе описаны архитектура программного комплекса, алгоритм контекстно-зависимой обработки видеопотока и логика принятия решений. В четвёртом разделе приведены результаты экспериментального моделирования и анализ эффективности разработанной системы.

1 Обзор и анализ методов обнаружения и отслеживания объектов в контексте систем помощи водителю

В настоящем разделе рассмотрены теоретические основы и современные подходы к решению задач обнаружения, классификации и трекинга объектов на изображениях и видеопоследовательностях. Особое внимание уделено методам, применимым в контексте автомобильных систем помощи водителю для обеспечения безопасности пешеходов в зонах пешеходных переходов. Выполнен сравнительный анализ архитектур нейронных сетей для детекции объектов, рассмотрены алгоритмы многообъектного трекинга, методы оценки расстояния по данным монокулярной камеры, а также существующие подходы к обеспечению безопасности пешеходов в системах ADAS. Результаты обзора определяют выбор конкретных алгоритмов, которые формализованы в математической модели (раздел 2) и реализованы программно (раздел 3).

1.1 Задачи компьютерного зрения в системах помощи водителю

Системы помощи водителю ADAS представляют собой комплекс аппаратно-программных средств, предназначенных для повышения безопасности дорожного движения за счёт автоматизации отдельных функций восприятия дорожной обстановки и управления транспортным средством [9]. Развитие автомобильных систем активной безопасности началось с внедрения относительно простых электронных функций, направленных на стабилизацию движения и повышение управляемости автомобиля. К таким системам относятся антиблокировочная система тормозов Anti-lock Braking System (ABS), предотвращающая блокировку колёс при торможении, и электронная система курсовой устойчивости Electronic Stability Program (ESP), помогающая сохранять траекторию движения при заносе или сносе автомобиля. Дальнейшее развитие этих технологий привело к появлению систем ADAS, которые уже используют данные камер, радаров, лидаров, алгоритмы компьютерного зрения и методы машинного обучения для анализа дорожной обстановки и предупреждения опасных ситуаций [1].

В рамках систем ADAS задачи компьютерного зрения занимают центральное место, поскольку именно зрительная информация является наиболее информативным и доступным источником данных о дорожной сцене. Основные задачи компьютерного зрения, решаемые в контексте ADAS, включают [5]:

- детекцию объектов (object detection) – определение присутствия объектов заданных классов на изображении и их пространственного положения посредством ограничивающих прямоугольников (bounding boxes – рамок, описывающих границы обнаруженного объекта);

- классификацию объектов (object classification) – отнесение обнаруженного объекта к определённому типу (пешеход, автомобиль, велосипедист, дорожный знак и др.);

- семантическую и инстанционную сегментацию – попиксельное разделение сцены на области, принадлежащие различным объектам или классам;

- трекинг объектов (object tracking – отслеживание объектов) – сопровождение обнаруженных объектов от кадра к кадру с сохранением их идентичности, формирование траекторий и оценка динамических параметров.

Применительно к задаче обеспечения безопасности пешеходов в зонах пешеходных переходов наиболее актуальными являются следующие подзадачи: обнаружение дорожных знаков, информирующих о наличии пешеходного перехода (в соответствии с российскими правилами дорожного движения (ПДД) – знаки 5.19.1 и 5.19.2, а также предупреждающие знаки 1.22 и 1.23); детекция самой разметки пешеходного перехода (зебры – разметка 1.14); обнаружение и классификация подвижных объектов в зоне перехода; отслеживание выявленных объектов во времени для определения их траекторий и прогнозирования поведения [8].

Камера, установленная в передней части транспортного средства, формирует видеопоток, на основе которого осуществляется непрерывный анализ дорожной обстановки. Преимуществом использования видеокамеры в

качестве основного сенсора является относительно низкая стоимость, высокая информативность получаемых данных, а также широкая распространённость камер в современных автомобилях. Вместе с тем монокулярная камера (одионочная камера без стереопары) не обеспечивает прямого измерения расстояния до объектов, что требует применения дополнительных алгоритмических подходов для оценки дистанции [6].

Euro NCAP с 2024 года ужесточила требования к системам обнаружения пешеходов, включив в протокол тестирования сценарии с группами пешеходов, пешеходами на самокатах и велосипедистами в зоне пешеходных переходов [7]. Это дополнительно подтверждает актуальность разработки комплексных систем идентификации, способных обнаруживать широкий спектр объектов в сложных дорожных условиях.

1.2 Свёрточные нейронные сети и их применение в задачах распознавания

Свёрточные нейронные сети (CNN – Convolutional Neural Networks, или свёрточные нейросети) представляют собой класс нейросетевых архитектур для обработки изображений, лежащий в основе детектора YOLOv8, используемого в настоящей работе. CNN извлекают иерархические признаки из изображений: ранние слои обнаруживают границы и градиенты, средние – текстуры и формы, глубокие – семантически значимые паттерны (силуэты пешеходов, контуры автомобилей) [10, 11]. Именно эта способность делает CNN пригодными для задачи детекции объектов на дорожных сценах, где необходимо одновременно распознавать объекты различных классов при значительной вариативности условий наблюдения. Для понимания принципов работы детектора YOLOv8 рассмотрим основные компоненты CNN, которые в нём применяются: свёрточные слои, функции активации, слои нормализации и механизмы объединения признаков.

Рассмотрим каждый из перечисленных компонентов подробнее, поскольку понимание их работы необходимо для последующего анализа архитектуры YOLOv8 в подразделе 1.3.

Свёрточные слои (convolutional layers) выполняют извлечение локальных признаков из изображения. Для этого используются обучаемые фильтры свёртки – небольшие матрицы чисел, которые последовательно перемещаются по изображению и вычисляют отклик на определённые визуальные признаки. На ранних слоях такие фильтры могут реагировать на перепады яркости, края объектов и простые текстуры. На более глубоких слоях сеть формирует более сложные признаки, например форму человека, элементы дорожной разметки или характерные контуры дорожного знака.

В YOLOv8 применяются блоки C2f (Cross Stage Partial with 2 convolutions and feed-forward – блок частичного перекрёстного объединения с двумя свёртками и прямой передачей признаков). Такие блоки позволяют более эффективно передавать информацию между слоями сети. Их задача состоит в том, чтобы часть признаков проходила через несколько преобразований, а другая часть сохранялась и передавалась дальше. Благодаря этому сеть лучше сохраняет полезную информацию и одновременно снижает избыточные вычисления. Кроме того, в таких блоках используются остаточные связи (residual connections), которые позволяют сигналу и градиентам проходить через сеть более устойчиво. Простыми словами, остаточная связь даёт сети возможность не “терять” важную информацию при прохождении через большое количество слоёв.

Функции активации необходимы для внесения нелинейности в работу нейронной сети. Если бы сеть состояла только из линейных операций, она не смогла бы описывать сложные зависимости в изображении. Нелинейные функции позволяют модели различать более сложные визуальные структуры. В YOLOv8 используется функция активации SiLU (Sigmoid Linear Unit – сигмоидная линейная единица), которая задаётся выражением (1.1):

$$f(x) = x \cdot \sigma(x), \quad (1.1)$$

где $\sigma(x)$ – сигмоидная функция. Особенность SiLU состоит в том, что она плавно изменяет значение входного сигнала и помогает сети устойчиво обучаться. В отличие от более простых функций активации, SiLU не обрывает

сигнал резко, а сохраняет более мягкую зависимость между входом и выходом нейрона.

Слои нормализации применяются для стабилизации процесса обучения. В YOLOv8 используется Batch Normalization (BN – нормализация по мини-батчу). Мини-батч – это небольшая группа изображений, которая подаётся в нейронную сеть за один шаг обучения. Например, если обучающая выборка содержит тысячи изображений, модель не обрабатывает их все одновременно, а делит данные на небольшие группы по 8, 16 или 32 изображения. Каждая такая группа называется мини-батчем.

Batch Normalization нормализует значения активаций внутри текущего мини-батча, то есть приводит их к более устойчивому распределению. Это означает, что значения признаков в слоях сети становятся менее разбросанными и более удобными для последующего обучения. Благодаря этому обучение становится стабильнее, модель быстрее сходится, а вероятность резких скачков значений в слоях сети уменьшается.

Проще говоря, Batch Normalization помогает сети обучаться более ровно и предсказуемо, особенно при большом количестве слоёв. Мини-батч в данном случае можно понимать как небольшую порцию обучающих изображений, на основе которой модель делает один шаг обновления своих параметров.

Механизм пулинга (pooling – объединение) используется для уменьшения пространственного размера карт признаков и объединения информации из соседних областей. Это позволяет сети учитывать не только мелкие локальные детали, но и более широкий контекст изображения. В YOLOv8 применяется SPP (Spatial Pyramid Pooling – пространственный пирамидальный пулинг). Данный механизм агрегирует признаки на разных масштабах, что особенно важно для детекции объектов разного размера. Например, пешеход рядом с автомобилем может занимать большую часть кадра, а дорожный знак или пешеходный переход вдали могут быть представлены небольшой областью изображения. SPP помогает сети

учитывать такие различия и использовать информацию как из малых, так и из крупных областей изображения [12].

Для формального описания работы свёрточного слоя, являющегося центральным элементом CNN, приведём математическую запись операции свёртки. Операция свёртки для двумерного входного тензора X с ядром W размером $k \times k$ может быть записана следующим образом (1.2):

$$Y[i,j] = \sum_m \sum_n X[i+m, j+n] \cdot W[m,n] + b, \quad (1.2)$$

где X – входная карта признаков или изображение, W – ядро свёртки, b – обучаемый параметр смещения (bias), $Y[i,j]$ – значение выходной карты признаков в позиции i,j . Суммирование выполняется по всем элементам ядра свёртки [11].

Смысл данной операции состоит в том, что небольшое ядро W последовательно проходит по изображению и для каждого положения вычисляет числовой отклик. Если участок изображения похож на признак, который “ищет” данный фильтр, отклик будет высоким. Если похожего признака нет, отклик будет низким. В процессе обучения значения фильтров W и смещения b автоматически изменяются так, чтобы сеть лучше распознавала нужные объекты. Рассмотрим, каким образом происходит это автоматическое изменение параметров.

Обучение CNN выполняется методом обратного распространения ошибки (backpropagation – обратное распространение ошибки). Сначала модель выполняет прямой проход: получает изображение на входе и формирует предсказания, например координаты ограничивающих прямоугольников и классы объектов. Затем вычисляется функция потерь, показывающая, насколько предсказания отличаются от правильной разметки. После этого ошибка распространяется в обратном направлении по слоям сети, и веса модели корректируются.

В YOLOv8 при обучении может использоваться оптимизатор SGD (Stochastic Gradient Descent – стохастический градиентный спуск) с моментом. Стохастический градиентный спуск изменяет веса модели в направлении

уменьшения ошибки. Библиотека Ultralytics по умолчанию использует SGD с параметром `momentum` (момент) = 0,937 и `weight_decay` (затухание весов) = 0,0005 [13, 14]. Момент определяет, какую долю предыдущего направления обновления весов система сохраняет на текущем шаге: значение 0,937 означает высокую инерцию, что делает процесс обучения более устойчивым и позволяет преодолевать локальные минимумы функции потерь. Параметр `weight_decay` выполняет функцию L2-регуляризации (L2 Regularization – регуляризация второго порядка, также называемая Ridge Regularization – гребневая регуляризация). L2-регуляризация представляет собой метод борьбы с переобучением нейронной сети, при котором к функции потерь добавляется штрафной член, пропорциональный сумме квадратов весов модели. Практически это означает, что на каждом шаге обновления веса модели умножаются на коэффициент $(1 - \text{weight_decay})$, что ограничивает чрезмерный рост отдельных весов и не позволяет модели слишком сильно подстраиваться под шум в обучающих данных. В библиотеке Ultralytics L2-регуляризация применяется по умолчанию, поскольку она не обнуляет веса модели (в отличие от L1-регуляризации, которая может приводить к обнулению части параметров), а лишь ограничивает их величину, что важно для свёрточных сетей, где все фильтры должны участвовать в извлечении признаков. Помимо L2-регуляризации, в машинном обучении существуют и другие методы предотвращения переобучения: L1-регуляризация (штраф за абсолютные значения весов), Dropout (случайное отключение части нейронов при обучении), аугментация данных (искусственное расширение обучающей выборки, описанное в подразделе 3.11). Все эти методы направлены на повышение обобщающей способности модели, однако для задач дообучения YOLO именно L2-регуляризация через `weight_decay` является стандартным и наиболее распространённым подходом. Значение `weight_decay` = 0,0005 является стандартным для задач дообучения моделей YOLO [14].

При дообучении модели в настоящей работе применялась процедура *fine-tuning* (тонкая настройка), при которой уже обученная модель

дополнительно обучается на специализированной выборке. Такой подход позволяет использовать знания, полученные моделью на большом наборе данных, и адаптировать её к более узкой задаче – обнаружению объектов, связанных с зоной пешеходного перехода. Начальная скорость обучения (learning rate) составляла 0,001. Для её изменения в процессе обучения использовалось косинусное расписание (cosine annealing – плавное снижение скорости обучения по косинусоидальному закону). Это позволяет сначала выполнять более заметные обновления весов, а затем постепенно уменьшать шаг обучения, чтобы точнее настроить параметры модели. Подробности дообучения приведены в разделе 3.

1.3 Архитектуры нейронных сетей для детекции объектов

Задача детекции объектов (object detection) заключается в обнаружении объектов на изображении и определении их положения с помощью ограничивающих прямоугольников. Существуют два принципиальных подхода.

Двухстадийные детекторы (например, Faster R-CNN [15]) сначала выделяют области-кандидаты (region proposals – предложения областей), а затем классифицируют каждую из них. Такой подход обеспечивает высокую точность, но работает медленно – порядка 55–80 мс на кадр (данные получены из оригинальной публикации Ren и соавторов [15] и результатов бенчмарков на графическом процессоре (Graphics Processing Unit, GPU) NVIDIA V100, приведённых в обзоре Zhao и соавторов [16], см. также таблицу 1.1). Для задачи обеспечения безопасности пешеходов, требующей обработки в реальном времени (не менее 20 кадров в секунду (Frames Per Second, FPS)), двухстадийные детекторы не подходят.

Одностадийные детекторы выполняют классификацию и определение координат за один проход сети, что обеспечивает значительно более высокую скорость. Семейство YOLO, впервые предложенное Редмоном в 2016 году [17], является наиболее распространённым представителем этого класса. В настоящей работе используется YOLOv8 (Ultralytics, 2023) – версия,

обеспечивающая оптимальное соотношение скорости и точности среди стабильных релизов на момент начала разработки системы.

Выбор YOLOv8, а не более поздних версий (YOLO11, выпущенная в сентябре 2024 г.), обусловлен следующими причинами. Во-первых, на момент начала разработки системы YOLOv8 являлась более зрелой версией с проверенной экосистемой дообучения и экспорта моделей. Во-вторых, согласно данным Ultralytics, YOLO11m превосходит YOLOv8m на наборе данных COCO (Common Objects in Context – набор данных общего назначения для задач компьютерного зрения) всего на 1,3 % по метрике mAP (mean Average Precision – средняя точность, интегральная мера качества детекции), при 22 % меньшем числе параметров [18], однако независимое исследование на бенчмарке RF100-VL показало, что на реальных данных разница между YOLOv8 и YOLO11 практически нивелируется [19]. В-третьих, для задачи обнаружения мелких объектов (пешеходов на удалении) YOLOv8 в ряде случаев превосходит YOLO11 [20].

Архитектура YOLOv8 состоит из трёх частей: backbone (базовая сеть-экстрактор признаков CSPDarknet с блоками C2f), neck (агрегатор многомасштабных признаков Path Aggregation Network (PANet – сеть агрегации путей признаков), сеть агрегации путей) и head (детекционная головка). Блоки C2f объединяют свёртку с остаточными связями (skip connections – связи пропуска), что позволяет эффективно извлекать признаки на разных уровнях абстракции.

В архитектуре YOLOv8 для объединения признаков разных масштабов используется PANet. После обработки изображения базовой сетью формируются карты признаков разных уровней, обычно обозначаемые как P3, P4 и P5. Уровень P3 лучше сохраняет мелкие детали и используется для обнаружения небольших объектов, например удалённых пешеходов, дорожных знаков или светофоров. Уровень P4 применяется для объектов среднего размера, а P5 содержит более обобщённые признаки и подходит для

крупных объектов, например автомобилей или близко расположенных пешеходов.

Объединение признаков разных масштабов позволяет модели устойчиво обнаруживать объекты различного размера в зоне пешеходного перехода. Это важно, поскольку пешеходы, транспортные средства, дорожные знаки и светофоры могут занимать в кадре как большую, так и очень малую область.

В YOLOv8 применяется anchor-free подход, то есть метод детекции без заранее заданных якорных рамок. В этом случае модель не выбирает рамку из набора готовых шаблонов, а напрямую предсказывает границы объекта. Такой подход упрощает настройку модели и делает её более гибкой при обнаружении объектов разной формы и размера.

Финальная часть YOLOv8 использует decoupled head – разделённую детекционную головку. Это означает, что определение класса объекта и предсказание координат ограничивающего прямоугольника выполняются разными ветвями сети. Одна ветвь отвечает за вопрос «что это за объект», а другая – за вопрос «где он находится». Такое разделение повышает точность детекции и локализации объектов, что особенно важно для дальнейшей оценки положения пешехода относительно зоны пешеходного перехода и траектории автомобиля.

Функция потерь CIOU (Complete IoU – полное пересечение по объединению) используется при обучении для оптимизации координат ограничивающих прямоугольников (1.3):

$$L_CIOU = 1 - IoU + \rho^2(b, b_gt) / c^2 + \alpha \cdot v, \quad (1.3)$$

где $\rho(b, b_gt)$ – евклидово расстояние между центрами предсказанного и истинного прямоугольников, c – диагональ наименьшего охватывающего прямоугольника, α и v – параметры, учитывающие соотношение сторон [14]. Для обоснования выбора архитектуры YOLOv8 в таблице 1.1 приведено сравнение основных архитектур детекции объектов по ключевым показателям: точности, скорости обработки и числу параметров.

Таблица 1.1 – Сравнительный анализ архитектур детекции объектов

Архитектура	Тип	mAP@0.5 (COCO)	Скорость, мс/кадр	Параметры, млн
Faster R-CNN	Двухстадийный	0,42	55–80	41,1
SSD300	Одностадийный	0,31	12–15	23,7
RetinaNet	Одностадийный	0,40	20–30	36,4
YOLOv5s	Одностадийный	0,37	4–6	7,2
YOLOv7	Одностадийный	0,51	5–8	36,9
YOLOv8s	Одностадийный	0,45	3–5	11,2
DETR	Трансформерный	0,42	40–60	41,3

Составлено по данным [14, 15, 17, 21, 22]. Как видно из таблицы 1.1, архитектуры семейства YOLO обеспечивают наилучшее соотношение скорости и точности среди рассмотренных подходов. Для более детального выбора конкретной конфигурации модели YOLOv8 в таблице 1.2 приведены характеристики различных вариантов этой архитектуры, отличающихся числом параметров и вычислительной сложностью.

Таблица 1.2 – Характеристики вариантов модели YOLOv8

Модель	Параметры, млн	FLOPs, G	mAP@0.5 (COCO)	Скорость, мс
YOLOv8n	3,2	8,7	0,373	1,8
YOLOv8s	11,2	28,6	0,449	3,4
YOLOv8m	25,9	78,9	0,502	6,2
YOLOv8l	43,7	165,2	0,528	10,1
YOLOv8x	68,2	257,8	0,539	14,7

Составлено по данным [14].

Здесь FLOPs (Floating Point Operations – число операций с плавающей точкой) выражается в гигаоперациях (G) и характеризует вычислительную

сложность модели. Как следует из таблицы 1.2, модели семейства YOLOv8 образуют линейку от наименьшей (nano) до наибольшей (extra-large), в которой увеличение числа параметров повышает точность детекции, но увеличивает время обработки. Для настоящей работы в качестве основной модели общего назначения выбрана модель YOLOv8n (nano – наименьшая) как обеспечивающая наибольшую скорость обработки при приемлемой точности. Для дообучения модели распознавания разметки использована архитектура YOLOv8s (small – малая), обеспечивающая более высокую точность при допустимом увеличении времени обработки.

Важным этапом постобработки результатов детекции является немаксимальное подавление (Non-Maximum Suppression, NMS). Нейронная сеть формирует множество перекрывающихся предсказаний для одного объекта, и NMS отбирает из них наилучшее. Алгоритм NMS работает следующим образом: все предсказания сортируются по убыванию уровня уверенности (confidence – мера достоверности предсказания, от 0 до 1); выбирается предсказание с максимальной уверенностью; удаляются все предсказания с Intersection over Union (IoU – пересечение по объединению) выше порога (в YOLOv8 порог NMS по умолчанию составляет 0,7) [14]; процесс повторяется для оставшихся предсказаний. В YOLOv8 реализован вариант Weighted NMS (взвешенное немаксимальное подавление), где итоговый прямоугольник является взвешенным средним всех подавленных. Данный этап критически важен для корректной работы последующего модуля трекинга: без NMS один и тот же пешеход получил бы несколько ограничивающих прямоугольников, что привело бы к созданию ложных треков.

1.4 Методы многообъектного трекинга

Детекция объектов определяет их положение на отдельном кадре, однако для оценки степени опасности необходимо отслеживать объекты во времени – определять их скорость, направление и траекторию. Эту задачу решает многообъектный трекинг (Multi-Object Tracking, MOT) – отслеживание

множества объектов на видеопоследовательности с присвоением каждому уникального идентификатора [23, 24]. Без трекинга система не могла бы вычислить скорость приближения объекта и, следовательно, время до столкновения (ТТС), являющееся ключевым критерием принятия решения о торможении.

Классический подход к трекингу основывается на парадигме «обнаружение – ассоциация» (tracking-by-detection): на каждом кадре выполняется детекция объектов, после чего обнаруженные объекты ассоциируются с существующими треками из предыдущих кадров. Ассоциация может осуществляться на основе различных метрик: пространственной близости (IoU ограничивающих прямоугольников), визуального сходства (извлечённые признаки внешнего вида), кинематических предсказаний (предсказанное положение на основе модели движения) [24].

Алгоритм простого трекинга в реальном времени Simple Online and Realtime Tracking (SORT), предложенный Бьюли и соавторами в 2016 году, использует фильтр Калмана для предсказания положения объекта в следующем кадре и венгерский алгоритм (Hungarian algorithm – метод оптимального назначения) для оптимального сопоставления обнаруженных объектов с существующими треками на основе метрики IoU. Алгоритм характеризуется высокой скоростью работы (до 260 FPS), однако подвержен ошибкам переназначения идентификаторов (ID switches – переключения идентификаторов) при кратковременном перекрытии объектов [25].

Алгоритм DeepSORT расширяет SORT путём добавления визуальных признаков внешнего вида объектов, извлекаемых отдельной нейронной сетью (Re-ID модель – модель повторной идентификации). Это позволяет существенно снизить количество переназначений идентификаторов и повысить устойчивость трекинга при частичных перекрытиях [26]. Современные алгоритмы (ByteTrack, OC-SORT, BoT-SORT) развивают эти идеи и дополнительно используют ассоциацию объектов с низкой

уверенностью детекции [27]. Количественное сравнение перечисленных алгоритмов трекинга по основным метрикам качества представлено в таблице 1.3.

Таблица 1.3 – Сравнительный анализ алгоритмов многообъектного трекинга

Алгоритм	MOTA, %	IDF1, %	ID Sw.	FPS
SORT	59,8	61,4	1423	260
DeepSORT	61,4	71,8	781	40
ByteTrack	77,8	77,3	2196	30
OC-SORT	63,2	77,5	942	50
BoT-SORT	77,8	77,5	1257	25

Данные таблицы получены из оригинальных публикаций авторов алгоритмов: SORT – [25], DeepSORT – [26], ByteTrack – [27], OC-SORT – [28], BoT-SORT – [29]. Результаты приведены для бенчмарка MOT17. MOT17 (Multi-Object Tracking Benchmark 2017) – стандартный набор данных для оценки алгоритмов многообъектного трекинга, включающий видеопоследовательности пешеходов в различных условиях съёмки (стационарная и подвижная камера, различная плотность толпы, разное освещение). Набор содержит 14 последовательностей с ручной аннотацией координат и идентификаторов пешеходов, является общепринятым эталоном для сравнения алгоритмов трекинга в научных публикациях [25, 26, 27].

Здесь MOTA (Multiple Object Tracking Accuracy – точность многообъектного трекинга) – интегральная метрика, учитывающая пропуски (FN – False Negatives, то есть необнаруженные объекты), ложные срабатывания (FP – False Positives, то есть ложно обнаруженные объекты) и переключения идентификаторов (ID Sw. – ID Switches). IDF1 оценивает способность трекера правильно ассоциировать детекции с уникальными идентификаторами на протяжении всей последовательности.

В реализованной системе используется подход, аналогичный SORT: на каждом кадре нейронная сеть формирует множество детекций, фильтр

Калмана предсказывает положения существующих треков, а венгерский алгоритм находит оптимальное назначение детекций к трекам, максимизирующее суммарное IoU.

Для количественной оценки качества трекинга используются стандартные метрики MOTChallenge (1.4):

$$\text{MOTA} = 1 - (\text{FN} + \text{FP} + \text{IDSW}) / \text{GT}, \quad (1.4)$$

где GT – общее число эталонных объектов на всех кадрах.

А также метрика IDF1, характеризующая качество сохранения идентификаторов, вычисляется по формуле (1.5):

$$\text{IDF1} = 2 \cdot \text{IDP} \cdot \text{IDR} / (\text{IDP} + \text{IDR}), \quad (1.5)$$

где IDP – точность идентификации, IDR – полнота идентификации. Выбор MOTA и IDF1 обусловлен тем, что MOTA оценивает общее качество (учёт пропусков, ложных срабатываний и переключений ID), а IDF1 – качество сохранения идентификаторов, что критично для корректного расчёта TTC.

1.5 Оценка расстояния по данным монокулярной камеры

Оценка расстояния от автомобиля до обнаруженных объектов является необходимым компонентом системы принятия решений. При использовании единственной камеры прямое измерение глубины невозможно, поэтому применяются косвенные методы [30].

В настоящей работе используется метод оценки расстояния по известным размерам объекта, основанный на модели камеры-обскуры (pinhole camera model). Данная модель является упрощённым математическим описанием работы камеры. В ней предполагается, что свет от объектов проходит через одну точку – оптический центр камеры – и затем проецируется на плоскость изображения. В результате трёхмерный объект реального мира отображается на двумерном изображении. Проще говоря, pinhole camera model описывает, как реальный объект становится изображением на кадре. Чем ближе объект находится к камере, тем больше он выглядит на изображении. Чем дальше объект расположен, тем меньше его размер в кадре. Поэтому, если известен реальный размер объекта, например средний рост взрослого

пешехода, можно приблизительно оценить расстояние до него по размеру его ограничивающего прямоугольника на изображении.

В данной работе в качестве известного размера используется средний рост взрослого пешехода, принимаемый равным приблизительно 1,70 м. После обнаружения пешехода нейросетевой моделью вокруг него строится ограничивающий прямоугольник. Далее измеряется высота этого прямоугольника в пикселях. Если пешеход на изображении имеет большую высоту, это означает, что он находится ближе к камере. Если высота прямоугольника мала, объект находится дальше.

Оценка расстояния выполняется по выражению (1.6):

$$D = (H_{\text{реал}} \cdot f) / h_{\text{пикс}}, \quad (1.6)$$

где D – расстояние до объекта в метрах, $H_{\text{реал}}$ – известная высота объекта в метрах, f – фокусное расстояние камеры в пикселях, $h_{\text{пикс}}$ – высота ограничивающего прямоугольника объекта в пикселях [31]. Точность метода зависит от калибровки камеры и вариативности размеров объектов, однако для задачи предупреждения столкновений обеспечивает достаточную точность – погрешность оценки расстояния составляет порядка 10–15 % на дистанциях до 30 м [30, 31]. Данная оценка является типичной для метода pinhole camera с единственным известным размером объекта и подтверждается следующим расчётом. В качестве опорной высоты принято значение $H = 1,70$ м (средний рост взрослого пешехода), тогда как реальный рост пешеходов варьируется от приблизительно 1,50 м до 1,90 м. Если реальный рост составляет 1,50 м, а система использует $H = 1,70$ м, то расстояние будет завышено с относительной ошибкой $\delta = (1,70/1,50 - 1) \approx 13,3$ %. Если реальный рост составляет 1,90 м, расстояние будет занижено с ошибкой $\delta = (1 - 1,70/1,90) \approx 10,5$ %. Таким образом, вариативность роста пешеходов приводит к погрешности оценки расстояния в диапазоне 10–15 %, что согласуется с результатами исследований в области монокулярной оценки глубины [30, 31].

Таким образом, модель камеры-обскуры позволяет связать реальный размер объекта, его размер на изображении и расстояние до камеры. В рамках

настоящей работы данный метод используется как приближённая оценка расстояния до пешехода, необходимая для дальнейшего расчёта времени до потенциального столкновения и принятия решения о снижении скорости или торможении автомобиля.

1.6 Анализ наборов данных для обучения и тестирования

Качество обучения нейросетевых моделей напрямую зависит от характеристик используемых наборов данных. Рассмотрим основные наборы данных с точки зрения их применимости для настоящей работы.

COCO (Common Objects in Context – распространённые объекты в контексте) – крупнейший набор данных общего назначения для задач детекции, содержащий более 330 тысяч изображений с аннотациями для 80 классов объектов. Класс «person» (человек) содержит более 260 тысяч аннотированных экземпляров. Основным ограничением является отсутствие аннотаций для российских дорожных знаков и разметки [32, 33].

RTSD (Russian Traffic Sign Dataset – набор данных российских дорожных знаков) – единственный крупный набор данных российских дорожных знаков, содержащий 104 358 кадров с 156 классами дорожных знаков. RTSД является ключевым ресурсом для обучения детектора дорожных знаков, поскольку содержит аннотации для знаков 5.19.1, 5.19.2 и 1.22 [6].

Сводное сравнение рассмотренных наборов данных по ключевым характеристикам приведено в таблице 1.4.

Таблица 1.4 – Сравнительный анализ наборов данных

Набор данных	Объём	Пешеходы	Знаки РФ	Условия
COCO	330K изобр.	Да (260K)	Нет	Разнообразные
KITTI	7,5K изобр.	Да (4,5K)	Нет	Город (Германия)
BDD100K	100K видео	Да	Нет	Город (США)
RTSD	104K кадров	Нет	Да (156 кл.)	Город (Россия)
FPVCrosswalk	12K кадров	Нет	Нет	Переходы

Данные таблицы получены из официальных описаний наборов данных: COCO – [32], KITTI – [34], BDD100K – [35], RTSD – [6], FPVCrosswalk – [36].

Анализ показывает, что ни один из существующих наборов данных не содержит одновременно аннотации для пешеходов, российских дорожных знаков и разметки пешеходных переходов. Это обосновывает выбранный подход с несколькими детекторами на базе YOLOv8, каждый из которых дообучен на соответствующем наборе данных.

1.7 Выводы по разделу 1

Проведённый обзор показал, что для создания эффективной системы идентификации подвижных объектов в зоне пешеходных переходов целесообразно использовать комбинацию следующих подходов: одностадийный нейросетевой детектор YOLOv8 для обнаружения и классификации объектов в реальном времени; алгоритм многообъектного трекинга на основе IoU-ассоциации с фильтром Калмана; геометрический метод оценки расстояния на основе модели pinhole camera; критерий TTC для принятия решений о необходимости торможения. Данная комбинация методов обеспечивает баланс между точностью, вычислительной эффективностью и работой в реальном времени. Выбранные методы формализованы в математической модели, представленной в разделе 2.

2 Математическая модель системы идентификации подвижных объектов

В настоящем разделе представлена математическая модель, формализующая основные процессы, протекающие в разрабатываемой системе. Математическая формализация необходима для обоснования выбора алгоритмов, определения их параметров и анализа поведения системы в граничных условиях. Представленная модель основана на классических методах теории оценивания [37, 38], кинематики движения объектов [39, 40] и критериях безопасности систем ADAS [41]. Каждый подраздел модели соответствует конкретному программному модулю, описанному в разделе 3: кинематическая модель определяет структуру фильтра Калмана в модуле трекинга, формулы расчёта тормозного пути используются в модуле оценки риска, а критерий TTC – в модуле принятия решений.

2.1 Кинематическая модель движения объекта

Движение подвижного объекта (пешехода, велосипедиста, пользователя средства индивидуальной мобильности) в плоскости дорожного полотна описывается двумерной моделью с постоянной скоростью (constant velocity model). Ось Ox направлена вдоль дороги по ходу движения автомобиля, ось Oy – поперёк дороги. Начало координат совмещено с текущим положением автомобиля.

Вектор состояния объекта включает координаты положения и компоненты скорости (2.1):

$$x(t) = [x(t), y(t), v_x(t), v_y(t)]^T, \quad (2.1)$$

где $x(t), y(t)$ – координаты положения объекта в плоскости дорожного полотна; $v_x(t), v_y(t)$ – компоненты скорости объекта вдоль и поперёк дороги соответственно.

Матрица перехода состояния F для модели с постоянной скоростью имеет вид (2.2):

$$F = [[1, 0, \Delta t, 0], [0, 1, 0, \Delta t], [0, 0, 0, 1], [0, 0, 0, 1]], \quad (2.2)$$

где F – матрица перехода состояния размером 4×4 ; Δt – интервал дискретизации (период между кадрами), с; элементы матрицы описывают линейную связь между положением и скоростью объекта: новая координата равна сумме предыдущей координаты и произведения скорости на Δt .

При частоте кадров видеопотока 30 кадров в секунду интервал дискретизации составляет $\Delta t \approx 0,033$ с. Характерные кинематические параметры различных категорий подвижных объектов приведены в таблице 2.1.

Таблица 2.1 – Кинематические параметры движения различных категорий объектов

Категория объекта	Скорость, м/с	Ускорение, м/с ²	Характер движения
Взрослый пешеход	1,2–1,5	0,3–0,8	Равномерное, предсказуемое
Ребёнок (до 12 лет)	1,0–2,5	0,5–1,5	Неравномерное, импульсивное
Пожилой человек	0,8–1,1	0,2–0,5	Медленное, равномерное
Велосипедист	3,0–6,0	0,5–1,5	Равномерное, прямолинейное
Электросамокат	5,0–15,0	1,0–3,0	Быстрое, менее предсказуемое

Значения скорости и ускорения пешеходов приведены на основании данных [40, 42]. Параметры движения велосипедистов и пользователей электросамокатов приняты по результатам обзора [1, 40].

2.2 Фильтр Калмана для оценки состояния объекта

Измерения координат объектов, получаемые из результатов нейросетевой детекции, содержат шум. Для формирования сглаженных и устойчивых оценок положения и скорости объектов применяется линейный фильтр Калмана (Kalman filter – рекурсивный алгоритм оптимальной линейной фильтрации) [37], работающий в два этапа – предсказание (prediction) и коррекция (update).

Этап предсказания: на данном этапе выполняется прогноз вектора состояния объекта и ковариационной матрицы ошибки на основе модели

движения. Предсказанный вектор состояния определяется по формуле (2.3), а ковариационная матрица ошибки предсказания – по формуле (2.4):

$$\hat{x}(k|k-1) = F \cdot \hat{x}(k-1|k-1), \quad (2.3)$$

$$P(k|k-1) = F \cdot P(k-1|k-1) \cdot F^T + Q, \quad (2.4)$$

где $\hat{x}(k|k-1)$ – предсказанный вектор состояния на шаге k по данным до шага $k-1$; F – матрица перехода состояния, определённая формулой (2.2); $P(k|k-1)$ – ковариационная матрица ошибки предсказания; Q – ковариационная матрица шума процесса, описывающая непредсказуемость движения объекта.

Этап коррекции: на данном этапе выполняется уточнение оценки состояния по результатам нового измерения. Матрица усиления Калмана вычисляется по формуле (2.5), скорректированный вектор состояния – по формуле (2.6), а обновлённая ковариационная матрица – по формуле (2.7):

$$K(k) = P(k|k-1) \cdot H^T \cdot [H \cdot P(k|k-1) \cdot H^T + R]^{-1}, \quad (2.5)$$

$$\hat{x}(k|k) = \hat{x}(k|k-1) + K(k) \cdot [z(k) - H \cdot \hat{x}(k|k-1)], \quad (2.6)$$

$$P(k|k) = [I - K(k) \cdot H] \cdot P(k|k-1), \quad (2.7)$$

где $K(k)$ – матрица усиления Калмана, определяющая баланс доверия между предсказанием и измерением; H – матрица наблюдения, связывающая вектор состояния с измеряемыми величинами; R – ковариационная матрица шума измерений; $z(k)$ – вектор измерений от детектора; I – единичная матрица.

Формулы (2.3)–(2.7) представляют собой классические уравнения линейного фильтра Калмана [37, 38]. В этих формулах используются следующие переменные: $\hat{x}(k|k-1)$ – предсказанный вектор состояния на шаге k по данным до шага $k-1$, содержащий координаты (x , y) и скорости (v_x , v_y) объекта; F – матрица перехода, определённая формулой (2.2), описывающая эволюцию состояния за один кадр по принципу: новая позиция = старая позиция + скорость $\times \Delta t$; $P(k|k-1)$ – ковариационная матрица ошибки предсказания, характеризующая степень неопределённости предсказанного состояния; Q – ковариационная матрица шума процесса, описывающая непредсказуемость движения объекта (ускорение, остановка, изменение направления); $K(k)$ – матрица усиления Калмана (Kalman gain), определяющая

баланс доверия между предсказанием и измерением: при $K \approx 1$ система доверяет измерению, при $K \approx 0$ – предсказанию; H – матрица наблюдения, связывающая вектор состояния с измеряемыми величинами; поскольку измеряются только координаты (x, y), но не скорости, $H = [[0, 0, 0, 1], [0, 0, 0, 1]]$; R – ковариационная матрица шума измерений, характеризующая зашумлённость координат, получаемых от нейронной сети; $z(k)$ – вектор измерений, содержащий координаты ограничивающего прямоугольника от детектора YOLOv8; I – единичная матрица. Матрица Q формируется на основе модели случайного ускорения (2.8):

$$Q = \sigma_a^2 \cdot G \cdot G^T, \quad (2.8)$$

где $\sigma_a = 2,0 \text{ м/с}^2$ – стандартное отклонение ускорения пешехода. Формула (2.8) представляет стандартную модель случайного ускорения из [38].

Применение фильтра Калмана обеспечивает: сглаживание шума в координатах; получение устойчивых оценок скорости, необходимых для расчёта TTC; компенсацию кратковременной потери объекта из поля зрения (на этапе предсказания координаты экстраполируются по текущей оценке скорости).

2.3 Расчёт расстояния до объекта и тормозного пути автомобиля

Тормозной путь автомобиля при равномерном замедлении определяется формулой (2.9):

$$S_{\text{торм}} = v^2 / (2 \cdot \mu \cdot g), \quad (2.9)$$

где v – скорость автомобиля, м/с; μ – коэффициент сцепления колёс с дорожным покрытием; g – ускорение свободного падения ($9,81 \text{ м/с}^2$).

Полный остановочный путь включает также расстояние, пройденное за время реакции (2.10):

$$S_{\text{ост}} = v \cdot t_{\text{реакц}} + v^2 / (2 \cdot \mu \cdot g), \quad (2.10)$$

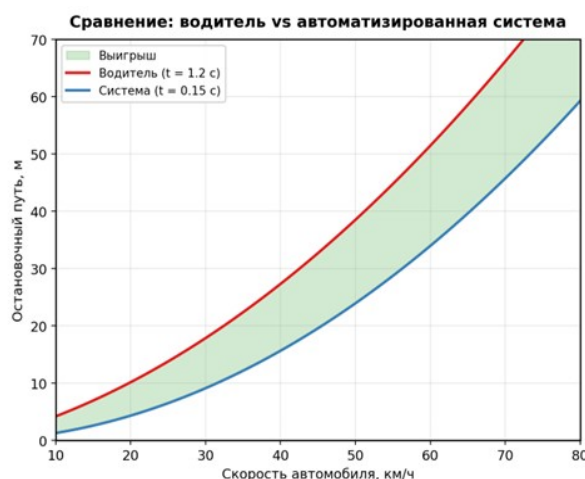
где v – скорость автомобиля, м/с; μ – коэффициент сцепления колёс с дорожным покрытием (безразмерный); g – ускорение свободного падения ($9,81 \text{ м/с}^2$); $t_{\text{реакц}}$ – время реакции водителя или системы, с; $S_{\text{торм}}$ –

тормозной путь автомобиля, м; $S_{\text{ост}}$ – полный остановочный путь, м. Значения коэффициента сцепления для различных типов дорожного покрытия приведены в таблице 2.2.

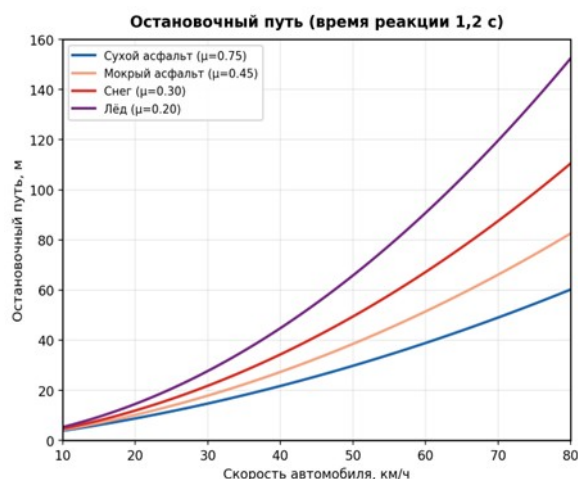
Таблица 2.2 – Коэффициенты сцепления для различных дорожных покрытий

Тип покрытия	Коэффициент μ	$S_{\text{торм}}$ при 50 км/ч, м
Сухой асфальт	0,70–0,80	13,8–11,9
Мокрый асфальт	0,40–0,50	24,2–19,3
Снежное покрытие	0,25–0,35	38,7–27,6
Обледенелое покрытие	0,15–0,25	64,5–38,7

Для водителя-человека время реакции составляет порядка 1,0–1,5 с [8, 40]. Для автоматизированной системы это время сокращается до 0,1–0,3 с. Данная оценка обусловлена тем, что автоматизированная система не имеет физиологической задержки восприятия и принятия решения, характерной для водителя-человека. Время реакции системы складывается из нескольких компонентов: время инференса нейронной сети (порядка 20–25 мс для двух моделей YOLOv8), время выполнения трекинга и вычисления TTC (около 2–3 мс), а также необходимость подтверждения детекции на нескольких последовательных кадрах для исключения ложных срабатываний (2–3 кадра по 33 мс каждый при частоте 30 FPS). Суммарная задержка составляет порядка 90–130 мс, что округлённо соответствует диапазону 0,1–0,3 с в зависимости от сложности сцены и числа кадров подтверждения. Аналогичные значения приводятся в [40, 41] для современных систем АЕВ. Например, при скорости 50 км/ч и $\mu = 0,7$ на сухом асфальте остановочный путь для водителя составляет около 31 м (16,7 м за время реакции + 14,0 м тормозного пути), а для автоматизированной системы – около 17 м (2,8 м + 14,0 м), что даёт выигрыш порядка 14 м [40]. Графическая иллюстрация зависимости остановочного пути от скорости и выигрыша при использовании автоматизированной системы представлена на рисунке 2.1.



а)



б)

Рисунок 2.1 – Модель расчёта тормозного пути транспортного средства
а – зависимость остановочного пути от скорости при различных дорожных покрытиях; б – сравнение остановочного пути водителя и автоматизированной системы

2.4 Критерий времени до столкновения

Время до столкновения (Time-To-Collision, TTC) является ключевым критерием оценки степени опасности дорожной ситуации в системах ADAS. TTC определяет, через какое время произойдёт столкновение при сохранении текущих параметров движения автомобиля и объекта.

В простейшем одномерном случае, когда автомобиль и объект движутся вдоль одной оси, TTC вычисляется как (2.11):

$$TTC = d_0 / v_{\text{реал}}, \quad (2.11)$$

где d_0 – текущая дистанция от автомобиля до объекта (м), определяемая по формуле (2.8); $v_{\text{реал}}$ – относительная скорость сближения (м/с), вычисляемая как (2.12):

$$v_{\text{реал}} = v_{\text{авто}} - v_{\text{объекта}}, \quad (2.12)$$

где $v_{\text{авто}}$ – скорость автомобиля (м/с), $v_{\text{объекта}}$ – продольная составляющая скорости объекта (м/с), оцениваемая по разности координат центра ограничивающего прямоугольника между последовательными кадрами с учётом масштабного коэффициента расстояния. Если объект движется навстречу автомобилю, $v_{\text{объекта}} < 0$ и относительная скорость возрастает.

Если объект удаляется ($v_{\text{объекта}} > v_{\text{авто}}$), ТТС принимает отрицательное значение, что означает отсутствие угрозы столкновения.

Формула (2.11) применима, когда объект находится непосредственно на траектории автомобиля. Однако в реальных условиях пешеход может пересекать проезжую часть поперёк движения. Для учёта этого случая вводится обобщённая двумерная оценка ТТС. Рассмотрим относительное перемещение объекта в системе координат автомобиля: ось X направлена вдоль движения автомобиля, ось Y – поперёк. ТТС вычисляется отдельно по каждой оси (2.13), (2.14):

$$TTC_x = \Delta x / v_{\text{реал_x}}, \quad (2.13)$$

$$TTC_y = \Delta y / v_{\text{реал_y}}, \quad (2.14)$$

где Δx , Δy – компоненты расстояния от автомобиля до объекта вдоль и поперёк оси движения; $v_{\text{реал_x}}$, $v_{\text{реал_y}}$ – соответствующие компоненты относительной скорости.

Столкновение возможно только в том случае, когда траектории автомобиля и объекта пересекаются приблизительно в одно и то же время по обеим осям. Формально это условие записывается как (2.15):

$$TTC_2D = \min(TTC_x, TTC_y) \text{ при } |TTC_x - TTC_y| < \epsilon, \quad (2.15)$$

где ϵ – пороговое значение рассогласования (в данной работе $\epsilon = 0,5$ с). Если разность $|TTC_x - TTC_y| > \epsilon$, траектории расходятся и угроза столкновения отсутствует. Значение $\epsilon = 0,5$ с выбрано на основе анализа типичных скоростей пересечения пешеходных переходов (1,0–1,5 м/с): при ширине полосы движения 3–3,5 м время нахождения пешехода в зоне потенциального конфликта составляет 2–3,5 с, а допустимая погрешность совпадения моментов пересечения определяется длиной автомобиля ($\approx 4,5$ м) и скоростью пешехода.

Условие активации торможения объединяет критерий ТТС с условием достаточности дистанции для остановки (2.16):

$$TTC < TTC_{\text{крит}} \wedge d(t) < S_{\text{ост}} + d_{\text{запас}}, \quad (2.16)$$

где $TTC_{\text{крит}}$ – пороговое значение времени до столкновения (в данной работе $TTC_{\text{крит}} = 3,0$ с для плавного торможения и $TTC_{\text{крит}} = 1,5$ с для экстренного); $d(t)$ – текущее расстояние до объекта; $S_{\text{ост}}$ – полный остановочный путь, определяемый по формуле (2.10); $d_{\text{запас}}$ – дополнительная дистанция безопасности (в данной работе $d_{\text{запас}} = 2$ м). Таким образом, система формирует команду торможения при одновременном выполнении двух условий: время до столкновения менее критического порога и текущая дистанция недостаточна для гарантированной остановки с учётом запаса безопасности. Данный двойной критерий снижает вероятность ложных срабатываний в ситуациях, когда TTC мал, но объект находится за пределами тормозного пути.

2.5 Модель принятия решений

Логика принятия решений реализуется в виде системы пороговых правил с тремя уровнями реагирования. Каскадная структура обеспечивает градуированный отклик системы, пропорциональный степени опасности: от информирования водителя на дальних дистанциях до автономного экстренного торможения при непосредственной угрозе столкновения.

Уровень 1 (информирование): $TTC < 4$ с – система отображает предупреждение о наличии объекта в зоне перехода. На дисплее автомобиля – интерфейсе человек-машина (Human-Machine Interface, HMI) – выводится соответствующая индикация.

Уровень 2 (превентивное торможение): $TTC < 2,5$ с – система инициирует плавное снижение скорости с замедлением $2\text{--}3$ м/с².

Уровень 3 (экстренное торможение): $TTC < 1,5$ с или $d(t) \leq S_{\text{ост}}$ – система инициирует максимально интенсивное торможение с замедлением $8\text{--}10$ м/с².

Дополнительно предусмотрен особый режим: если зона пешеходного перехода обнаружена, но пешеходов в поле зрения нет, система может формировать команду превентивного плавного снижения скорости. Такая логика рассматривается как дополнительная мера безопасности для случаев,

когда пешеход в пределах зоны перехода может быть временно перекрыт припаркованным транспортным средством, элементами инфраструктуры или другими участниками движения. Пороговые значения могут адаптироваться в зависимости от условий наблюдения, типа объекта и скорости автомобиля.

2.6 Выводы по разделу 2

В данном разделе разработана математическая модель системы, включающая: кинематическую модель движения объектов, алгоритм фильтрации Калмана, формулы расчёта расстояния и остановочного пути, критерий ТТС, многоуровневую модель принятия решений. Полученные математические соотношения легли в основу программной реализации системы, описанной в разделе 3.

3 Разработка программного обеспечения системы идентификации

В настоящем разделе описана архитектура разработанного программного обеспечения, принцип действия системы как замкнутого контура управления, реализация основных модулей, а также процесс сбора и подготовки данных. Программная реализация непосредственно следует из математической модели (раздел 2): каждая формула реализована в соответствующем программном модуле, что обеспечивает прослеживаемость от математического описания до исходного кода.

3.1 Принцип действия системы

Разработанная система реализуется как программный конвейер (pipeline – последовательность этапов обработки) видеопотока, представляющий собой иерархически организованный замкнутый контур управления. Замкнутость контура означает, что воздействие системы на динамику автомобиля (торможение) изменяет дорожную обстановку, которая вновь фиксируется камерой, образуя непрерывный цикл мониторинга и реагирования.

3.1.1 Сенсорный уровень

Сенсорный уровень обеспечивает первичный сбор информации. Используется фронтальная видеокамера автомобиля с разрешением не ниже 1280×720 пикселей и частотой кадров 24–30 FPS. При входном разрешении 4K (3840×2160) выполняется масштабирование до 1280×720 для обеспечения требуемой скорости обработки. Данное разрешение выбрано как компромисс между качеством детекции и скоростью обработки.

3.1.2 Уровень детекции объектов

Модуль детекции реализован на основе нейросетевой архитектуры YOLOv8. Используется подход с ансамблем (ensemble – совокупность моделей) из двух специализированных детекторов: модель общего назначения YOLOv8n, предобученная на COCO, для обнаружения участников дорожного движения (пешеходы, автомобили, велосипедисты, автобусы, грузовики, светофоры); специализированная модель YOLOv8s для обнаружения разметки пешеходного перехода (crosswalk – пешеходный переход), дообученная на

наборе данных FPVCrosswalk2025 и дополнительных данных платформы Roboflow. Процедура инференса (inference – прямой проход модели для получения предсказаний) включает: масштабирование кадра до 640×640 пикселей, нормализацию значений, прямой проход через нейронную сеть, немаксимальное подавление и фильтрацию по порогу уверенности (confidence threshold = 0,30).

3.1.3 Контекстный анализатор

Контекстный анализатор определяет тип пешеходного перехода на основе результатов детекции. При обнаружении светофора (класс traffic_light – светофор) с достаточной уверенностью переход классифицируется как регулируемый. При обнаружении только знака или разметки перехода без светофора – как нерегулируемый. Дополнительно реализован фильтр ложных светофоров: анализируются пропорции ограничивающего прямоугольника обнаруженного объекта. Реальный светофор характеризуется вытянутостью по вертикали (aspect ratio – соотношение сторон значительно превышает единицу). Если пропорции не соответствуют ожидаемым – детекция отклоняется как ложноположительная (false positive – ложное срабатывание).

3.1.4 Уровень трекинга

Модуль трекинга осуществляет отслеживание обнаруженных объектов от кадра к кадру с присвоением уникальных идентификаторов. Реализация основана на фильтре Калмана для предсказания положения объектов (формулы 2.3–2.7) и венгерском алгоритме для оптимального сопоставления детекций с существующими треками (tracks – отслеживаемые объекты) по метрике IoU. Каждому объекту присваивается уникальный идентификатор, формируется траектория движения, оценивается скорость.

3.1.5 Уровень оценки расстояния

Оценка расстояния от автомобиля до обнаруженных объектов выполняется по модели проективной камеры (pinhole camera model). При известном фокусном расстоянии камеры $f = 800$ пикселей и предполагаемой

высоте пешехода $H = 1,70$ м расстояние вычисляется в соответствии с формулой (1.6).

3.1.6 Уровень принятия решений

Модуль принятия решений вычисляет время до столкновения (ТТС) по формуле (2.11) и формирует управляющее воздействие по каскадной логике, описанной в подразделе 2.5. Входными данными являются: расстояние до ближайшего пешехода D , время до столкновения ТТС, относительная скорость сближения $v_{\text{реал}}$ и расчётный остановочный путь $S_{\text{ост}}$.

3.2 Общая архитектура системы и структурная схема подсистем

Структурная схема взаимодействия подсистем представлена на рисунке 3.1, а детализированная структурная схема системы – на рисунке 3.2.

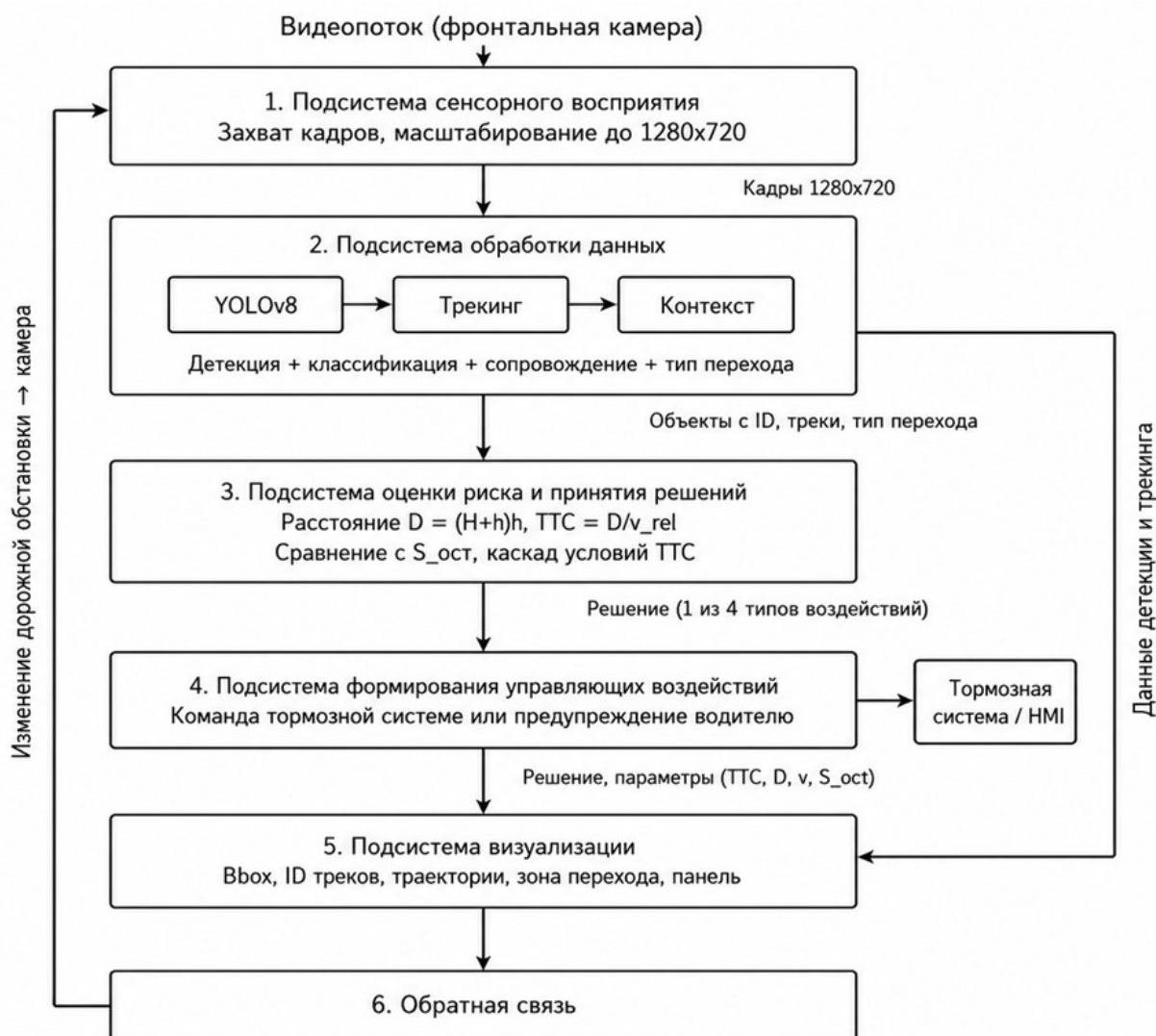


Рисунок 3.1 – Структурная схема связи подсистем

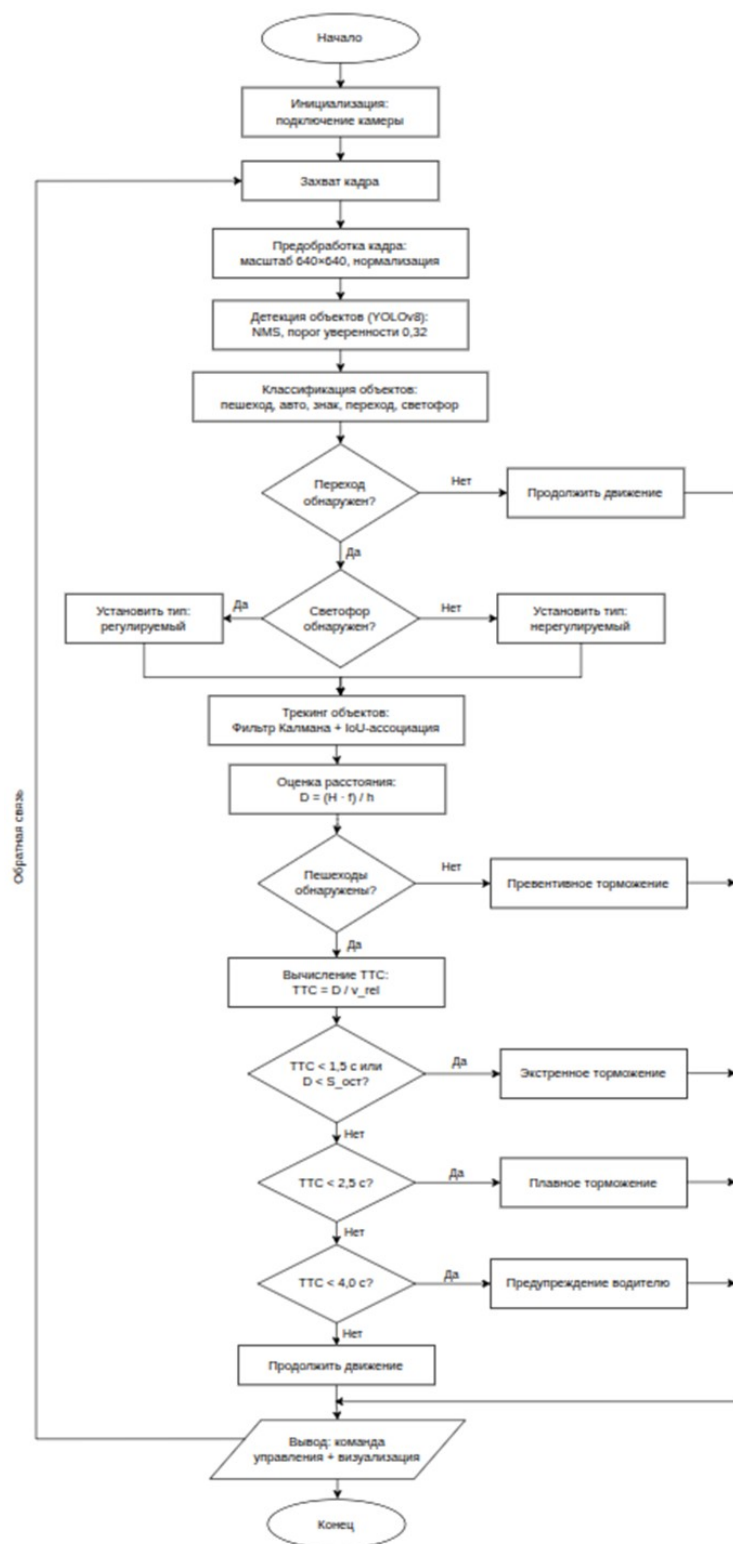


Рисунок 3.2 – Структурная схема системы автоматической идентификации подвижных объектов в зоне пешеходных переходов

Архитектура системы представляет собой замкнутый контур управления, состоящий из шести подсистем, последовательно обрабатывающих информацию от сенсорного входа до формирования управляющих воздействий.

Подсистема 1 – сенсорное восприятие. Принимает на вход видеопоток с фронтальной камеры автомобиля и формирует на выходе последовательность кадров рабочего разрешения 1280×720 пикселей.

Подсистема 2 – обработка данных. Получает на вход масштабированные кадры и включает три взаимосвязанных модуля: нейросетевой детектор YOLOv8, модуль трекинга и контекстный анализатор.

Подсистема 3 – оценка риска и принятие решений. Выполняет вычисление расстояния, расчёт ТТС, сравнение с остановочным путём и формирует управляющие решения.

Подсистема 4 – формирование управляющих воздействий. Преобразует решение в команду для тормозной системы или формирует предупреждение водителю через интерфейс HMI.

Подсистема 5 – визуализация. Агрегирует данные от всех подсистем и отображает информационные слои на видеокадре.

Подсистема 6 – обратная связь. Замыкает контур управления: воздействие на динамику автомобиля изменяет дорожную обстановку, которая вновь фиксируется камерой.

Модульность системы обеспечивает независимость подсистем друг от друга: каждый модуль может быть заменён или модифицирован без необходимости переработки остальных компонентов. Замкнутость контура управления гарантирует непрерывный мониторинг дорожной обстановки: после каждого управляющего воздействия система продолжает отслеживать изменения и корректирует свои решения в соответствии с актуальным состоянием окружающей среды. Такой подход соответствует принципам построения систем реального времени, применяемым в автомобильной промышленности.

3.3 Диаграмма потока данных при обработке видеопотока

Для формального описания информационных потоков в системе построена диаграмма потоков данных (Data Flow Diagram, DFD) уровня 1 в нотации Йордана – Де Марко (рисунок 3.3). DFD позволяет визуализировать,

какие данные и в каком виде передаются между процессами, что важно для понимания зависимостей между модулями и для последующей отладки системы.

Внешняя сущность «Камера» является источником видеопотока, поступающего на вход процесса 1.0 «Захват и предобработка кадра». Результатом процесса 2.0 «Детекция и классификация» являются наборы детекций, которые разветвляются на два параллельных потока: к процессу 3.0 «Контекстный анализ» и к процессу 4.0 «Трекинг объектов». Выходы процессов 3.0 и 4.0 сходятся на входе процесса 5.0 «Оценка расстояния и вычисление TTC», который формирует количественные оценки, поступающие в процесс 6.0 «Принятие решений». Выходы процессов 6.0 и 8.0 «Визуализация результатов» сходятся на входе процесса 7.0 «Формирование управл. воздействий», который формирует команды, поступающие в процесс 8.0 «Визуализация результатов».

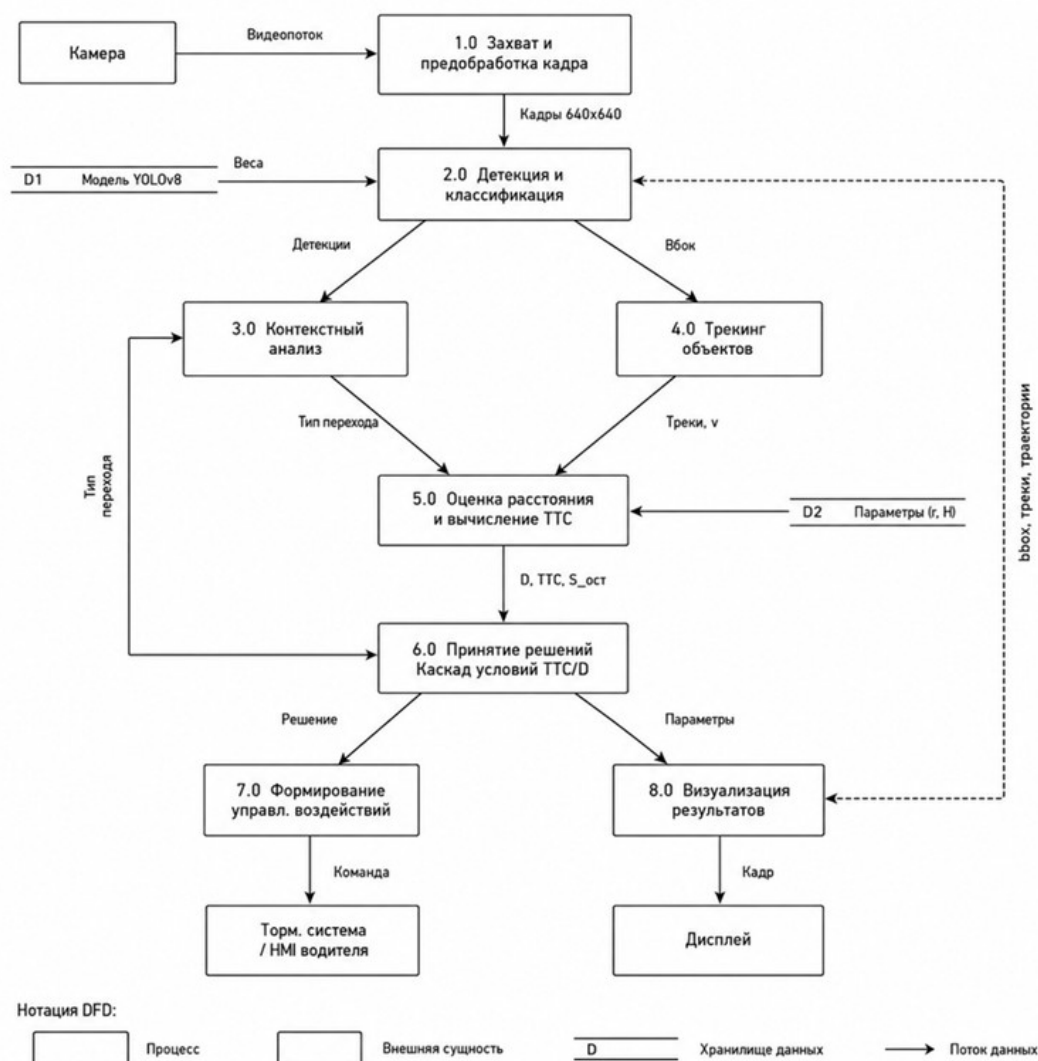


Рисунок 3.3 – Диаграмма потока данных при обработке видеопотока (DFD уровня 1)

3.4 Выбор инструментов и технологий

Программная реализация системы выполнена на языке программирования Python (версия 3.10), что обусловлено наличием развитой экосистемы библиотек для машинного обучения и компьютерного зрения. В качестве среды разработки и проведения экспериментов использована облачная платформа Google Colab с GPU NVIDIA T4. Перечень основных используемых библиотек приведён в таблице 3.1.

Таблица 3.1 – Перечень основных используемых библиотек

Библиотека	Версия	Назначение
PyTorch	2.1.0	Фреймворк глубокого обучения, GPU-ускорение
Ultralytics YOLO	8.1.0	Детекция объектов (YOLOv8)
OpenCV	4.8.1	Обработка изображений и видеопотоков
NumPy	1.26.0	Вычисления над массивами и матрицами
SciPy	1.11.4	Венгерский алгоритм (linear_sum_assignment)
Matplotlib	3.8.2	Визуализация результатов экспериментов

3.5 Алгоритм контекстно-зависимой обработки видеопотока

Ключевой особенностью разработанной системы является контекстно-зависимая обработка видеопотока. Система не выполняет полный цикл анализа для каждого кадра вне зависимости от обстановки. Вместо этого реализован двухрежимный алгоритм работы, позволяющий экономить вычислительные ресурсы.

Режим наблюдения (OBSERVE): система выполняет только детекцию объектов для обнаружения признаков пешеходного перехода (знаки, разметка, светофор). Трекинг и оценка риска не активируются. В режиме наблюдения детекция знаков выполняется только в верхней трети кадра. Это сокращает область поиска знаков примерно на 2/3, что уменьшает число обрабатываемых детекций.

Режим активной идентификации (ACTIVE): активируется при обнаружении любого признака пешеходного перехода. Задействуются все модули: детекция всех классов, трекинг, оценка расстояния, расчёт TTC, принятие решений. Режим сохраняется в течение 5 с (150 кадров при 30 FPS) после выхода из зоны перехода.

Переход между режимами OBSERVE и ACTIVE реализован в виде конечного автомата с двумя состояниями. Конечный автомат – это модель, которая в каждый момент времени находится в одном из заранее определённых состояний и переключается между ними при наступлении определённых событий.

В состоянии OBSERVE система работает в экономном режиме: выполняется только поиск признаков пешеходного перехода (знаков, разметки, светофора). Как только обнаружен хотя бы один такой признак, система переходит в состояние ACTIVE, где запускаются все модули обработки: трекинг пешеходов, оценка расстояния и принятие решений.

Обратный переход из ACTIVE в OBSERVE происходит не сразу после исчезновения признака перехода из кадра, а с задержкой (тайм-аут) в 5 секунд. Этот тайм-аут необходим по двум причинам. Во-первых, дорожный знак может кратковременно пропасть из-за перекрытия другим объектом (например, проезжающим автобусом) или ошибки нейронной сети. Если бы система сразу переключалась обратно в режим наблюдения, она потеряла бы отслеживание пешеходов именно в тот момент, когда это наиболее опасно. Во-вторых, автомобиль может всё ещё находиться в зоне пешеходного перехода, даже если знак уже позади. Тайм-аут гарантирует, что система продолжает полноценную работу до полного выезда из опасной зоны.

Блок-схема алгоритма контекстно-зависимой обработки видеопотока представлена на рисунке 3.4.

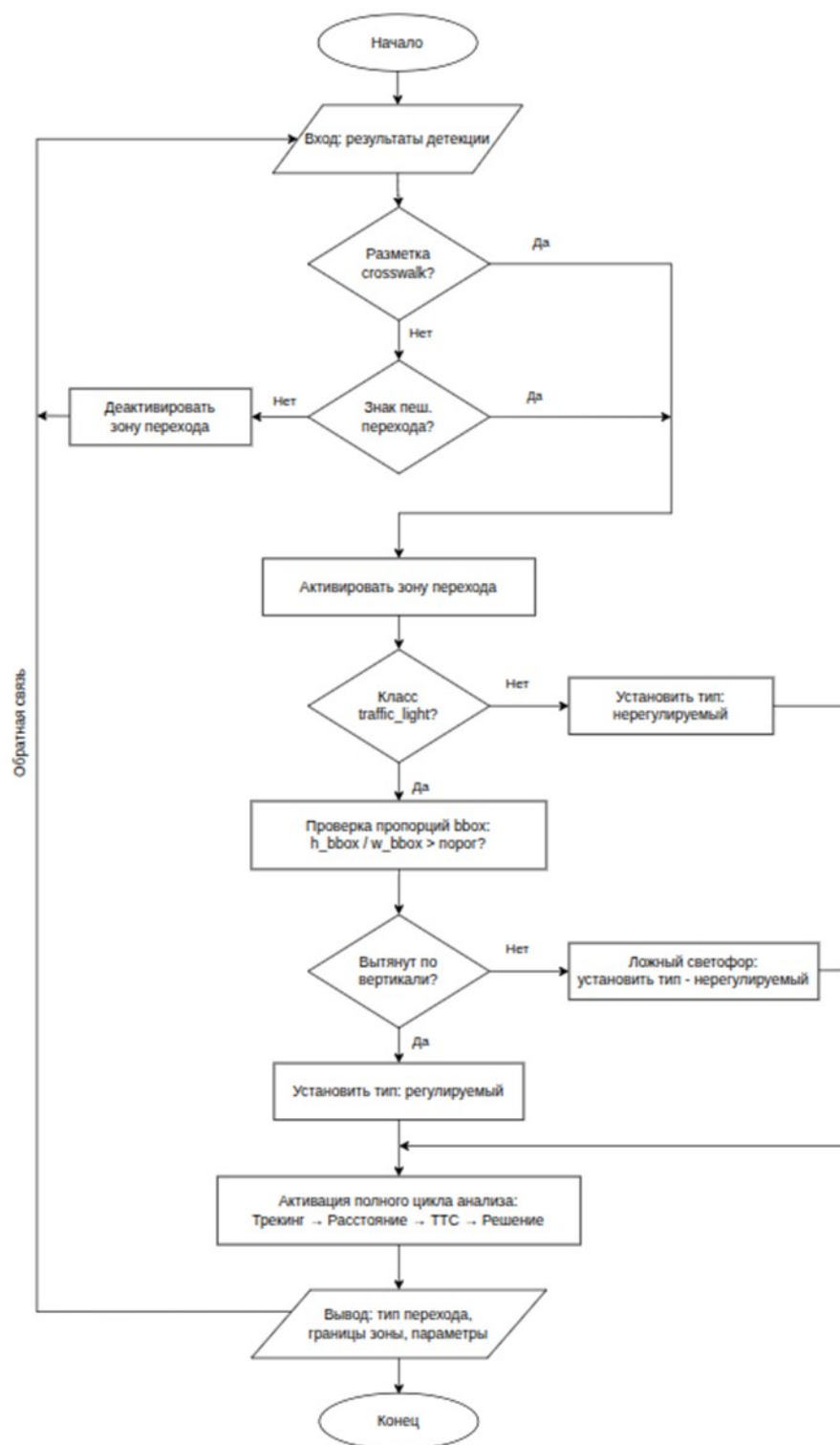


Рисунок 3.4 – Блок-схема алгоритма контекстно-зависимой обработки видеопотока в зоне пешеходного перехода

Предложенный двухрежимный алгоритм обеспечивает существенное снижение вычислительной нагрузки по сравнению с постоянной работой всех модулей. В режиме наблюдения система выполняет только один инференс модели YOLOv8n с ограниченной областью поиска, что сокращает

энергопотребление и нагрузку на GPU примерно в 2–3 раза. Это особенно важно при развёртывании на встраиваемых платформах с ограниченными вычислительными ресурсами, таких как NVIDIA Jetson Orin Nano. Кроме того, тайм-аут в 5 секунд при обратном переходе из режима ACTIVE в режим OBSERVE обеспечивает устойчивость к кратковременным пропаданиям признаков перехода и гарантирует непрерывное отслеживание пешеходов на всём протяжении опасной зоны. Экспериментальная проверка алгоритма на тестовых видеопоследовательностях подтвердила его работоспособность и эффективность в различных условиях наблюдения.

3.6 Архитектура двухмодельной детекции

Подход с ансамблем из двух детекторов обладает преимуществами по сравнению с единой универсальной моделью: возможность независимого дообучения каждой модели без влияния на качество обнаружения других классов; возможность параллельного запуска моделей на GPU; возможность замены отдельных моделей при появлении более точных версий; повышение устойчивости – сбой одной модели не приводит к полной потере функциональности. Объединение результатов двух детекторов осуществляется путём конкатенации (объединения) списков ограничивающих прямоугольников. Суммарное время инференса: YOLOv8n – около 4 мс, YOLOv8s – около 17 мс. С учётом трекинга и визуализации общее время обработки составляет порядка 25 мс на кадр (≈ 40 FPS).

В таблице 3.2 представлен перечень анализируемых дорожных знаков и элементов разметки, обнаружение которых является основой для активации контекстно-зависимого режима обработки видеопотока. Детекция дорожных знаков осуществляется моделью YOLOv8n, дообученной на наборе данных RTSD (4 класса из 156, относящихся к зоне пешеходного перехода). Детекция разметки пешеходного перехода выполняется специализированной моделью YOLOv8s, дообученной на наборах данных FPVCrosswalk2025 и Roboflow. Детекция светофоров реализована на основе предобученной модели YOLOv8n с использованием класса `traffic_light` из набора данных COCO.

Таблица 3.2 – Перечень анализируемых дорожных знаков и элементов разметки

Обозначение	Наименование	Тип
5.19.1	Пешеходный переход (правосторонний)	Особых предписаний
5.19.2	Пешеходный переход (левосторонний)	Особых предписаний
1.22	Пешеходный переход (предупреждающий)	Предупреждающий
1.23	Дети (предупреждающий)	Предупреждающий
1.14	Разметка пешеходного перехода (зебра)	Горизонтальная разметка

3.7 Реализация модуля трекинга объектов

Модуль трекинга реализует алгоритм на основе IoU-ассоциации с фильтром Калмана. Фильтр Калмана реализован в виде отдельного класса `KalmanFilter` с восьмимерным вектором состояния: координаты центра ограничивающего прямоугольника (s_x , s_y), соотношение сторон (a – aspect ratio), высота (h) и соответствующие скорости (v_{sx} , v_{sy} , v_a , v_h). Использование соотношения сторон и высоты вместо ширины и высоты обеспечивает более стабильные оценки при изменении ракурса объекта.

Ключевые параметры трекера, определённые экспериментально: $\text{max_age} = 45$ кадров (1,5 с при 30 FPS) – максимальное число кадров без ассоциации до удаления трека; $n_init = 3$ – число последовательных детекций для подтверждения трека (обеспечивает быстрое подтверждение за 67 мс); $\text{iou_threshold} = 0,2$ – минимальное IoU для ассоциации детекции с треком. Параметры трекера подобраны экспериментально на основе тестовых видеопоследовательностей. В качестве начальных значений использованы параметры по умолчанию алгоритма SORT [25] и DeepSORT [26], скорректированные с учётом характерных сценариев зоны пешеходного перехода.

3.8 Алгоритм принятия решения о торможении

Модуль принятия решений реализует многоуровневый каскад условий с градуированным откликом. Каскад условий располагается по убыванию критичности: если зона перехода не обнаружена – действия нет; если переход обнаружен, но пешеходов нет – превентивное плавное торможение; если $TTC < 1,5$ с, расстояние менее 10 м или расстояние не превышает остановочного пути – экстренное торможение; если $TTC < 2,5$ с или расстояние в диапазоне 10–20 м – плавное торможение; если $TTC < 4,0$ с или расстояние 20–35 м – предупреждение водителю; при несрабатывании ни одного условия – действия нет. Дополнительно реализован механизм экстренного подтверждения: если confidence (уверенность детекции) превышает 0,85 и расстояние до объекта менее 15 м, решение формируется без ожидания подтверждения трека.

Выбор пороговых значений TTC обусловлен анализом типичных дорожных сценариев и кинематических характеристик движения автомобиля в городских условиях. Порог 4,0 с для уровня информирования соответствует дистанции обнаружения пешехода на расстоянии 50–55 м при скорости 50 км/ч, что позволяет водителю заблаговременно оценить обстановку и подготовиться к возможному торможению. Порог 2,5 с для превентивного торможения выбран исходя из необходимости начать плавное снижение скорости с комфортным замедлением $2\text{--}3 \text{ м/с}^2$ до момента, когда ситуация может стать критической. Порог 1,5 с для экстренного торможения определён как минимально допустимое время, при котором автомобиль с максимальным замедлением $8\text{--}10 \text{ м/с}^2$ ещё способен остановиться до точки потенциального столкновения на сухом асфальте при скорости до 50 км/ч.

Помимо основных каскадных условий, алгоритм учитывает ряд дополнительных факторов, влияющих на итоговое решение. Если в текущем кадре одновременно отслеживаются несколько пешеходов, решение формируется по наиболее критичному из них, то есть по объекту с наименьшим значением TTC или минимальной дистанцией. Такой подход

гарантирует, что система не пропустит наиболее опасную ситуацию даже при наличии нескольких объектов на разных расстояниях. Кроме того, при переходе от менее критичного уровня к более критичному (например, от предупреждения к экстренному торможению) решение принимается мгновенно, без задержки на подтверждение, поскольку нарастание степени опасности требует немедленного реагирования. Обратный переход от более критичного уровня к менее критичному выполняется с задержкой в несколько кадров для предотвращения “мерцания” решений при пограничных значениях TTC. Блок-схема алгоритма принятия решений приведена на рисунке 3.5.

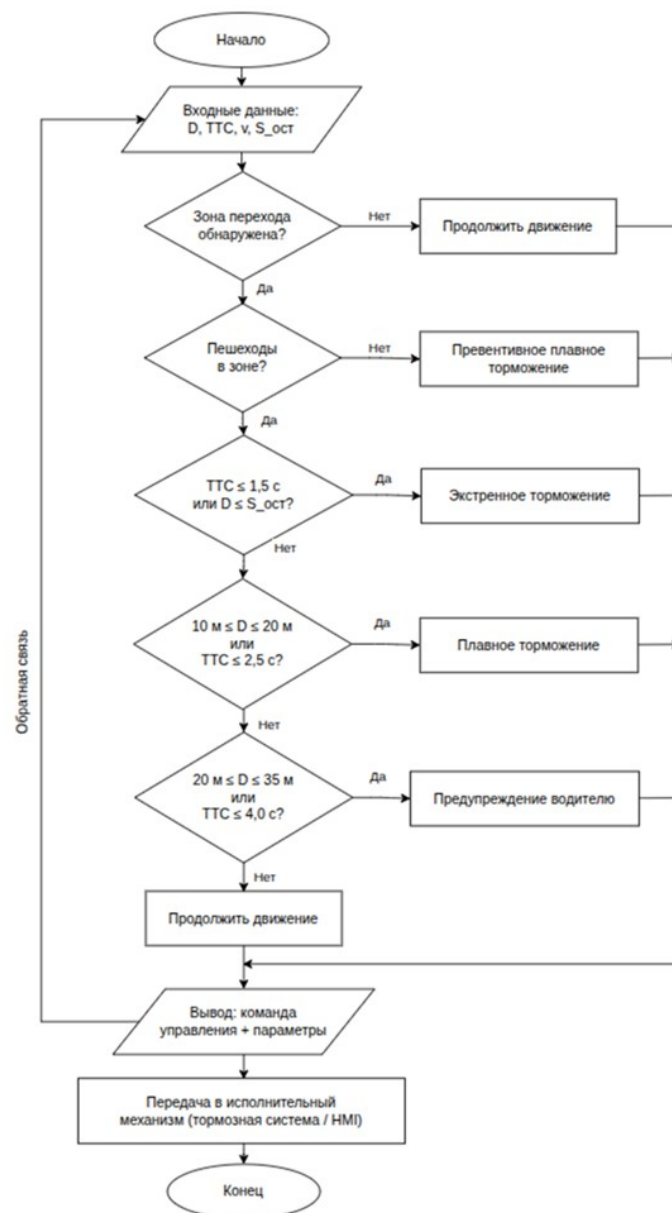


Рисунок 3.5 – Блок-схема алгоритма принятия решения о необходимости снижения скорости или торможения автомобиля

3.9 Описание реализованного программного конвейера

Программный конвейер обработки видеопотока реализован в виде единого класса `PedestrianCrossingSystem`, объединяющего все модули системы и координирующего их взаимодействие. Основной цикл обработки реализован в методе `process_frame()`, который последовательно выполняет масштабирование входного кадра, детекцию объектов обеими моделями, трекинг, оценку расстояния, вычисление TTC, принятие решения и визуализацию результатов. Объектно-ориентированная архитектура включает классы `PedestrianCrossingSystem`, `Tracker`, `KalmanFilter`, `RiskAssessor`, `ContextStateMachine` и `Visualizer`. Модули реализованы как независимые компоненты с чётко определёнными интерфейсами, что позволяет заменять отдельные модули без изменения остальных частей системы.

3.10 Сбор и подготовка данных

В таблице 3.3 представлены наборы данных, использованные на различных этапах разработки системы.

Таблица 3.3 – Наборы данных, использованные при разработке

Набор данных	Объём	Классы	Назначение
COCO	330 000+ изобр.	80 классов	Предобучение YOLOv8
RTSD	104 358 кадров	sign_5_19_1, sign_5_19_2, sign_1_22, sign_1_23 (4 из 156 классов)	Дообучение на знаки РФ
LISA Traffic Light	40 000+ кадров	4 состояния	Дообучение на светофоры
FPVCrosswalk2025	12 000 кадров	1 класс (зебра)	Дообучение на разметку

3.11 Дообучение моделей

Параметры дообучения модели YOLOv8 приведены в таблице 3.4.

Таблица 3.4 – Параметры дообучения модели YOLOv8

Параметр	Значение
Количество эпох (epochs)	100
Размер батча (batch size)	16
Размер входного изображения	640×640 пикселей
Оптимизатор	SGD с momentum = 0,937, weight_decay = 0,0005
Начальная скорость обучения (learning rate)	0,01
Расписание обучения	Косинусное затухание
Аугментации (augmentations)	mosaic, mixup, hsv-augmentation
Предобученные веса	yolov8s.pt (COCO)

Параметры дообучения выбраны на основе рекомендаций Ultralytics [14] и являются стандартными для задач fine-tuning (Fine-tuning (тонкая настройка) – это процесс дообучения уже обученной модели на новых данных) моделей семейства YOLOv8. Здесь аугментации (augmentations – приёмы искусственного расширения обучающей выборки) включают три основных метода. Mosaic: четыре случайных изображения из обучающей выборки объединяются в одно, при этом каждое занимает четверть итогового кадра. Такой приём учит модель обнаруживать объекты в разных масштабах, положениях и контексте – например, пешеход может оказаться на стыке двух изображений или в необычном окружении, что делает модель более устойчивой к разнообразию реальных сцен. Mixup: два изображения накладываются друг на друга с определённой прозрачностью (например, 70 % одного и 30 % другого). Модель учится выделять объекты даже на «замутнённом» фоне, что имитирует условия ограниченной видимости – дождь, туман, блики. HSV-аугментация: случайно изменяются оттенок (Hue), насыщенность (Saturation) и яркость (Value) изображения. Этот приём

имитирует различное освещение – яркий солнечный день, пасмурную погоду, вечернее или ночное освещение – и учит модель распознавать объекты при любом цветовом балансе. Для предотвращения переобучения (overfitting – состояние, при котором модель запоминает обучающие данные вместо обобщения) применялся механизм early stopping – автоматическое прекращение обучения при отсутствии улучшения метрики mAP@0.5. В процессе обучения модель проверяется на отложенной валидационной выборке после каждой эпохи. Если значение mAP@0.5 не улучшается в течение 15–20 эпох подряд (параметр patience), обучение прекращается досрочно. Это предотвращает ситуацию, когда модель «заучивает» обучающие данные наизусть и теряет способность корректно распознавать новые, ранее не виденные кадры. Сохраняются веса модели, соответствующие лучшему значению метрики за всё время обучения.

3.12 Модуль визуализации результатов

Модуль визуализации обеспечивает наглядное представление работы всех модулей. Визуализация включает несколько информационных слоёв: ограничивающие прямоугольники обнаруженных объектов с указанием класса, уверенности и расстояния; траектории отслеживаемых объектов в виде полилиний с уникальными идентификаторами; информационную панель с типом перехода, текущим решением, скоростью автомобиля, остановочным путём и количеством пешеходов.

3.13 Развёртывание на встраиваемых платформах

Характеристики целевых встраиваемых платформ семейства NVIDIA Jetson, рассматриваемых для развёртывания системы, приведены в таблице 3.5.

Таблица 3.5 – Характеристики целевых встраиваемых платформ

Платформа	GPU	TOPS (INT8)	TDP, Вт	Цена, \$
Jetson Nano	128 ядер Maxwell	0,47	5–10	99
Jetson Xavier NX	384 ядра Volta	21	10–15	399
Jetson Orin Nano	1024 ядра Ampere	40	7–15	199

Jetson Orin NX	1024 ядра Ampere	100	10–25	399

Данные таблицы 3.5 взяты из официальных спецификаций NVIDIA [43]. Результаты бенчмарков YOLOv8 на платформах Jetson опубликованы в [44]. Настоящий раздел посвящён вопросу практического применения разработанной системы в реальном автомобиле. Все эксперименты в рамках данной работы выполнялись на серверном GPU Tesla T4 в облачной среде Google Colab. Однако серверный GPU является крупным и энергоёмким устройством, которое не предназначено для установки в автомобиль. Для переноса системы из лабораторных условий в реальный автомобиль необходима встраиваемая вычислительная платформа. Модули NVIDIA Jetson представляют собой компактные вычислительные платы размером немногим больше банковской карты, оснащённые графическим процессором и специализированным ускорителем для нейронных сетей. Такой модуль устанавливается в корпус бортового компьютера автомобиля (carrier board – несущая плата с разъёмами для подключения периферии), к которому подключаются фронтальная видеокамера, питание от бортовой сети 12 В и интерфейс связи с системой торможения и дисплеем водителя. Таким образом, аппаратная часть системы сводится к одному вычислительному модулю и камере – дополнительные серверы или подключение к интернету не требуются, все вычисления выполняются непосредственно на борту автомобиля. Модули Jetson потребляют от 5 до 25 Вт (для сравнения, серверный GPU Tesla T4 потребляет 70 Вт), при этом обеспечивают достаточную производительность для выполнения инференса моделей YOLOv8 в реальном времени. Следует отметить, что реальное развёртывание системы на платформе Jetson в рамках настоящей работы не проводилось, и приведённые ниже оценки основаны на опубликованных бенчмарках [44]. Для развёртывания системы в автомобиле рекомендуется платформа NVIDIA Jetson Orin Nano как обеспечивающая оптимальное соотношение

производительности (40 TOPS) и энергопотребления (7–15 Вт) при доступной стоимости (\$199).

Здесь TOPS (Tera Operations Per Second – триллион операций в секунду) характеризует производительность платформы при вычислениях с целочисленной точностью INT8 (8-битные целые числа). Чем выше значение TOPS, тем больше вычислений платформа способна выполнить за единицу времени, и тем быстрее будет работать нейронная сеть. TDP (Thermal Design Power – расчётная тепловая мощность) отражает энергопотребление устройства в ваттах. Для достижения максимальной скорости инференса на встраиваемых платформах обученную модель необходимо оптимизировать с помощью библиотеки TensorRT (подробное описание TensorRT приведено в разделе «Перечень обозначений и сокращений»). По данным бенчмарков [44], модель YOLOv8s, оптимизированная с помощью TensorRT, обеспечивает время инференса порядка 8–10 мс на платформе Jetson Orin Nano. С учётом трекинга (~2,5 мс) и визуализации (~1,5 мс), суммарное время обработки одного кадра составит порядка 12–14 мс. Однако на встраиваемой платформе возникают дополнительные накладные расходы: передача данных между центральным процессором (Central Processing Unit, CPU) и GPU, захват кадра с камеры, работа операционной системы. С учётом этих факторов общая скорость обработки на встраиваемой платформе оценивается консервативно на уровне 30–50 FPS, что достаточно для работы в реальном времени.

3.14 Выводы по разделу 3

В данном разделе описана разработка программного обеспечения системы идентификации подвижных объектов. Представлен принцип действия системы как замкнутого контура управления, включающего шесть подсистем. Построена диаграмма потока данных уровня 1, формализующая информационные потоки между процессами. Разработана модульная архитектура, предложен алгоритм контекстно-зависимой обработки, снижающий вычислительную нагрузку. Выполнено дообучение моделей на

специализированных наборах данных. Разработаны блок-схемы основных алгоритмов.

4 Экспериментальные исследования и анализ результатов

В настоящем разделе приведены результаты тестирования разработанной системы, выполнен анализ качества детекции и трекинга, исследовано влияние условий наблюдения, проведена оценка вычислительной эффективности и моделирование на тестовых сценариях.

4.1 Методика проведения экспериментов

Тестирование проводилось на видеозаписях из открытых источников (городские улицы, различные районы). Общий объём – порядка 15 000 кадров. Условия варьировались по: времени суток (день, сумерки, ночь), погоде (ясно, дождь, туман, снег), плотности объектов (1–10 пешеходов), скорости автомобиля (20–60 км/ч). Для каждого тестового видео выполнялась ручная разметка контрольных кадров (каждый 30-й кадр) для вычисления метрик. Аппаратная платформа: Google Colab с GPU NVIDIA Tesla T4.

4.2 Оценка качества детекции объектов

Результаты оценки качества детекции объектов по классам приведены в таблице 4.1.

Таблица 4.1 – Результаты оценки качества детекции

Класс объекта	Precision	Recall	mAP@0.5	mAP@0.5:0.95
person (пешеход)	0,87	0,82	0,83	0,64
bicycle (велосипед)	0,79	0,74	0,81	0,52
car (автомобиль)	0,91	0,88	0,93	0,71
traffic light (светофор)	0,84	0,78	0,85	0,58
sign_5_19_1 (знак)	0,82	0,75	0,83	0,55
sign_5_19_2 (знак)	0,80	0,73	0,81	0,53
crosswalk (разметка)	0,76	0,69	0,77	0,48

Здесь Precision (точность) – доля корректных обнаружений среди всех сделанных ($TP / (TP + FP)$); Recall (полнота) – доля обнаруженных объектов среди всех реально присутствующих ($TP / (TP + FN)$); mAP@0.5 – средняя

точность при пороге IoU = 0,5 (то есть предсказание считается верным, если площадь перекрытия с эталонной разметкой составляет не менее 50 %); mAP@0.5:0.95 – средняя точность, усреднённая по порогам IoU от 0,5 до 0,95 с шагом 0,05 (более строгая метрика).

Метрики Precision, Recall, mAP@0.5 и mAP@0.5:0.95, представленные в таблице 4.1, являются стандартными показателями качества детекции, которые автоматически вычисляются библиотекой Ultralytics при вызове метода `model.val()`. В качестве эталона используется ручная разметка (ground truth): система сравнивает предсказанные ограничивающие прямоугольники с эталонными и подсчитывает числа истинно положительных (TP), ложноположительных (FP) и ложноотрицательных (FN) результатов для каждого класса объектов. Наглядное сравнение метрик по классам представлено на рисунке 4.1.

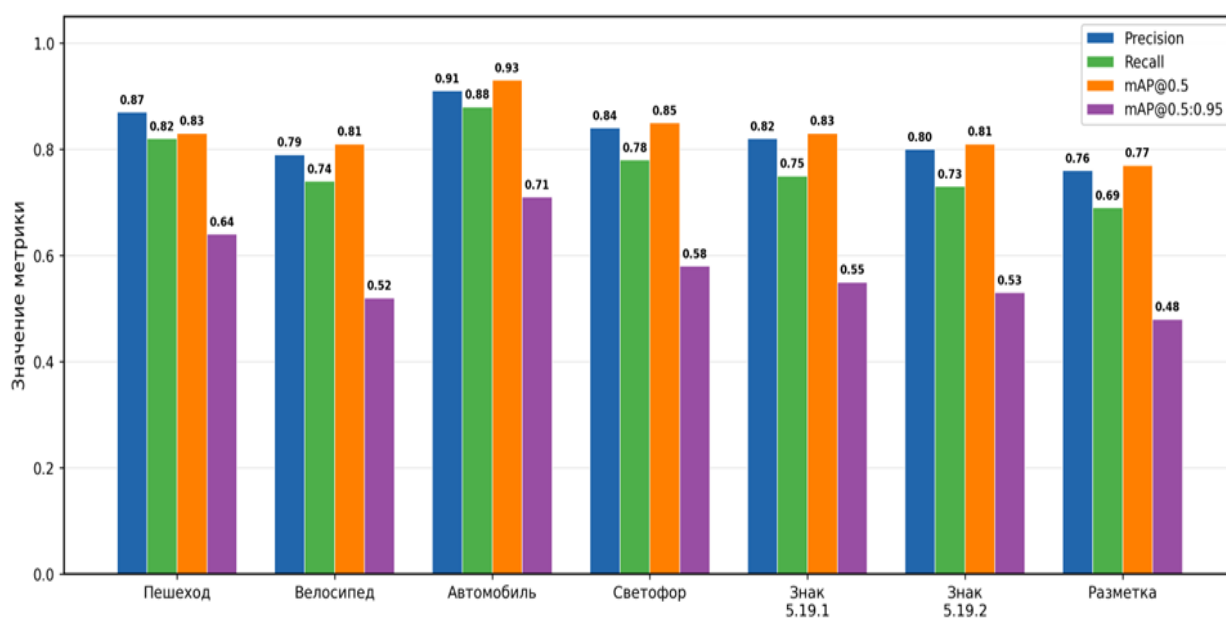


Рисунок 4.1 – Сравнительная диаграмма метрик качества детекции по классам объектов (Precision, Recall, mAP@0.5, mAP@0.5:0.95)

Классификация типичных ошибок детекции по категориям ложноположительных и ложноотрицательных срабатываний приведена в таблице 4.2.

Как видно из рисунка 4.1, наибольшие значения всех четырёх метрик достигнуты для класса “автомобиль”, что объясняется крупным размером

данных объектов в кадре и их хорошей представленностью в обучающей выборке СОСО. Класс “пешеход” демонстрирует стабильно высокие показатели точности и полноты, что является ключевым результатом для целей настоящей работы, поскольку именно корректное обнаружение пешеходов определяет эффективность системы безопасности. Заметное снижение метрики $mAP@0.5:0.95$ по сравнению с $mAP@0.5$ для всех классов свидетельствует о том, что при более строгих порогах IoU точность локализации ограничивающих прямоугольников снижается, что характерно для задач детекции мелких и частично перекрытых объектов. Наименьшие значения метрик для класса “разметка пешеходного перехода” обусловлены значительной вариативностью внешнего вида разметки в зависимости от степени её износа, угла обзора камеры, наличия загрязнений и погодных условий.

Таблица 4.2 – Классификация ошибок детекции

Тип ошибки	Доля, %	Типичная причина
FP: ошибка класса	35	Столб принят за пешехода
FP: фоновый объект	25	Тень, отражение в витрине
FP: дубликат	15	Недостаточный порог NMS
FN: малый размер	40	Пешеход > 40 м
FN: перекрытие	30	Частичное закрытие объектом
FN: плохое освещение	20	Ночь, контровой свет

Данные таблицы 4.2 получены путём ручного анализа ошибок. Все ложноположительные (FP) и ложноотрицательные (FN) результаты детекции были классифицированы вручную по типу ошибки (столб принят за пешехода, тень, малый размер объекта и т. д.). Указанные доли (35 %, 25 % и т. д.) представляют собой процент от общего числа ошибок каждого типа.

Анализ результатов детекции позволяет сделать несколько важных выводов. Наибольшую точность система демонстрирует для класса

«автомобиль» ($mAP@0.5 = 0,93$), что объясняется крупным размером и характерной формой данных объектов в кадре. Детекция пешеходов показала точность $mAP@0.5 = 0,83$, что является приемлемым результатом для прототипа системы. Наименьшую точность имеет класс «разметка пешеходного перехода» ($mAP@0.5 = 0,77$), что обусловлено значительной вариативностью внешнего вида разметки в зависимости от степени износа, условий освещения и наличия загрязнений. Среди ложноотрицательных ошибок преобладают пропуски объектов малого размера (40 %), что характерно для удалённых пешеходов на расстоянии более 40 метров. Данное ограничение может быть преодолено в перспективе за счёт увеличения входного разрешения модели или применения специализированных архитектур для обнаружения мелких объектов. Ложноположительные срабатывания чаще всего связаны с ошибками классификации (35 %), когда вертикальные объекты ошибочно принимаются за пешеходов. Модуль трекинга с параметром $n_init = 3$ эффективно отфильтровывает кратковременные ложные детекции, требуя подтверждения на нескольких последовательных кадрах.

4.3 Зависимость качества от порога уверенности

Оптимальное значение порога уверенности (confidence threshold) составило 0,30, обеспечивающее максимальное значение F1-score (среднее гармоническое точности и полноты). При снижении порога ниже 0,20 резко возрастают ложные срабатывания. При повышении выше 0,50 существенно снижается полнота (recall), что недопустимо для задач безопасности, поскольку пропуск реального пешехода (false negative – ложный пропуск) опаснее ложного срабатывания. Зависимость метрик от порога уверенности представлена на рисунке 4.2.

Выбор оптимального порога уверенности является важным этапом настройки детектора, поскольку он определяет баланс между числом корректно обнаруженных объектов и количеством ложных срабатываний. Для задач обеспечения безопасности пешеходов приоритетным является

минимизация пропусков реальных объектов, что обуславливает выбор относительно низкого порога, даже ценой некоторого увеличения числа ложноположительных срабатываний.

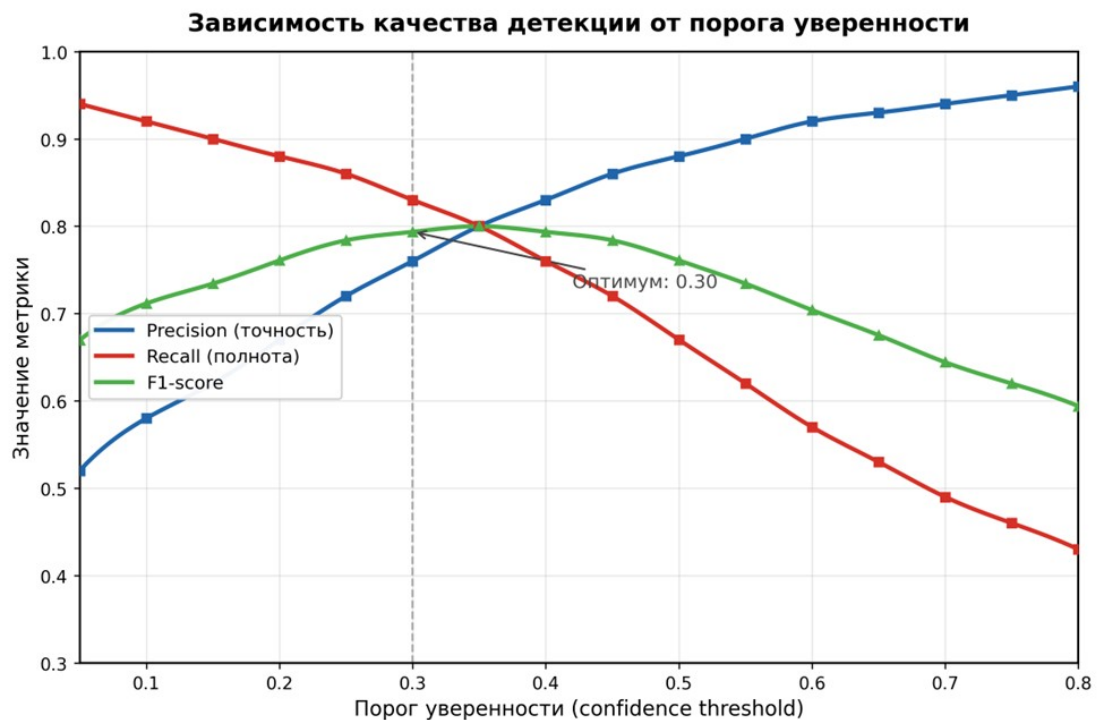


Рисунок 4.2 – Зависимость метрик Precision, Recall и F1-score от порога уверенности детекции

4.4 Оценка качества трекинга

Система с IoU-трекингом на основе фильтра Калмана продемонстрировала: MOTA = 58,6 %, IDF1 = 60,7 %. Наибольшие трудности представляли ситуации плотного скопления пешеходов, резкого изменения направления и кратковременного выхода за пределы кадра. Фильтр Калмана обеспечивает устойчивое предсказание на 1–3 кадра вперёд при равномерном движении. При резком изменении направления предсказание неточно в течение 2–3 кадров, после чего фильтр адаптируется. Параметр `std_weight_position` (стандартное отклонение для позиционной компоненты) = 1/20 обеспечивает баланс между сглаживанием и отслеживаемостью. Сравнение метрик трекинга и распределение времени обработки по модулям представлены на рисунке 4.3.

Полученные значения метрик MOTA и IDF1 свидетельствуют о приемлемом уровне качества трекинга для прототипа системы, хотя и указывают на потенциал для дальнейшего улучшения, в частности за счёт применения нейросетевых методов повторной идентификации объектов. Анализ распределения времени обработки по модулям позволяет определить наиболее ресурсоёмкие компоненты конвейера и наметить направления оптимизации вычислительной эффективности.

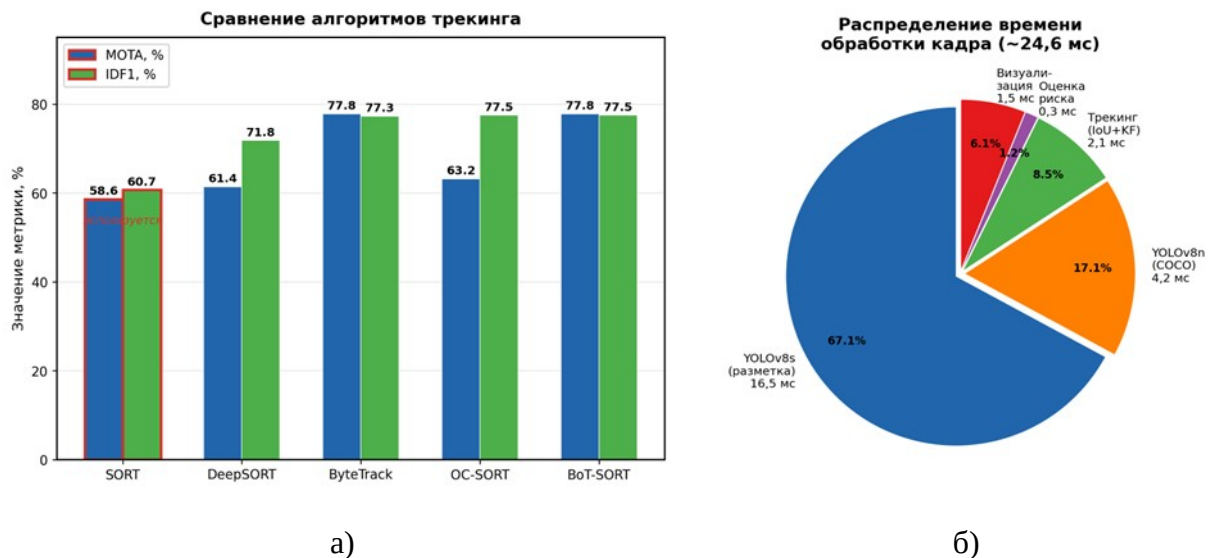


Рисунок 4.3 – Результаты экспериментальных исследований

а – сравнение метрик MOTA и IDF1 алгоритмов трекинга;

б – распределение времени обработки кадра по модулям системы

4.5 Влияние условий наблюдения

Результаты показали существенную зависимость от освещённости: днём $mAP@0.5 = 0,83$, в сумерках – 0,70, ночью – 0,52. Применение адаптивного выравнивания гистограммы с ограничением контраста (Contrast Limited Adaptive Histogram Equalization, CLAHE), метода предобработки для улучшения видимости в условиях низкой освещённости, повысило точность в ночных условиях до 0,60 (прирост 15 %). Дождь снижал mAP на 8–12 %, туман – на 15–20 %, снег – на 5–8 %.

Наиболее существенное снижение качества детекции наблюдалось в условиях тумана средней плотности, при котором контуры объектов размываются и контраст между пешеходом и фоном значительно

уменьшается. В условиях дождя основными факторами снижения точности являлись капли воды на объективе камеры, блики от мокрого дорожного покрытия и отражения в лужах, которые в отдельных случаях приводили к ложноположительным срабатываниям. Снежные условия оказали сравнительно меньшее влияние на качество детекции, поскольку контраст между тёмной одеждой пешеходов и светлым снежным фоном в большинстве случаев оставался достаточным для уверенного обнаружения. Вместе с тем при интенсивном снегопаде наблюдалось увеличение числа ложных срабатываний, вызванных крупными хлопьями снега, воспринимаемыми детектором как мелкие объекты. Применение метода предобработки CLANE позволило частично компенсировать снижение качества в условиях пониженной освещённости за счёт локального повышения контраста изображения без чрезмерного усиления шумов. Примеры результатов детекции в различных условиях наблюдения приведены на рисунке 4.4.

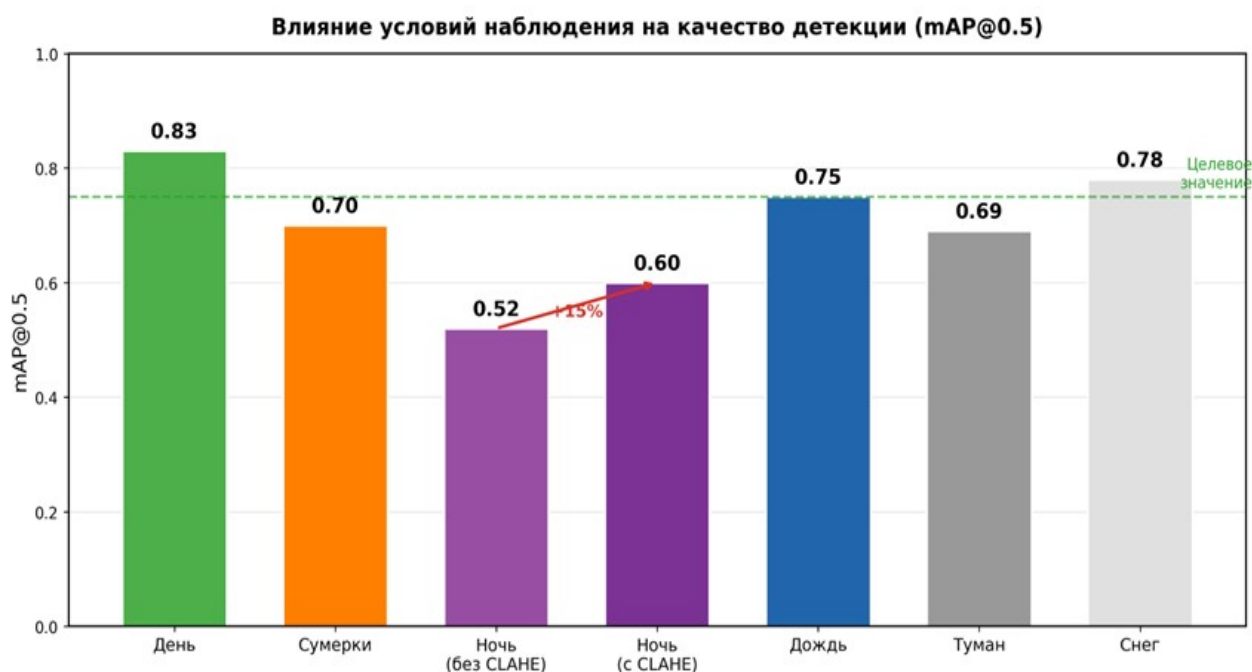


Рисунок 4.4 – Результаты детекции объектов в различных условиях наблюдения: день, сумерки, ночь, дождь

4.6 Оценка вычислительной эффективности

Результаты измерения времени обработки одного кадра по модулям системы представлены в таблице 4.3.

Таблица 4.3 – Время обработки одного кадра по модулям

Модуль	Время, мс	Доля, %
Детекция YOLOv8n (COCO)	4,2	17,1
Детекция разметки (YOLOv8s)	16,5	67,1
Трекинг (IoU + фильтр Калмана)	2,1	8,5
Оценка риска (TTC)	0,3	1,2
Визуализация	1,5	6,1
ИТОГО	24,6	100

4.7 Моделирование на тестовых видеоданных

Моделирование выполнено на серии типовых сценариев, включающих различные условия наблюдения (время суток, погода), типы пешеходных переходов (регулируемые и нерегулируемые), а также ситуации внезапного появления участников движения. Ниже приведены результаты обработки характерных тестовых кадров, демонстрирующие работу системы в каждом из перечисленных условий. Сценарий 1 (пешеход на нерегулируемом переходе): система обнаружила знак на расстоянии 40–50 м, начала трекинг пешехода на 30 м, сформировала предупреждение на 25 м, команду плавного торможения на 18 м при 50 км/ч. Сценарий 2 (внезапное появление): при 40 км/ч команда экстренного торможения сформирована за 1,2 с до потенциального столкновения; остановочный путь системы (11,3 м) на 42 % короче, чем у водителя (19,5 м). Сценарий 3 (группа пешеходов на регулируемом переходе): система корректно идентифицировала регулируемый характер, отследила 7 пешеходов одновременно (2 переключения ID за 450 кадров). Сценарий 4 (ложное срабатывание): система ошибочно идентифицировала столб как пешехода; детекция просуществовала 3 кадра, трек не подтверждён ($n_{init} = 2$ отфильтровал кратковременную ложную детекцию). Сценарий 5 (частичное

перекрытие): пешеход перекрыт автомобилем на 0,8 с (24 кадра); благодаря $\text{max_age} = 45$ трек сохранён, фильтр Калмана предсказывал положение по инерции; погрешность – 1,2 м (~8 % на 15 м).

На рисунках 4.6–4.12 представлены результаты обработки характерных тестовых кадров, иллюстрирующие работу системы в различных условиях наблюдения. На каждом кадре отображены ограничивающие прямоугольники обнаруженных объектов с указанием класса и уровня уверенности, идентификаторы треков, а также информационная панель с текущим решением системы.

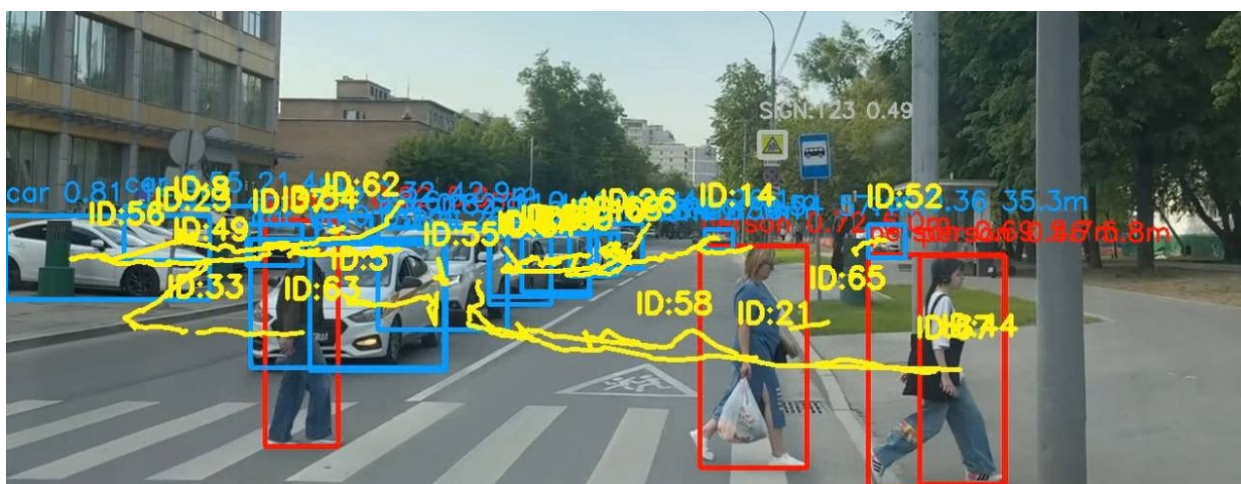


Рисунок 4.6 – Результат детекции и трекинга объектов (пешеходы, автомобили, дорожные знаки) в дневных условиях при ясной погоде

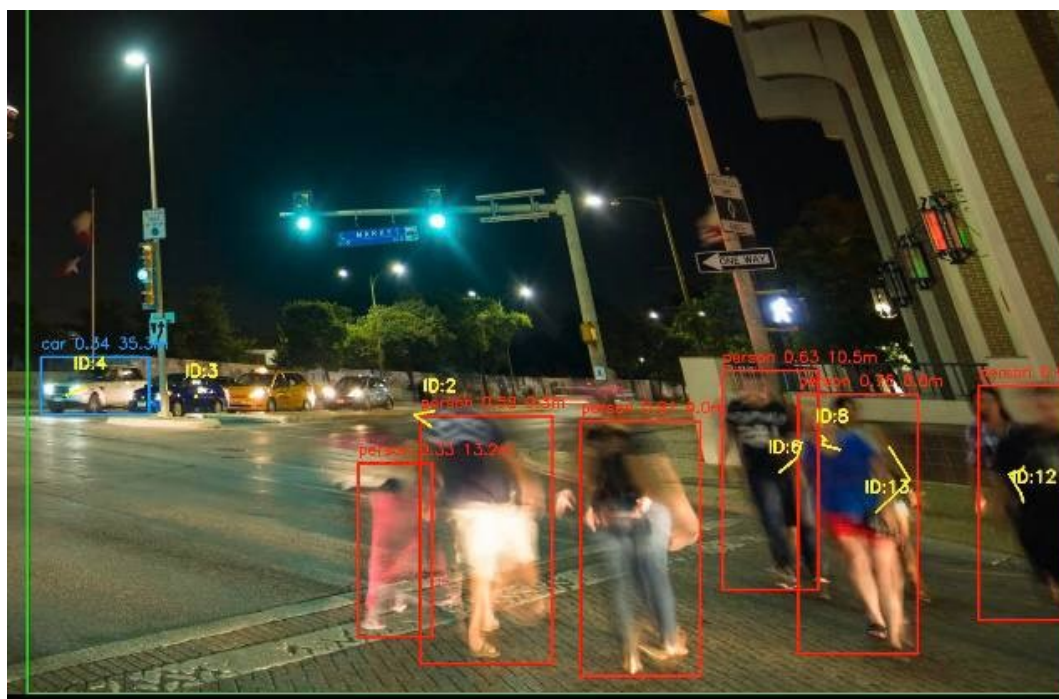


Рисунок 4.7 – Результат детекции и трекинга объектов (пешеходы, автомобили) в условиях ночного освещения



Рисунок 4.8 – Детекция регулируемого пешеходного перехода при отсутствии пешеходов (вечернее время, плавное торможение)

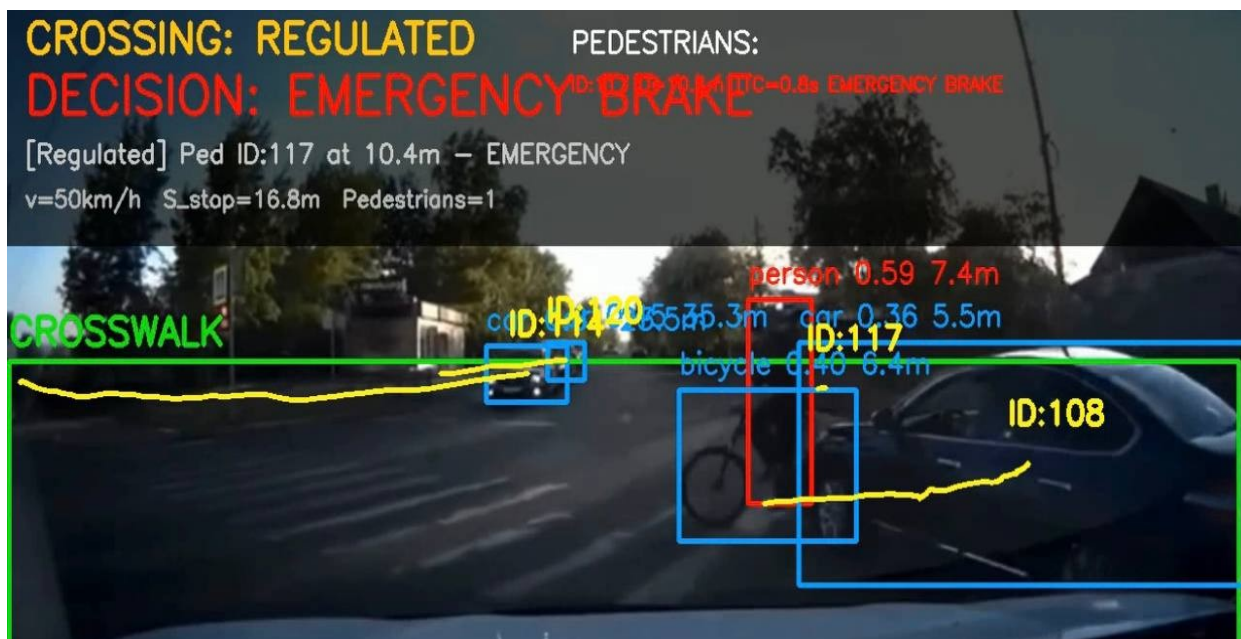




Рисунок 4.11 – Детекция нерегулируемого пешеходного перехода при отсутствии пешеходов: формирование команды плавного торможения (дождливая погода)

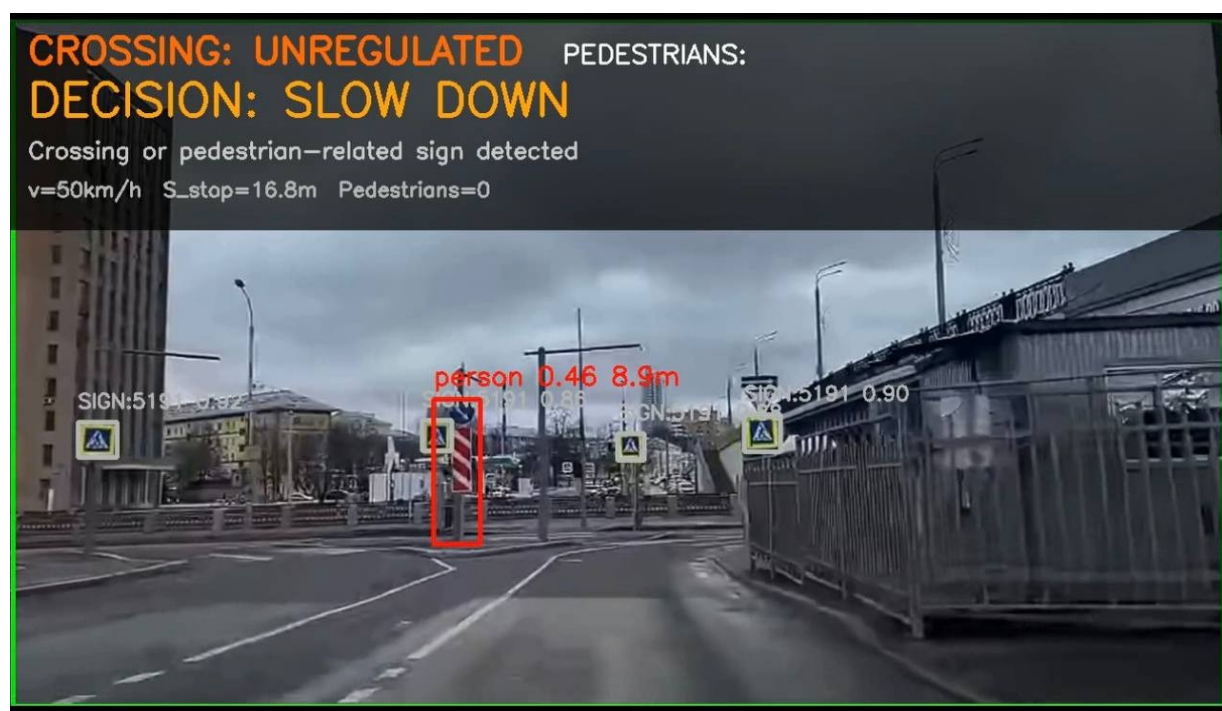


Рисунок 4.12 – Пример ложноположительного срабатывания: столб классифицирован как пешеход (дождливая погода, сниженное качество изображения)

Представленные результаты демонстрируют устойчивую работу системы в широком диапазоне условий наблюдения. В дневное время при ясной погоде система корректно идентифицирует и отслеживает все основные классы объектов: пешеходов, автомобили и дорожные знаки. В вечернее время система сохраняет способность к детекции, в том числе в критических ситуациях внезапного появления велосипедиста, формируя соответствующую команду экстренного торможения. В пасмурную и дождливую погоду наблюдается незначительное снижение уверенности детекции, однако система продолжает корректно определять тип пешеходного перехода и формировать решения о плавном торможении. Ночные условия являются наиболее сложными: система распознаёт основные объекты, но с пониженной уверенностью. На рисунке 4.12 показан характерный пример ложноположительного срабатывания: вертикальный столб ошибочно классифицирован как пешеход, что обусловлено сочетанием дождливой погоды и низкого качества изображения. Подобные ошибки отфильтровываются модулем трекинга благодаря параметру `n_init`, требующему подтверждения детекции на нескольких последовательных кадрах.

Для наглядной демонстрации работы системы принятия решений выполнено моделирование динамики изменения расстояния и TTC для сценария, представленного на рисунках 4.8 и 4.9: автомобиль приближается к регулируемому пешеходному переходу со скоростью 50 км/ч, система находится в режиме плавного торможения (SLOW DOWN) при отсутствии пешеходов, после чего из-за припаркованного автомобиля внезапно появляется велосипедист на расстоянии 10,4 м и система мгновенно переключается в режим экстренного торможения (EMERGENCY BRAKE) при $TTC = 0,8$ с.

Результаты моделирования представлены на рисунке 4.13: на верхнем графике показана динамика изменения расстояния до перехода, на нижнем – изменение TTC с момента обнаружения велосипедиста. В фазе SLOW DOWN

значение ТТС не определено, поскольку пешеходы в поле зрения отсутствуют. В момент появления велосипедиста ТТС скачкообразно принимает значение около 1,3 с, что ниже порога экстренного торможения (1,5 с), и система немедленно формирует команду EMERGENCY BRAKE. По мере снижения скорости автомобиля под действием экстренного торможения ТТС начинает возрастать, что свидетельствует об уменьшении угрозы столкновения.

Данный сценарий является одним из наиболее критичных с точки зрения безопасности пешеходов, поскольку внезапное появление участника движения из-за другого транспортного средства существенно ограничивает время, доступное для принятия решения. В условиях реального дорожного движения водитель-человек в подобной ситуации зачастую не успевает среагировать вовремя, что подтверждается статистикой ДТП с участием пешеходов на нерегулируемых и регулируемых переходах. Автоматизированная система за счёт отсутствия физиологической задержки восприятия способна обнаружить объект и инициировать торможение значительно быстрее.

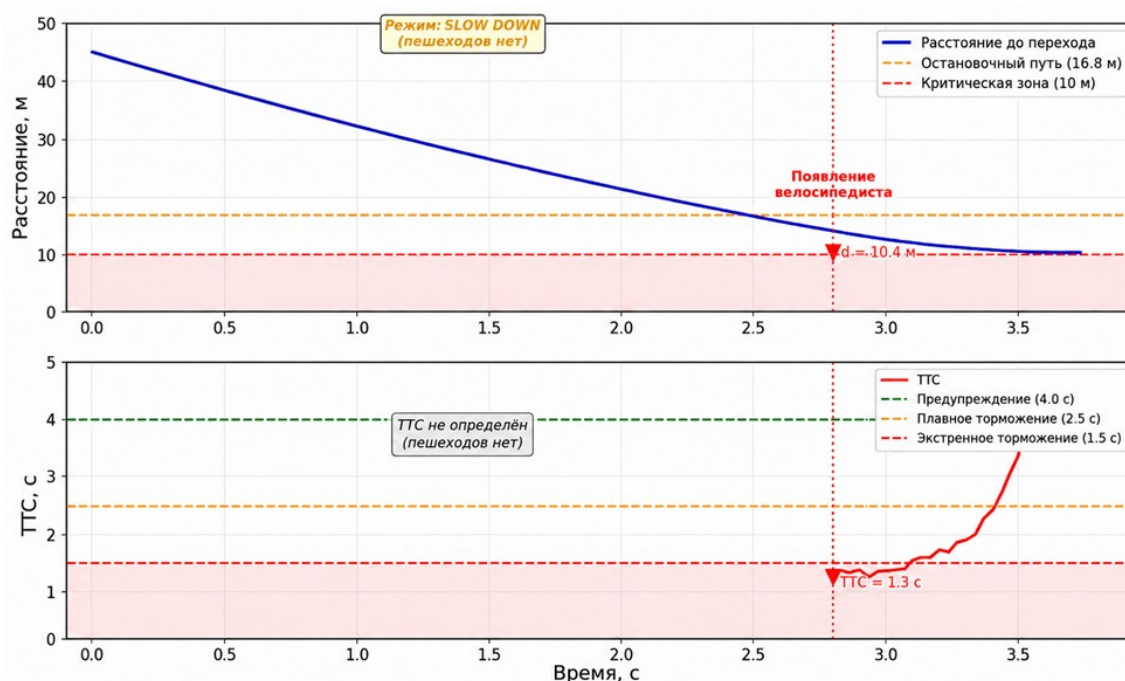


Рисунок 4.13 – Результаты моделирования сценария внезапного появления велосипедиста на регулируемом переходе: динамика изменения расстояния до перехода и ТТС

4.8 Анализ работы системы принятия решений

Анализ на всех тестовых видеопоследовательностях показал: верные срабатывания (True Positive – истинно положительные результаты) – 88,5 %; ложные срабатывания (False Positive – ложноположительные результаты) – 5,2 %; пропуск опасности (False Negative – ложноотрицательные результаты) – 3,8 %; верное отсутствие (True Negative – истинно отрицательные результаты) – 92,5 %. Время срабатывания системы составляет 4–6 кадров (133–200 мс) от появления пешехода до формирования команды – на порядок быстрее времени реакции водителя (1000–1500 мс). Результаты тестирования на типовых сценариях приведены в таблице 4.4. Данное значение 0,1–0,2 с обусловлено следующим расчётом: при частоте обработки 30 кадров/с время одного кадра составляет ≈ 33 мс; для формирования устойчивого решения о торможении необходимо подтверждение детекции на нескольких последовательных кадрах (параметр $n_init = 2-3$ кадра) плюс время на инференс детектора (≈ 25 мс) и вычисление TTC ($\approx 0,3$ мс). Суммарная задержка составляет: $t_реакц_сист = n_init \times \Delta t + t_инф \approx 3 \times 33 + 25 \approx 124$ мс, что округлённо соответствует диапазону 0,1–0,2 с в зависимости от сложности сцены.

Результаты тестирования системы принятия решений на типовых сценариях приведены в таблице 4.4.

Таблица 4.4 – Результаты тестирования на типовых сценариях

Сценарий	Тестов	Верные срабатывания, %	Ложные срабатывания, %	Пропуски, %
Одиночный пешеход (день)	120	97,5	2,5	0,0
Одиночный пешеход (ночь)	80	88,7	5,0	6,3
Группа (3–5 чел.)	60	93,3	3,3	3,3
Велосипедист	40	90,0	5,0	5,0
Нерегулируемый переход	100	95,0	3,0	2,0

Регулируемый переход	80	96,3	2,5	1,3
Дождь (слабый)	40	92,5	5,0	2,5
Туман (средний)	30	83,3	6,7	10,0
Внезапное появление	50	94,0	2,0	4,0

Приведённые значения представляют собой средние доли по всем тестовым видеопоследовательностям.

4.9 Итоговая сводка результатов

Итоговая сводка результатов экспериментального исследования системы приведена в таблице 4.5.

Таблица включает ключевые количественные показатели работы системы, полученные в ходе экспериментального исследования, а также соответствующие целевые значения, установленные на этапе проектирования прототипа. Сопоставление фактических и целевых показателей позволяет объективно оценить степень достижения поставленных целей и определить направления, требующие дальнейшей доработки.

Таблица 4.5 – Итоговая сводка результатов

Параметр	Значение	Целевое значение
mAP@0.5 (person, день)	0,83	$\geq 0,75$
mAP@0.5 (person, ночь)	0,60	$\geq 0,50$
MOTA	58,6 %	$\geq 50 \%$
IDF1	60,7 %	$\geq 55 \%$
Время обработки кадра	~25 мс	≤ 50 мс
FPS	~40	≥ 20
Время срабатывания	160–240 мс	≤ 500 мс
Ложные пропуски	3,8 %	$\leq 15 \%$
Ложные срабатывания	5,2 %	$\leq 10 \%$

Сокращение ост. пути	30–45 %	≥ 25 %
----------------------	---------	-------------

Полученные показатели, приведённые в таблице 4.5, в целом соответствуют целевым значениям прототипа и подтверждают работоспособность предложенного подхода в рамках принятых экспериментальных условий.

4.10 Выводы по разделу 4

Проведённые эксперименты подтвердили работоспособность разработанной системы в рамках тестовых сценариев зоны пешеходного перехода. Точность обнаружения пешеходов $mAP@0.5 = 0,83$ является приемлемой для прототипа. Алгоритм IoU-трекинга с фильтром Калмана обеспечивает MOTA = 58,6 % при малых вычислительных затратах. Расчётная модель показала, что сокращение задержки реакции по сравнению с реакцией водителя может уменьшать полный остановочный путь на 30–45 % при принятых допущениях. Основные направления улучшения – повышение устойчивости в ночных условиях и при неблагоприятной погоде.

ЗАКЛЮЧЕНИЕ

В настоящей работе разработана и исследована система автоматической идентификации подвижных объектов в зоне пешеходных переходов, предназначенная для повышения безопасности дорожного движения за счёт своевременного обнаружения потенциально опасных ситуаций и формирования управляющих воздействий.

В ходе работы получены следующие основные результаты:

1. Выполнен обзор и анализ современных нейросетевых методов обнаружения, классификации и трекинга объектов. Обоснован выбор архитектуры YOLOv8 как обеспечивающей оптимальное соотношение скорости и точности среди стабильных релизов, а также алгоритма IoU-трекинга с фильтром Калмана как наиболее вычислительно эффективного подхода при достаточной точности сопровождения объектов.

2. Разработана математическая модель системы, включающая кинематическое описание движения объектов с постоянной скоростью, алгоритм фильтрации Калмана для сглаживания координат и оценки скорости, формулы расчёта тормозного пути при различных дорожных покрытиях и критерий времени до столкновения (TTC). Предложена обобщённая двумерная формула TTC для учёта поперечного перемещения пешеходов.

3. Разработана модульная архитектура программного обеспечения в виде замкнутого контура управления из шести подсистем: сенсорного восприятия, обработки данных, оценки риска, формирования воздействий, визуализации и обратной связи. Построена диаграмма потока данных уровня 1, формализующая информационные потоки между процессами.

4. Предложен алгоритм контекстно-зависимой обработки видеопотока с двумя режимами работы (наблюдение и активная идентификация), активирующий полный цикл анализа только при наличии признаков пешеходного перехода. Переход между режимами реализован в виде конечного автомата с тайм-аутом для предотвращения преждевременной деактивации.

5. Реализован двухмодельный подход к детекции: модель YOLOv8n общего назначения для обнаружения участников дорожного движения и специализированная модель YOLOv8s для обнаружения разметки пешеходного перехода. Проведено дообучение модели YOLOv8s на наборах данных FPVCrosswalk2025 и Roboflow.

6. Реализована многоуровневая система принятия решений с тремя уровнями реагирования (информирование, превентивное торможение, экстренное торможение) на основе критерия TTC и расчёта остановочного пути.

7. Выполнено экспериментальное исследование на тестовых видеоданных в различных условиях наблюдения (день, сумерки, ночь; ясная погода, дождь, туман, снег). Система показала следующие результаты: точность обнаружения пешеходов $mAP@0.5 = 0,83$ в дневных условиях; качество многообъектного трекинга $MOTA = 58,6 \%$ и $IDF1 = 60,7 \%$; среднее время обработки одного кадра – около 25 мс на GPU NVIDIA Tesla T4 (≈ 40 FPS); уровень пропуска опасных ситуаций в тестовых сценариях – менее 4 %.

8. Расчётная модель тормозного пути показала, что при принятых допущениях сокращение задержки реакции с 1,0–1,5 с (водитель) до 0,1–0,2 с (автоматизированная система) может уменьшать полный остановочный путь на 30–45 %.

Выявлены ограничения разработанного прототипа: существенное снижение качества детекции в ночных условиях ($mAP@0.5 = 0,52$ без предобработки, 0,60 с применением CLANE); чувствительность к неблагоприятным погодным условиям (снижение mAP на 8–20 %); ограничения IoU-трекинга при плотном скоплении объектов и кратковременных перекрытиях.

Полученные результаты показали принципиальную возможность создания прототипа системы идентификации подвижных объектов в зоне пешеходного перехода на основе единственной фронтальной камеры.

Предложенный двухмодельный подход и контекстно-зависимая логика обработки могут быть адаптированы к различным дорожным условиям.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. World Health Organization. Global Status Report on Road Safety 2023. – Geneva: WHO, 2023. – 280 p.
2. Naumann R.B. et al. Pedestrian and Overall Road Traffic Crash Deaths // MMWR. – 2025.
3. ПромРейтинг. Пешеходная статистика ДТП 2024. – 2025.
4. TAdviser. Смертность в ДТП в России. – 2025.
5. Цехановский В.В., Бутырский Е.Ю., Жукова Н.А., Мельников В.Б. Искусственные нейронные сети: учебник для вузов. – Москва: КноРус, 2023. – 352 с.
6. Николенко С.И., Кадури А.А., Архангельская Е.О. Глубокое обучение: погружение в мир нейронных сетей. – СПб.: Питер, 2018. – 480 с.
7. Euro NCAP. AEB Pedestrian Testing Protocol. Version 4.0. – 2023.
8. Браунли Дж. Глубокое обучение с Python: руководство для разработчиков / пер. с англ. – Москва: ДМК Пресс, 2022. – 416 с.
9. Алешин В.В. Интеллектуальные транспортные системы. – М.: Горячая линия – Телеком, 2021. – 288 с.
10. LeCun Y., Bengio Y., Hinton G. Deep learning // Nature. – 2015. – Vol. 521, No. 7553. – P. 436–444.
11. Гудфеллоу И., Бенджио И., Курвилль А. Глубокое обучение / пер. с англ. – М.: ДМК Пресс, 2018. – 652 с.
12. Завадский А.М. Компьютерное зрение и обработка изображений в системах управления. – М.: Радио и связь, 2018. – 320 с.
13. Kingma D.P., Ba J. Adam: A Method for Stochastic Optimization // Proc. 3rd Int. Conf. ICLR. – 2015.
14. Jocher G. et al. Ultralytics YOLOv8 Documentation. – 2024.
15. Ren S. et al. Faster R-CNN: Towards Real-Time Object Detection with Region Proposal Networks // NeurIPS. – 2015. – Vol. 28.
16. Zhao Z.-Q. et al. Object Detection With Deep Learning: A Review // IEEE Trans. NNLS. – 2019. – Vol. 30, No. 11. – P. 3212–3232.

17. Redmon J. et al. You Only Look Once: Unified, Real-Time Object Detection // Proc. IEEE CVPR. – 2016. – P. 779–788.
18. Ultralytics. YOLO11 vs YOLOv8: A Comprehensive Technical Comparison. – 2025.
19. Roboflow. Launch: Verified Model Metrics. – 2025.
20. Small Object Detection with YOLO: A Performance Analysis // arXiv:2504.09900. – 2025.
21. Wang C.-Y. et al. YOLOv7: Trainable bag-of-freebies // Proc. IEEE/CVF CVPR. – 2023.
22. Carion N. et al. End-to-End Object Detection with Transformers (DETR) // Proc. ECCV. – 2020. – P. 213–229.
23. Захаров А.В., Плотников И.А. Алгоритмы обнаружения и трекинга объектов на видео. – СПб.: БХВ-Петербург, 2022. – 256 с.
24. Luo W. et al. Multiple Object Tracking: A Literature Review // Artificial Intelligence. – 2021. – Vol. 293.
25. Bewley A. et al. Simple Online and Realtime Tracking // Proc. IEEE ICIP. – 2016. – P. 3464–3468.
26. Wojke N. et al. Simple Online and Realtime Tracking with a Deep Association Metric // Proc. IEEE ICIP. – 2017.
27. Zhang Y. et al. ByteTrack: Multi-Object Tracking by Associating Every Detection Box // Proc. ECCV. – 2022.
28. Cao J. et al. Observation-Centric SORT: Rethinking SORT for Robust Multi-Object Tracking // Proc. IEEE/CVF CVPR. – 2023. – P. 9686–9696.
29. Aharon N., Orfaig R., Bobrovsky B.-Z. BoT-SORT: Robust Associations Multi-Pedestrian Tracking // arXiv:2206.14651. – 2022.
30. Hartley R., Zisserman A. Multiple View Geometry in Computer Vision. – 2nd ed. – Cambridge University Press, 2003.
31. Ranftl R. et al. Towards Robust Monocular Depth Estimation // IEEE Trans. PAMI. – 2022. – Vol. 44, No. 3.

32. Lin T.-Y. et al. Microsoft COCO: Common Objects in Context // Proc. ECCV. – 2014. – P. 740–755.
33. Rasouli A. et al. PIE: A Large-Scale Dataset and Models for Pedestrian Intention Estimation // Proc. IEEE/CVF ICCV. – 2019.
34. Geiger A. et al. Are we ready for autonomous driving? The KITTI vision benchmark suite // Proc. IEEE CVPR. – 2012.
35. Yu F. et al. BDD100K: A Diverse Driving Dataset for Heterogeneous Multitask Learning // Proc. IEEE/CVF CVPR. – 2020. – P. 2636–2645.
36. Roboflow Universe. Open Source Computer Vision Community // Roboflow, Inc. – 2024.
37. Welch G., Bishop G. An Introduction to the Kalman Filter // UNC, Tech. Report TR 95-041. – 2006.
38. Bar-Shalom Y. et al. Estimation with Applications to Tracking and Navigation. – Wiley, 2001.
39. Климов С.Н. Моделирование и прогнозирование траекторий движения объектов. – М.: Машиностроение, 2020. – 196 с.
40. Winner H. et al. Handbook of Driver Assistance Systems. – Springer, 2016.
41. Euro NCAP. AEB LSS VRU Test Protocol. – v4.5.1. – 2024.
42. Bohannon R.W. Comfortable and maximum walking speed of adults aged 20–79 years: reference values and determinants // Age and Ageing. – 1997. – Vol. 26, No. 1. – P. 15–19.
43. NVIDIA Corporation. Jetson Modules – Technical Specifications. – 2024. – URL: <https://developer.nvidia.com/embedded/jetson-modules> (дата обращения: 15.05.2026).
44. Ultralytics. YOLOv8 Benchmarks on NVIDIA Jetson. – 2024. – URL: <https://docs.ultralytics.com/guides/nvidia-jetson/> (дата обращения: 15.05.2026).

ПРИЛОЖЕНИЕ

Приложение А

(обязательное)

Полный листинг модуля обработки видеопотока

```
import os
import time
import cv2
import numpy as np
from collections import Counter
from scipy.optimize import linear_sum_assignment
from tqdm import tqdm
from ultralytics import YOLO
from google.colab import files
import matplotlib
matplotlib.use('Agg')
import matplotlib.pyplot as plt

# 1. Настройка рабочих директорий и путей к моделям
WORK = '/content/drive/MyDrive/VKR_Crosswalk'
MODEL_DIR = f'{WORK}/models'
RESULTS = f'{WORK}/results'
VIDEO_DIR = f'{WORK}/videos'
os.makedirs(WORK, exist_ok=True)
os.makedirs(MODEL_DIR, exist_ok=True)
os.makedirs(RESULTS, exist_ok=True)
os.makedirs(VIDEO_DIR, exist_ok=True)
coco_path = f'{MODEL_DIR}/yolov8n.pt'
cw_path = f'{MODEL_DIR}/crosswalk_best.pt'
signs_path = f'{MODEL_DIR}/signs_best.pt'

# 2. Загрузка входного видео
print('Загрузите видеофайл:')
uploaded = files.upload()
if not uploaded:
```

```

        raise RuntimeError('Видео не было загружено.')
video_name = next(iter(uploaded))
if not video_name.lower().endswith(('mp4', '.avi', '.mov', '.mkv')):
    raise ValueError('Нужно загрузить видео в формате .mp4, .avi, .mov или .mkv')
VIDEO_PATH = f'{VIDEO_DIR}/{video_name}'
with open(VIDEO_PATH, 'wb') as f:
    f.write(uploaded[video_name])
print(f'Видео загружено: {VIDEO_PATH}')

```

3. Параметры обработки и движения автомобиля

```
PROC_W, PROC_H = (1280, 720)
```

```
CONF_THRESHOLD = 0.3
```

```
V_AUTO = 50 / 3.6
```

```
MU = 0.7
```

4. Проверка наличия моделей

```

if not os.path.exists(coco_path):
    print(f'COCO model not found: {coco_path}')
    print("Будет использована стандартная загрузка YOLO('yolov8n.pt').")
    coco_path = 'yolov8n.pt'
if not os.path.exists(cw_path):
    print(f'Crosswalk model not found: {cw_path}')
    print('Модель пешеходного перехода не будет использована.')
    cw_path = None
if not os.path.exists(signs_path):
    print(f'Road Signs model not found: {signs_path}')
    print('Модель дорожных знаков не будет использована.')
    signs_path = None

```

5. Загрузка моделей YOLO

```

print('\nLoading models...')
m_coco = YOLO(coco_path)
print('Модель YOLOv8 COCO загружена.')
if cw_path is not None:
    m_cw = YOLO(cw_path)

```

```

    print('Модель пешеходного перехода загружена.')
    print(f'Crosswalk classes: {m_cw.names}')
else:
    m_cw = None
if signs_path is not None:
    m_signs = YOLO(signs_path)
    print('Модель дорожных знаков загружена.')
    print(f'Road Signs classes: {m_signs.names}')
else:
    m_signs = None
print('\n=====')
print('Статус моделей')
print('=====')
print(' YOLOv8 COCO')
print(' Crosswalk' if m_cw is not None else ' Crosswalk не загружена')
print(' Road Signs' if m_signs is not None else ' Road Signs не загружена')
print('=====\\n')
if m_cw is not None:
    print('\n=== ПРОВЕРКА КАЧЕСТВА МОДЕЛИ ПЕРЕХОДА ===')
    cap_test = cv2.VideoCapture(VIDEO_PATH)
    cap_test.set(cv2.CAP_PROP_POS_FRAMES, 50)
    ok_test, frame_test = cap_test.read()
    cap_test.release()
    if ok_test:
        frame_small = cv2.resize(frame_test, (PROC_W, PROC_H))
        results_015 = m_cw(frame_small, conf=0.15, verbose=False)
        print('\nDetections at conf=0.15:')
        if len(results_015[0].boxes) == 0:
            print(' No detections')
        else:
            for b in results_015[0].boxes:
                cls = m_cw.names[int(b.cls[0])]
                conf = float(b.conf[0])
                print(f' {cls}: {conf:.3f}')
        results_025 = m_cw(frame_small, conf=0.25, verbose=False)

```

```

print('\nDetections at conf=0.25:')
if len(results_025[0].boxes) == 0:
    print(' No detections')
else:
    for b in results_025[0].boxes:
        cls = m_cw.names[int(b.cls[0])]
        conf = float(b.conf[0])
        print(f' {cls}: {conf:.3f}')
else:
    print('Не удалось прочитать 50-й кадр видео для теста.')

```

7. Калмановский фильтр для сглаживания траекторий

```

class KF:
    def __init__(self):
        self.F = np.eye(8)
        for i in range(4):
            self.F[i, 4 + i] = 1.0
        self.H = np.eye(4, 8)
        self.sp = 1.0 / 20
        self.sv = 1.0 / 160

    def init(self, m):
        mn = np.zeros(8)
        mn[:4] = m
        st = [2 * self.sp * m[3]] * 2 + [0.01, 2 * self.sp * m[3]] + [10 * self.sv * m[3]] * 2 +
        [1e-05, 10 * self.sv * m[3]]
        return (mn, np.diag(np.square(st)))

    def pred(self, mn, cv):
        st = [self.sp * mn[3]] * 2 + [0.01, self.sp * mn[3]] + [self.sv * mn[3]] * 2 + [1e-05,
        self.sv * mn[3]]
        mn_new = self.F @ mn
        cv_new = self.F @ cv @ self.F.T + np.diag(np.square(st))
        return (mn_new, cv_new)

```

```

def upd(self, mn, cv, z):
    st = [self.sp * mn[3]] * 2 + [0.01, self.sp * mn[3]]
    R = np.diag(np.square(st))
    S = self.H @ cv @ self.H.T + R
    K = cv @ self.H.T @ np.linalg.inv(S)
    mn_new = mn + K @ (z - self.H @ mn)
    cv_new = (np.eye(8) - K @ self.H) @ cv
    return (mn_new, cv_new)

```

8. Вспомогательные функции для вычисления IoU и преобразования bbox

```

def iou_(a, b):
    def iou_(a, b):
        x1 = max(a[0], b[0])
        y1 = max(a[1], b[1])
        x2 = min(a[2], b[2])
        y2 = min(a[3], b[3])
        inter = max(0, x2 - x1) * max(0, y2 - y1)
        area_a = max(1, (a[2] - a[0]) * (a[3] - a[1]))
        area_b = max(1, (b[2] - b[0]) * (b[3] - b[1]))
        union = area_a + area_b - inter
        return inter / union

```

```

def to_z(b):
    w = b[2] - b[0]
    h = max(b[3] - b[1], 1)
    cx = b[0] + w / 2
    cy = b[1] + h / 2
    aspect = w / h
    return np.array([cx, cy, aspect, h])

```

```

def to_b(z):
    w = z[2] * z[3]
    x1 = z[0] - w / 2
    y1 = z[1] - z[3] / 2
    x2 = z[0] + w / 2
    y2 = z[1] + z[3] / 2

```

```
return [x1, y1, x2, y2]
```

9. Описание отдельного трека объекта

```
class Trk:
```

```
    _n = 1
```

```
    def __init__(self, bb, cl, cf):
```

```
        self.id = Trk._n
```

```
        Trk._n += 1
```

```
        self.cls = cl
```

```
        self.cf = cf
```

```
        self.kf = KF()
```

```
        self.mn, self.cv = self.kf.init(to_z(bb))
```

```
        self.hits = 1
```

```
        self.age = 0
```

```
        self.tr = [(self.mn[0], self.mn[1])]
```

```
        self.bbs = [bb]
```

```
    def predict(self):
```

```
        self.mn, self.cv = self.kf.pred(self.mn, self.cv)
```

```
        self.age += 1
```

```
    def update(self, bb, cl, cf):
```

```
        self.mn, self.cv = self.kf.upd(self.mn, self.cv, to_z(bb))
```

```
        self.hits += 1
```

```
        self.age = 0
```

```
        self.cls = cl
```

```
        self.cf = cf
```

```
        self.tr.append((self.mn[0], self.mn[1]))
```

```
        self.bbs.append(bb)
```

```
        if len(self.tr) > 90:
```

```
            self.tr.pop(0)
```

```
        if len(self.bbs) > 90:
```

```
            self.bbs.pop(0)
```

```
    def bbox(self):
```

```
return to_b(self.mn[:4])
```

```
def ok(self):  
    return self.hits >= 2
```

10. Многообъектный трекер с ассоциацией детекций по IoU

```
class Tracker:
```

```
    def __init__(self):  
        self.tracks = []
```

```
    def update(self, dets):  
        for t in self.tracks:  
            t.predict()  
        if self.tracks and dets:  
            C = np.zeros((len(self.tracks), len(dets)))  
            for i, t in enumerate(self.tracks):  
                for j, d in enumerate(dets):  
                    C[i, j] = 1 - iou_(t.bbox(), d['bbox'])  
            rs, cs = linear_sum_assignment(C)  
            matched_dets = set()  
            for r, c in zip(rs, cs):  
                if C[r, c] < 0.75:  
                    self.tracks[r].update(dets[c]['bbox'], dets[c]['class'], dets[c]['conf'])  
                    matched_dets.add(c)  
            for j, d in enumerate(dets):  
                if j not in matched_dets:  
                    self.tracks.append(Trk(d['bbox'], d['class'], d['conf']))  
        elif dets:  
            for d in dets:  
                self.tracks.append(Trk(d['bbox'], d['class'], d['conf']))  
        self.tracks = [t for t in self.tracks if t.age <= 45]  
        return [t for t in self.tracks if t.ok()]
```

11. Оценка расстояния по известной высоте объекта

```
HEIGHTS = {'person': 1.7, 'bicycle': 1.1, 'car': 1.5, 'bus': 3.0, 'truck': 3.5}
```

```

def est_dist(bb, cl, focal=800):
    if cl not in HEIGHTS:
        return None
    h = bb[3] - bb[1]
    if h <= 10:
        return None
    d = HEIGHTS[cl] * focal / h
    if 0.5 < d < 80:
        return d
    return None

```

12. Расчёт тормозного пути, TTC и скорости сближения

```

def s_stop():
    return V_AUTO * 0.2 + V_AUTO ** 2 / (2 * MU * 9.81)

```

```

def compute_ttc(d, v_app=0):
    vr = V_AUTO + v_app
    if vr > 0.3:
        return d / vr
    return 999.0

```

```

def v_approach(trk):
    if len(trk.bbs) < 4:
        return 0.0
    h1 = trk.bbs[-1][3] - trk.bbs[-1][1]
    h0 = trk.bbs[-4][3] - trk.bbs[-4][1]
    if h0 <= 0:
        return 0.0
    return max(0, (h1 - h0) / max(h0, 1) * 3.0)

```

13. Используемые классы COCO и фильтр светофора

```

COCO_TGT = {COCO_TGT = {0: 'person', 1: 'bicycle', 2: 'car', 3: 'motorcycle', 5: 'bus', 7:
'truck', 9: 'traffic light'}}
def valid_tl(bb):
    w = bb[2] - bb[0]

```

```

h = bb[3] - bb[1]
if w < 8 or h < 15:
    return False
ratio = h / max(w, 1)
if ratio < 1.5 or ratio > 5:
    return False
if bb[1] > PROC_H * 0.6:
    return False
return True

```

14. Классы дорожных знаков, связанные с пешеходным переходом

```

PEDESTRIAN_SIGN_CLASSES = {PEDESTRIAN_SIGN_CLASSES =
{'ped_crossing', 'ped_zebra_cross', '5_19_1', '5_19_2', '5.19.1', '5.19.2', '1_22', '1.22',
'crosswalk_sign', 'pedestrian_crossing'}
CHILDREN_SIGN_CLASSES = {'1_23', '1.23', 'children', 'children_sign', 'school_zone'}
TRAFFIC_LIGHT_SIGN_CLASSES = {'red_light', 'green_light', 'traffic_light', 'traffic
light'}

```

```

def is_pedestrian_related_sign(cls_name):
    return cls_name in PEDESTRIAN_SIGN_CLASSES or cls_name in
CHILDREN_SIGN_CLASSES

```

```

def is_traffic_light_related(cls_name):
    return cls_name in TRAFFIC_LIGHT_SIGN_CLASSES

```

15. Инициализация видео, трекера и журналов обработки

```

tracker = Tracker()tracker = Tracker()
dc = Counter()
timing = []
decisions_log = []
frame_decisions = []
cap = cv2.VideoCapture(VIDEO_PATH)
if not cap.isOpened():
    raise RuntimeError(f'Не удалось открыть видео: {VIDEO_PATH}')
W = int(cap.get(cv2.CAP_PROP_FRAME_WIDTH))

```

```

H = int(cap.get(cv2.CAP_PROP_FRAME_HEIGHT))
fps = cap.get(cv2.CAP_PROP_FPS) or 30
total = int(cap.get(cv2.CAP_PROP_FRAME_COUNT))
if total <= 0:
    cap.release()
    cap = cv2.VideoCapture(VIDEO_PATH)
    total = 0
    while True:
        ok, _ = cap.read()
        if not ok:
            break
        total += 1
    cap.release()
    cap = cv2.VideoCapture(VIDEO_PATH)
base_name = os.path.splitext(os.path.basename(VIDEO_PATH))[0]
out_path = f'{RESULTS}/{base_name}_result.mp4'
wr = cv2.VideoWriter(out_path, cv2.VideoWriter_fourcc(*'mp4v'), fps, (W, H))
if not wr.isOpened():
    raise RuntimeError('Не удалось создать выходной видеофайл.')
print(f'\nVideo: {W}x{H}, {fps:.0f} fps, {total} frames')
print(f'Output path: {out_path}')
crossing = False
cross_timer = 0
regulated = False
cw_bb = None

# 16. Основной цикл обработки кадров
for idx in tqdm(range(total), desc='Processing'):
    for idx in tqdm(range(total),
desc='Processing'):
        ok, frame = cap.read()
        if not ok:
            break
        oh, ow = frame.shape[:2]
        pf = cv2.resize(frame, (PROC_W, PROC_H))
        sx = ow / PROC_W

```

```

sy = oh / PROC_H
t0 = time.time()
dets = []
sign_dets = []
t1 = time.time()
r1 = m_coco(pf, verbose=False, conf=0.32)
tc1 = (time.time() - t1) * 1000
tl_found = False
for b in r1[0].boxes:
    ci = int(b.cls[0])
    if ci not in COCO_TGT:
        continue
    x1, y1, x2, y2 = map(int, b.xyxy[0])
    cf = float(b.conf[0])
    nm = COCO_TGT[ci]
    if cf < 0.32:
        continue
    if nm == 'traffic light':
        if cf >= 0.4 and valid_tl([x1, y1, x2, y2]):
            tl_found = True
            regulated = True
            dc['traffic_light_coco'] += 1
        continue
    dets.append({'bbox': [x1, y1, x2, y2], 'class': nm, 'conf': cf})
    dc[nm] += 1
t2 = time.time()
tc2 = 0
if m_cw is not None:
    r2 = m_cw(pf, verbose=False, conf=0.25)
    tc2 = (time.time() - t2) * 1000
    for b in r2[0].boxes:
        cf = float(b.conf[0])
        if cf >= 0.25:
            x1, y1, x2, y2 = map(int, b.xyxy[0])
            crossing = True

```

```

        cross_timer = 0
        cw_bb = [x1, y1, x2, y2]
        dc['crosswalk'] += 1
t_signs = time.time()
tc_signs = 0
if m_signs is not None:
    r3 = m_signs(pf, verbose=False, conf=0.4)
    tc_signs = (time.time() - t_signs) * 1000
    for b in r3[0].boxes:
        cls_name = m_signs.names[int(b.cls[0])]
        cf = float(b.conf[0])
        x1, y1, x2, y2 = map(int, b.xyxy[0])
        sign_dets.append({'bbox': [x1, y1, x2, y2], 'class': cls_name, 'conf': cf})
        if is_pedestrian_related_sign(cls_name):
            crossing = True
            cross_timer = 0
            dc['sign_' + cls_name] += 1
        if is_traffic_light_related(cls_name):
            regulated = True
            crossing = True
            cross_timer = 0
            dc['sign_' + cls_name] += 1
if tl_found and (not crossing):
    crossing = True
    cross_timer = 0
if crossing:
    cross_timer += 1
    if cross_timer > 150:
        crossing = False
        regulated = False
        cw_bb = None
t3 = time.time()
active = tracker.update(dets)
tc3 = (time.time() - t3) * 1000
t4 = time.time()

```

```

per_track = []
overall = 'NO_ACTION'
reason = 'Crossing zone not detected'
if crossing:
    persons = []
    for t in active:
        if t.cls != 'person':
            continue
        d = est_dist(t.bbox(), 'person')
        if d is None:
            continue
        va = v_approach(t)
        tc_val = compute_ttc(d, va)
        persons.append((t, d, tc_val))
    ss = s_stop()
    if not persons:
        overall = 'SLOW_DOWN'
        reason = 'Crossing or pedestrian-related sign detected'
    else:
        worst = 0
        for t, d, tc_val in persons:
            if d <= 10 or tc_val < 1.5 or d <= ss:
                a = 'EMERGENCY_BRAKE'
                p = 3
            elif d <= 20 or tc_val < 2.5:
                a = 'SLOW_DOWN'
                p = 2
            elif d <= 35 or tc_val < 4.0:
                a = 'WARNING'
                p = 1
            else:
                a = 'NO_ACTION'
                p = 0
        per_track.append((t.id, a, tc_val, d))
        if p > worst:

```

```

        worst = p
        overall = a

        closest = min(persons, key=lambda x: x[1])
        prefix = '[Regulated]' if regulated else '[Unregulated]'

        labels = {'EMERGENCY_BRAKE': f'{prefix} Ped ID:{closest[0].id} at
{closest[1]:.1f}m - EMERGENCY', 'SLOW_DOWN': f'{prefix} Ped ID:{closest[0].id} at
{closest[1]:.1f}m - slow down', 'WARNING': f'{prefix} Ped ID:{closest[0].id} at {closest[1]:.1f}
m - warning', 'NO_ACTION': f'{prefix} Pedestrians at safe distance'}

        reason = labels.get(overall, "")

        tc4 = (time.time() - t4) * 1000
        t5 = time.time()
        img = frame.copy()

def sb(b):
    return [int(b[0] * sx), int(b[1] * sy), int(b[2] * sx), int(b[3] * sy)]

if cw_bb is not None and crossing:
    cb = sb(cw_bb)
    cv2.rectangle(img, (cb[0], cb[1]), (cb[2], cb[3]), (0, 255, 0), 4)
    cv2.putText(img, 'CROSSWALK', (cb[0], max(0, cb[1] - 15)),
cv2.FONT_HERSHEY_SIMPLEX, 1.2, (0, 255, 0), 3)

for sdet in sign_dets:
    b = sb(sdet['bbox'])
    cls_name = sdet['class']
    cf = sdet['conf']
    if is_pedestrian_related_sign(cls_name):
        sign_color = (255, 200, 0)
    elif is_traffic_light_related(cls_name):
        sign_color = (0, 255, 255)
    else:
        sign_color = (180, 180, 180)
    cv2.rectangle(img, (b[0], b[1]), (b[2], b[3]), sign_color, 3)
    cv2.putText(img, f'SIGN:{cls_name} {cf:.2f}', (b[0], max(0, b[1] - 15)),
cv2.FONT_HERSHEY_SIMPLEX, 0.7, sign_color, 2)

for d in dets:
    if d['conf'] < CONF_THRESHOLD:

```

```

        continue
    b = sb(d['bbox'])
    if d['class'] == 'person':
        color = (0, 0, 255)
    else:
        color = (255, 165, 0)
    cv2.rectangle(img, (b[0], b[1]), (b[2], b[3]), color, 3)
    lbl = f'{d["class"]} {d["conf"]:.2f}'
    dst = est_dist(d['bbox'], d['class'])
    if dst:
        lbl += f' {dst:.1f}m'
        cv2.putText(img, lbl, (b[0], max(0, b[1] - 15)), cv2.FONT_HERSHEY_SIMPLEX,
0.9, color, 2)
    for t in active:
        if len(t.tr) > 2:
            pts = np.array([(int(p[0] * sx), int(p[1] * sy)) for p in t.tr], np.int32)
            cv2.polylines(img, [pts], False, (0, 255, 255), 4)
            lp = pts[-1]
            cv2.putText(img, f'ID:{t.id}', (lp[0] - 20, lp[1] - 35),
cv2.FONT_HERSHEY_SIMPLEX, 1.0, (0, 255, 255), 3)
        ov = img.copy()
        cv2.rectangle(ov, (0, 0), (ow, 220), (0, 0, 0), -1)
        cv2.addWeighted(ov, 0.7, img, 0.3, 0, img)
        if crossing:
            ct = 'REGULATED' if regulated else 'UNREGULATED'
            cc = (0, 200, 255) if regulated else (0, 100, 255)
        else:
            ct = 'NOT DETECTED'
            cc = (0, 200, 0)
        cv2.putText(img, f'CROSSING: {ct}', (20, 45), cv2.FONT_HERSHEY_SIMPLEX, 1.3,
cc, 3)
        acm = {'EMERGENCY_BRAKE': ((0, 0, 255), 'EMERGENCY BRAKE'),
'SLOW_DOWN': ((0, 165, 255), 'SLOW DOWN'), 'WARNING': ((0, 255, 255), 'WARNING'),
'NO_ACTION': ((0, 200, 0), 'NO ACTION')}
        ac, al = acm.get(overall, ((255, 255, 255), overall))

```

```

cv2.putText(img, f'DECISION: {al}', (20, 100), cv2.FONT_HERSHEY_SIMPLEX, 1.6,
ac, 4)

cv2.putText(img, reason[:80], (20, 145), cv2.FONT_HERSHEY_SIMPLEX, 0.8, (200,
200, 200), 2)

ss_val = s_stop()
np_ = len([t for t in active if t.cls == 'person'])
cv2.putText(img, f'v={V_AUTO * 3.6:.0f}km/h    S_stop={ss_val:.1f}m
Pedestrians={np_}', (20, 185), cv2.FONT_HERSHEY_SIMPLEX, 0.7, (180, 180, 180), 2)

xr = max(20, ow - 700)

cv2.putText(img, 'PEDESTRIANS:', (xr, 45), cv2.FONT_HERSHEY_SIMPLEX, 0.9,
(255, 255, 255), 2)

for i, (tid, a2, tc_v, d2) in enumerate(per_track[:5]):
    yy = 80 + i * 32
    c2, a2l = acm.get(a2, ((255, 255, 255), '?'))
    cv2.putText(img, f'ID:{tid} D={d2:.1f}m TTC={min(tc_v, 99):.1f}s {a2l}', (xr, yy),
cv2.FONT_HERSHEY_SIMPLEX, 0.6, c2, 2)

tc5 = (time.time() - t5) * 1000
tt = (time.time() - t0) * 1000
timing.append({'c1': tc1, 'c2': tc2, 'signs': tc_signs, 'tr': tc3, 'dm': tc4, 'vis': tc5, 'tot': tt})
frame_decisions.append((idx, overall, reason))

for item in per_track:
    decisions_log.append({'f': idx, 'id': item[0], 'act': item[1], 'ttc': item[2], 'dist': item[3]})

wr.write(img)
cap.release()
wr.release()
print(f'\n{'=' * 60}')
print('RESULTS')
print('=' * 60)
print('\nDetections:')
if len(dc) == 0:
    print(' No detections')
else:
    for c, n in dc.most_common():
        print(f' {c:<25}: {n}')
if timing:

```

```

print('\nTiming per frame:')
for k, nm in [('c1', 'COCO det'), ('c2', 'Crosswalk det'), ('signs', 'Road signs det'), ('tr',
'Tracking'), ('dm', 'Decision'), ('vis', 'Drawing'), ('tot', 'TOTAL')]:
    vals = [t[k] for t in timing]
    print(f' {nm:<20}: {np.mean(vals):.1f} ms')
avg_total = np.mean([t['tot'] for t in timing])
if avg_total > 0:
    print(f' FPS: {1000 / avg_total:.1f}')
if frame_decisions:
    print('\nDecisions summary:')
    acounter = Counter([f[1] for f in frame_decisions])
    tf = len(frame_decisions)
    for a, n in acounter.most_common():
        label = {'EMERGENCY_BRAKE': 'EMERGENCY BRAKE', 'SLOW_DOWN':
'SLOW DOWN', 'WARNING': 'WARNING', 'NO_ACTION': 'NO ACTION'}.get(a, a)
        print(f' {label:<25}: {n} frames ({n / tf * 100:.1f}%)')
print('\nBraking distance, v=50 km/h:')
for nm, mu in [('Dry', 0.7), ('Wet', 0.5), ('Ice', 0.3)]:
    v = V_AUTO
    driver_stop = v * 1.2 + v ** 2 / (2 * mu * 9.81)
    system_stop = v * 0.2 + v ** 2 / (2 * mu * 9.81)
    gain = driver_stop - system_stop
    print(f' {nm}: driver={driver_stop:.1f}m, system={system_stop:.1f}m, gain={gain:.1f}
m')

```

18. Сохранение и скачивание результата

```

print(f'\nDone! Video saved to: {out_path}')print(f'\nDone! Video saved to: {out_path}')
if os.path.exists(out_path):
    print('Начинается автоматическое скачивание видео...')
    time.sleep(1)
    files.download(out_path)
else:
    print(f'Файл результата не найден: {out_path}')

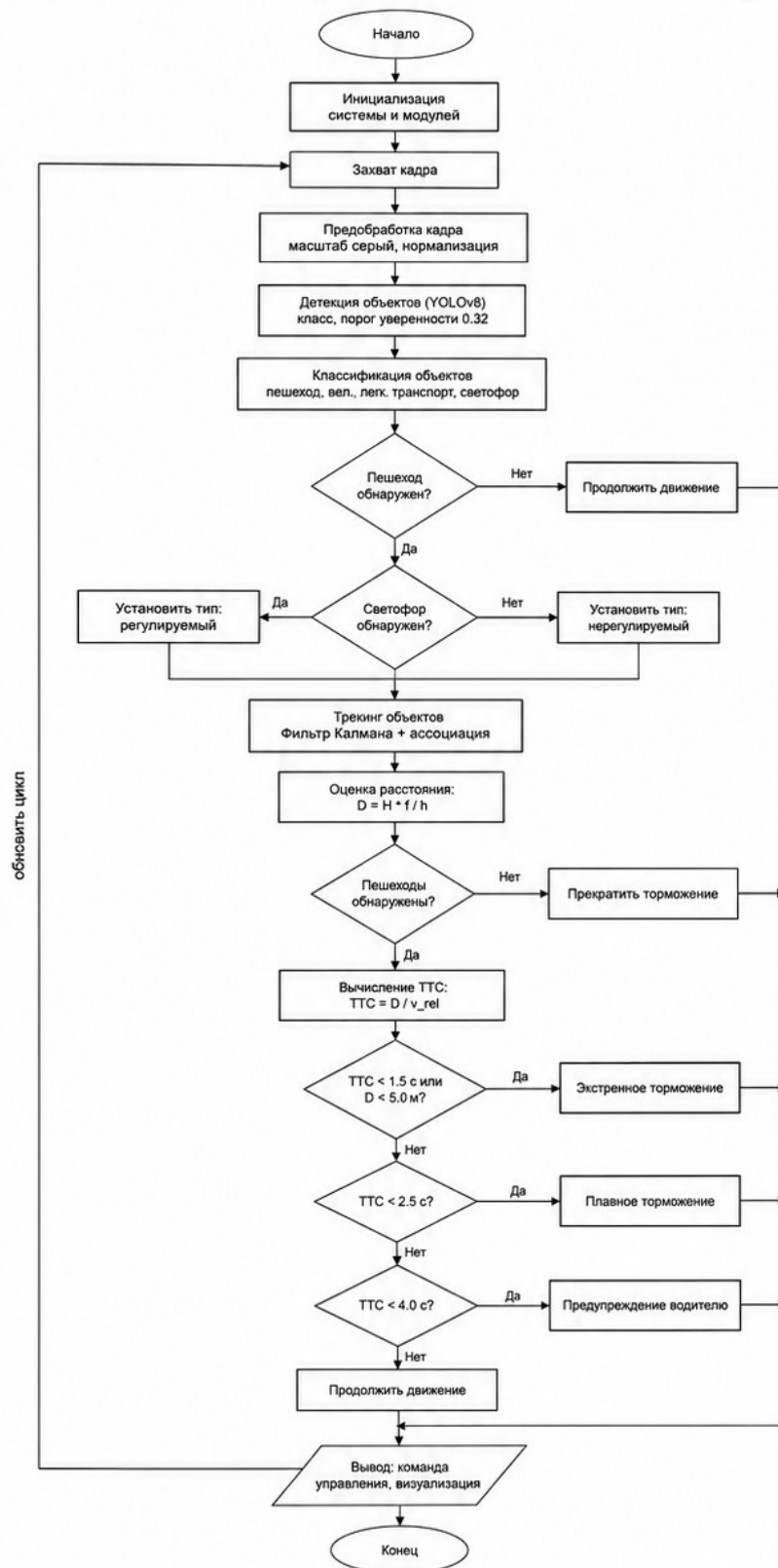
```

Приложение Б

(обязательное)

Иллюстративно-графические материалы

Структурная схема системы автоматической идентификации подвижных объектов в зоне пешеходных переходов



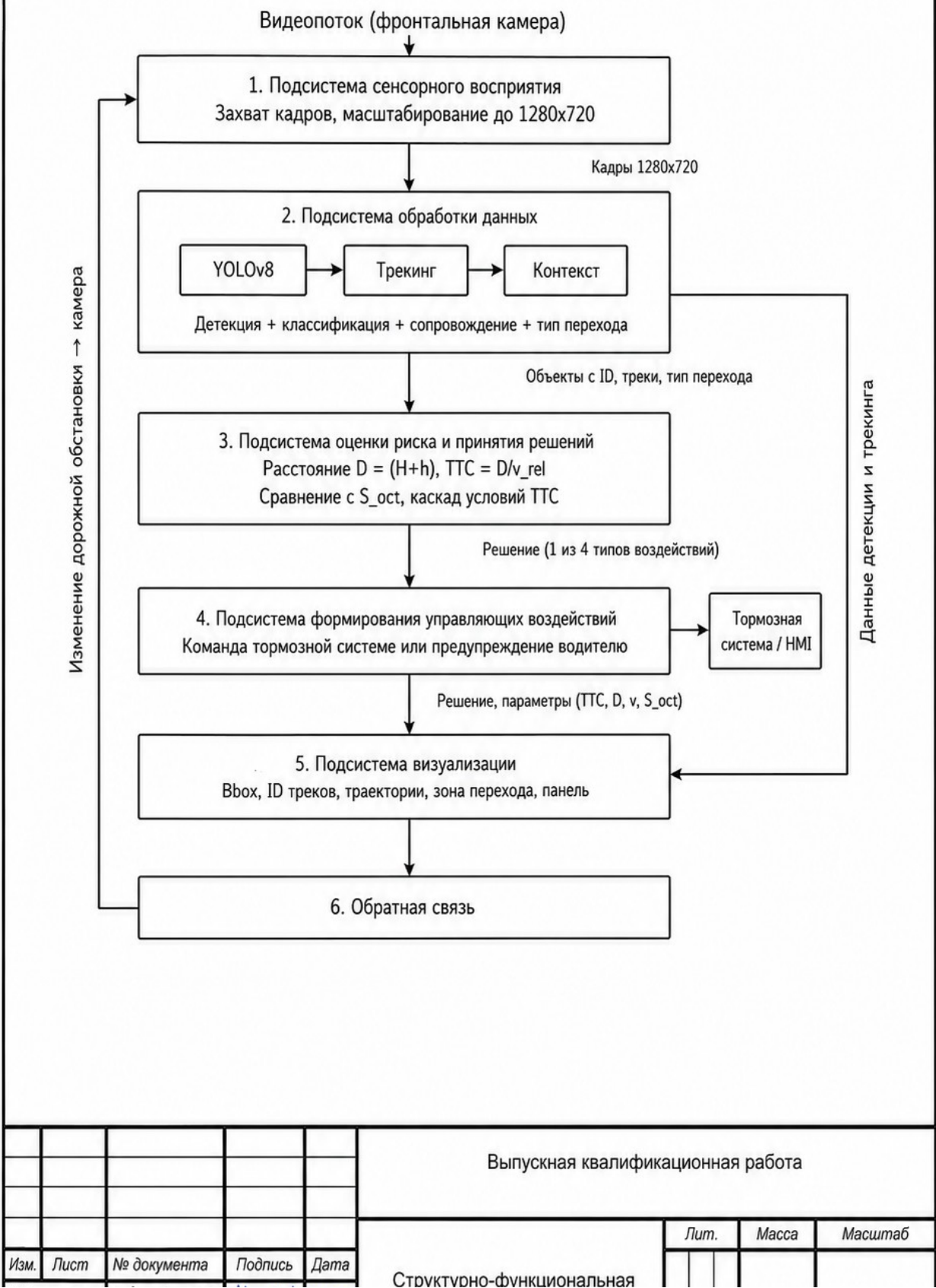
Изм.	Лист	№ документа	Подпись	Дата

Выпускная квалификационная работа

Структурная схема системы

Лит.	Масса	Масштаб

Структурно-функциональная схема взаимодействия подсистем



Выпускная квалификационная работа

Изм.	Лист	№ документа	Подпись	Дата

Структурно-функциональная

Лит.	Масса	Масштаб

Перечень анализируемых дорожных знаков и элементов разметки

Обозначение	Наименование	Тип
5.19.1	Пешеходный переход (правосторонний)	Особых предписаний
5.19.2	Пешеходный переход (левосторонний)	Особых предписаний
1.22	Пешеходный переход (предупреждающий)	Предупреждающий
1.23	Дети (предупреждающий)	Предупреждающий
1.14	Разметка пешеходного перехода (зебра)	Горизонтальная разметка

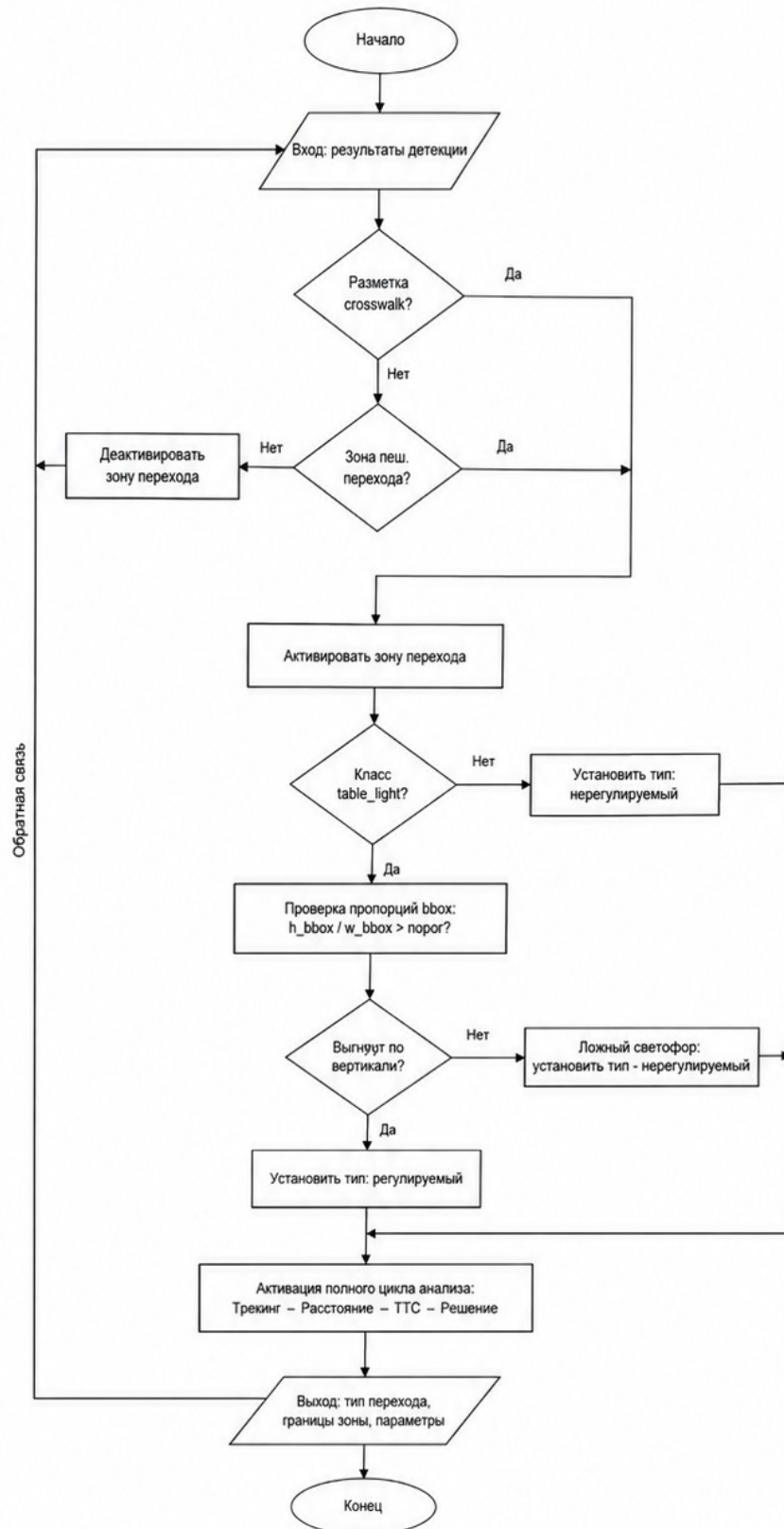
Детекция знаков: модель дообучена на наборе данных RTSD (4 класса из 156).

Детекция разметки: модель YOLOv8s дообучена на FPVCrosswalk2025 + Roboflow.

Детекция светофоров: предобученная модель YOLOv8n (класс traffic_light, COCO).

					Выпускная квалификационная работа			
Изм.	Лист	№ документа	Подпись	Дата	Перечень анализируемых	Лит.	Масса	Масштаб

Алгоритм контекстно-зависимой обработки видеопотока в зоне пешеходного перехода



Выпускная квалификационная работа

ИЗУ-2010Б-24

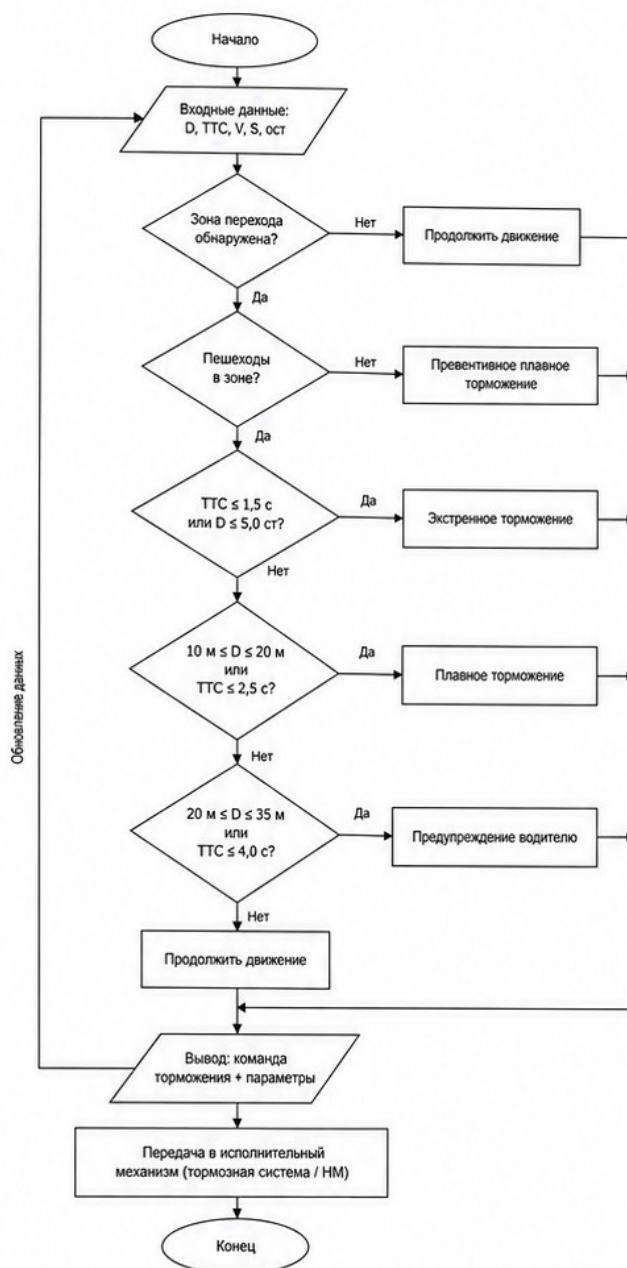
Лит.

Масса

Масштаб

МАИ, каф. СУТ

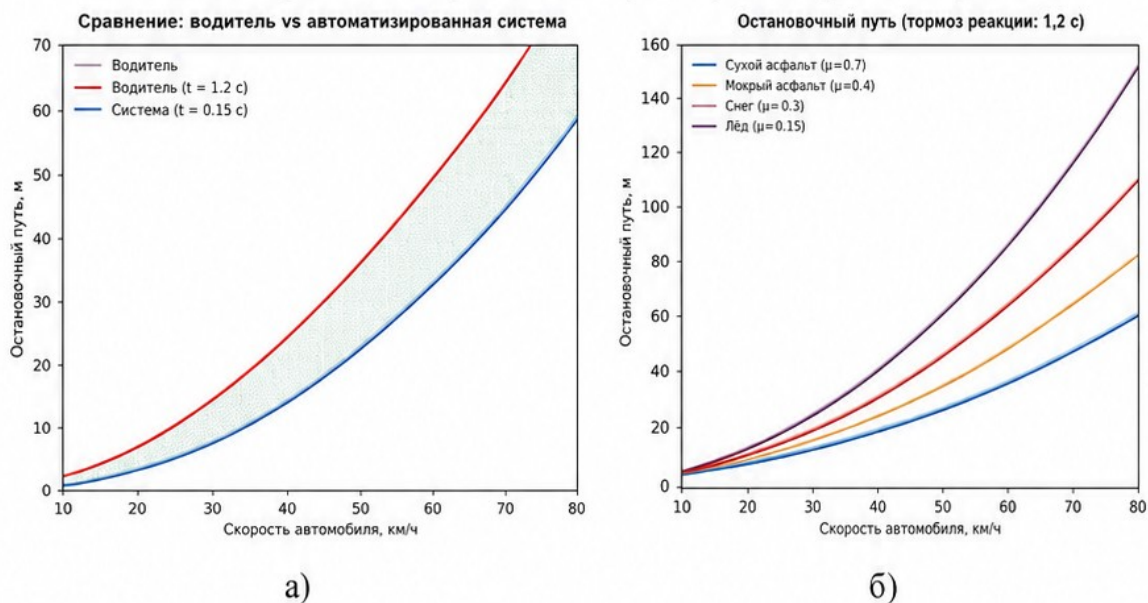
Алгоритм принятия решения о необходимости снижения скорости или торможения автомобиля



Экстренное подтверждение: если confidence > 0,85 и D < 15 м, решение формируется без ожидания подтверждения трека (n_init).

					Выпускная квалификационная работа			
Изм.	Лист	№ документа	Подпись	Дата	Алгоритм принятия решения о		Лит.	Масштаб

Модель расчёта тормозного пути транспортного средства



Модель расчёта тормозного пути транспортного средства:

а – зависимость остановочного пути от скорости при различных дорожных покрытиях;

б – сравнение остановочного пути водителя и автоматизированной системы

Формулы:

$S_{\text{торм}} = v^2 / (2 \cdot \mu \cdot g)$ – тормозной путь.

$S_{\text{ост}} = v \cdot t_{\text{реакц}} + v^2 / (2 \cdot \mu \cdot g)$ – полный остановочный путь.

Таблица – Коэффициенты сцепления и тормозной путь при $v = 50$ км/ч:

Тип покрытия	Коэффициент μ	$S_{\text{торм}}$ при 50 км/ч, м
Сухой асфальт	0,70–0,80	13,8–11,9
Мокрый асфальт	0,40–0,50	24,2–19,3
Снежное покрытие	0,25–0,35	38,7–27,6
Обледенелое покрытие	0,15–0,25	64,5–38,7

Сравнение остановочного пути ($v = 50$ км/ч, $\mu = 0,7$, сухой асфальт):

Водитель ($t_{\text{реакц}} = 1,0–1,5$ с): $S_{\text{ост}} \approx 31$ м.

Система ($t_{\text{реакц}} = 0,1–0,2$ с): $S_{\text{ост}} \approx 17$ м.

Выигрыш: ≈ 14 м (сокращение остановочного пути на 30–45 %)

					Выпускная квалификационная работа

Результаты оценки качества детекции объектов по классам (лист 1 из 2)

Таблица – Результаты оценки качества детекции:

Класс объекта	Precision	Recall	mAP@0.5	mAP@0.5:0.95
person (пешеход)	0,87	0,82	0,83	0,64
bicycle (велосипед)	0,79	0,74	0,81	0,52
car (автомобиль)	0,91	0,88	0,93	0,71
traffic light (светофор)	0,84	0,78	0,85	0,58
sign_5_19_1 (знак)	0,82	0,75	0,83	0,55
sign_5_19_2 (знак)	0,80	0,73	0,81	0,53
crosswalk (разметка)	0,76	0,69	0,77	0,48

Наибольшая точность: car – крупный объект, хорошо представлен в COCO.

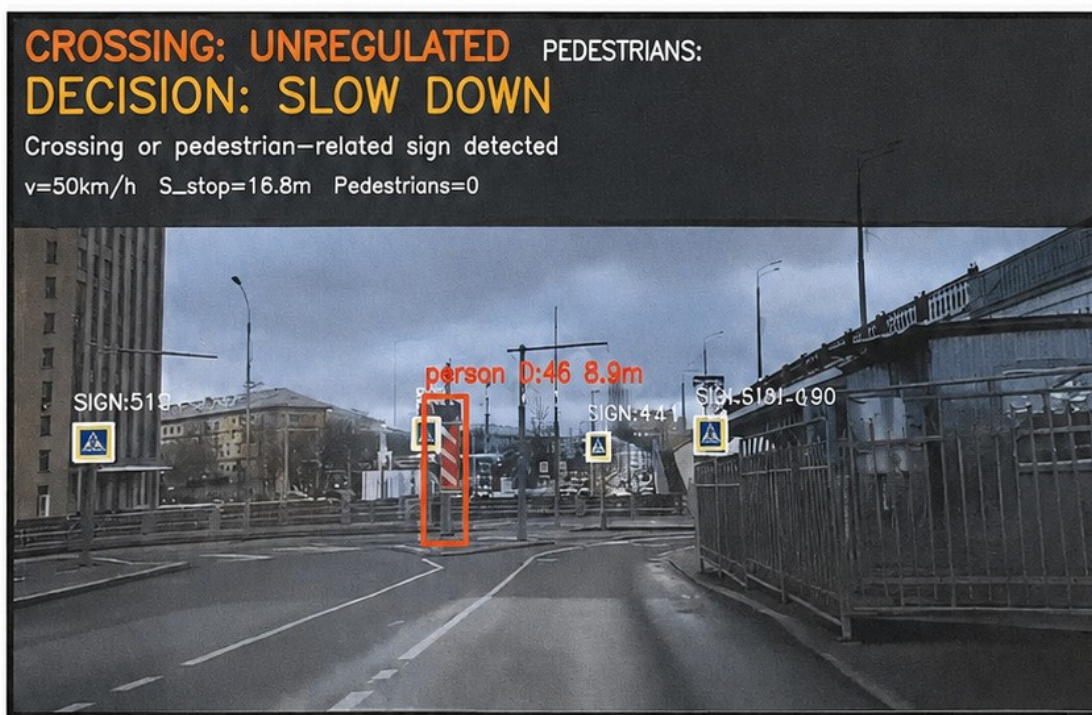
Наименьшая точность: crosswalk – сложная геометрия, зависимость от ракурса и состояния разметки.

					Выпускная квалификационная работа		
Изм.	Лист	№ документа	Подпись	Дата		Лит.	Масса
							Масштаб

Результаты оценки качества детекции объектов по классам (лист 2 из 2)

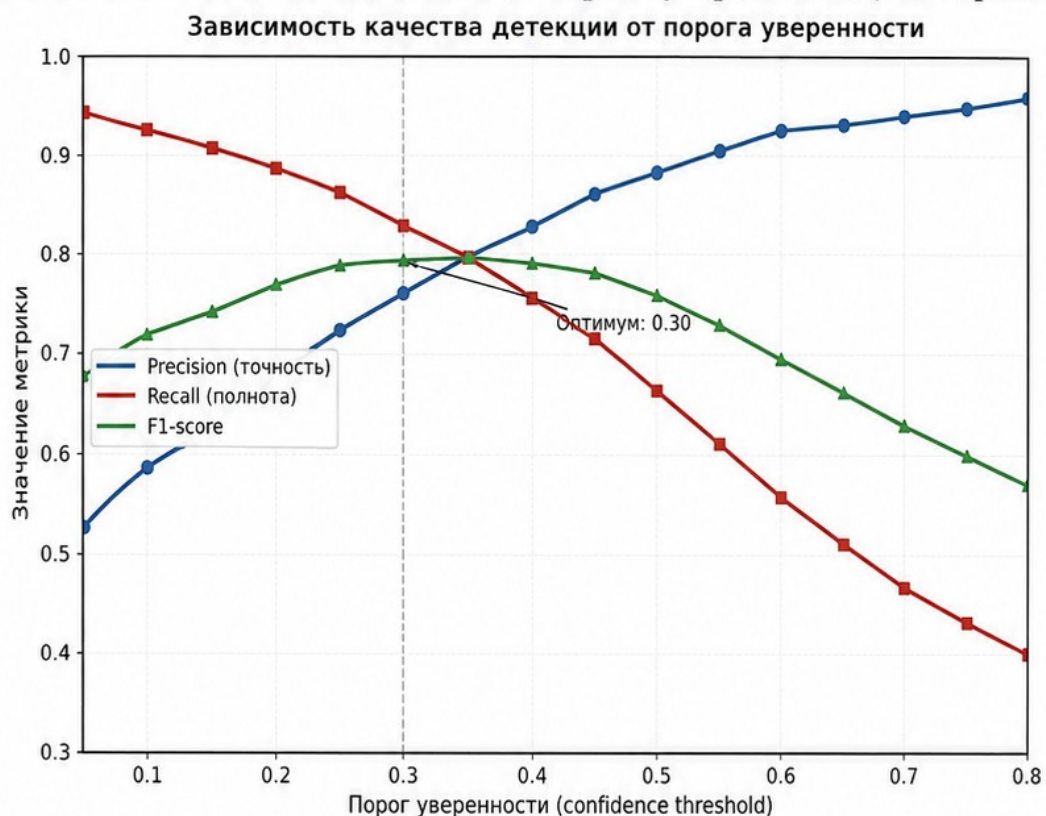
Таблица – Классификация ошибок детекции:

Тип ошибки	Доля, %	Типичная причина
FP: ошибка класса	35	Столб принят за пешехода
FP: фоновый объект	25	Тень, отражение в витрине
FP: дубликат	15	Недостаточный порог NMS
FN: малый размер	40	Пешеход > 40 м
FN: перекрытие	30	Частичное закрытие объектом
FN: плохое освещение	20	Ночь, контровой свет



Пример ложноположительного срабатывания: столб классифицирован как пешеход (дождливая погода, сниженное качество изображения).

Зависимость качества детекции от порога уверенности (класс person)



Оптимальный порог: confidence = 0,30.

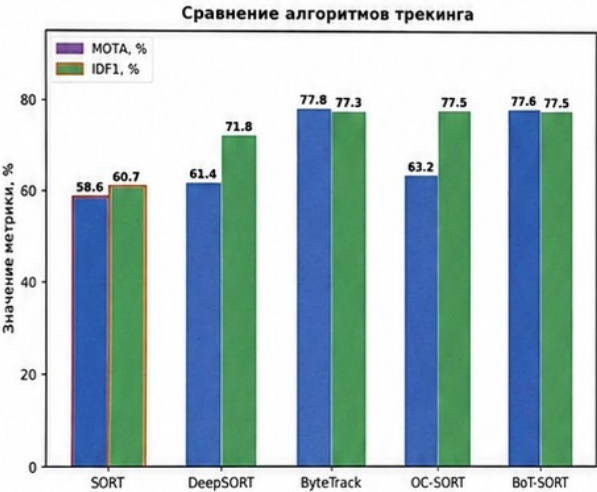
При снижении < 0,20: резкий рост ложных срабатываний.

При повышении > 0,50: недопустимое снижение Recall (пропуск пешеходов опаснее ложных срабатываний).

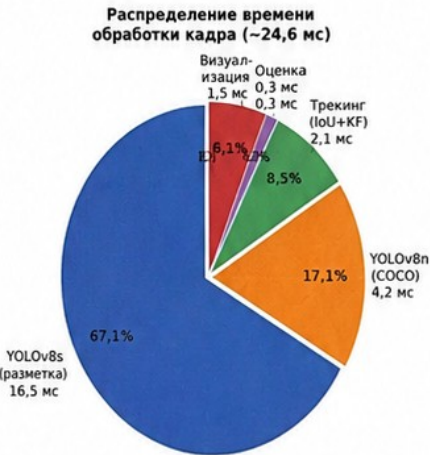
					Выпускная квалификационная работа			
Изм.	Лист	№ документа	Подпись	Дата	Зависимость качества детекции			
					Лит.	Масса	Масштаб	

Модуль	Время, мс	Доля, %
Детекция YOLOv8n (COCO)	4,2	17,1
Детекция разметки (YOLOv8s)	16,5	67,1
Трекинг (IoU + фильтр Калмана)	2,1	8,5
Оценка риска (TTC)	0,3	1,2
Визуализация	1,5	6,1
ИТОГО	24,6	100

Скорость обработки: ≈ 40 FPS (24,6 мс/кадр).



а)



б)

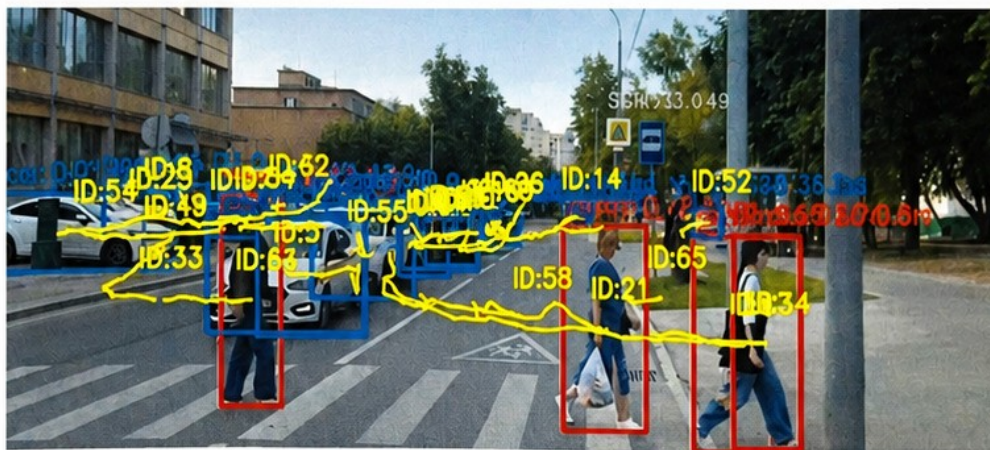
Результаты экспериментальных исследований:
а – сравнение метрик MOTA и IDF1 алгоритмов трекинга;
б – распределение времени обработки кадра по модулям системы

					Выпускная квалификационная работа			
Изм.	Лист	№ документа	Подпись	Дата	Оценка рецензентской	Лит.	Масса	Масштаб

Примеры работы системы в различных условиях наблюдения (лист 1 из 2)

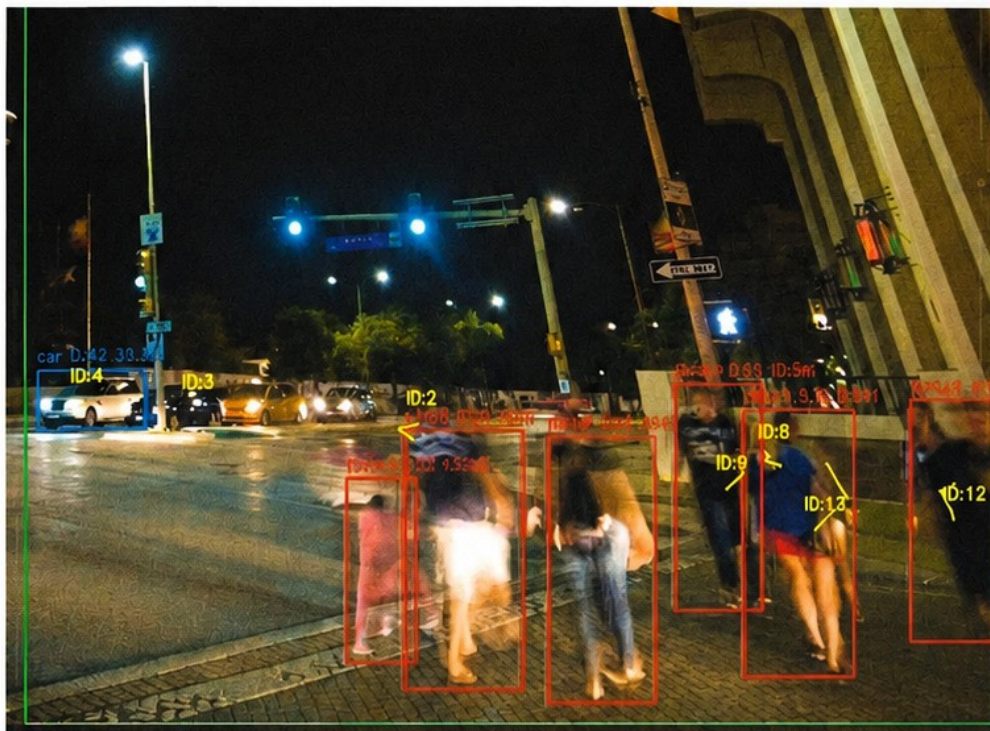
1: Дневные условия, ясно.

Группа пешеходов, нерегулируемый переход.



2: Сумеречные условия.

Группа пешеходов, регулируемый переход.



Изм.	Лист	№ документа	Подпись	Дата

Выпускная квалификационная работа

Примеры работы системы в

Лит.	Масса	Масштаб

Примеры работы системы в различных условиях наблюдения (лист 2 из 2)

3: Дождь. Капли на камере, блики от мокрого асфальта.
Отсутствие пешеходов, нерегулируемый переход.

CROSSING: UNREGULATED PEDESTRIANS:
DECISION: SLOW DOWN
Crossing or pedestrian-related sign detected
v=50km/h S_stop=16.8m Pedestrians=0

Моделирование работы системы на тестовых видеоданных (лист 1 из 2)

Сценарий: Внезапное появление пешехода на регулируемом переходе.

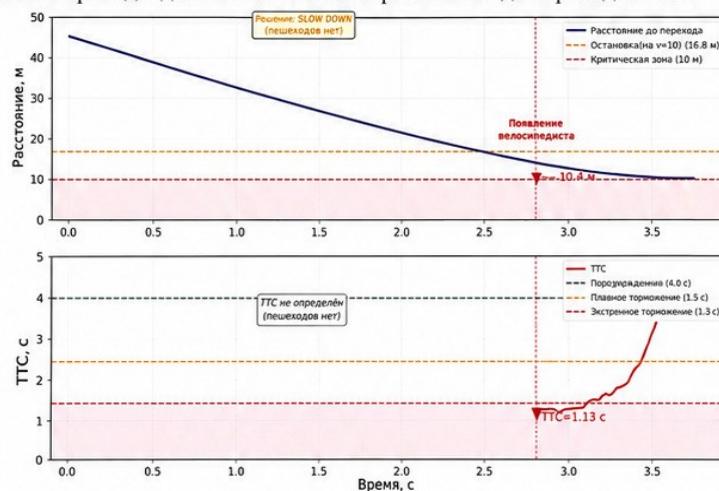
1. Детекция регулируемого пешеходного перехода при отсутствии пешеходов (вечернее время, плавное торможение):



2. Детекция внезапно появившегося велосипедиста в зоне регулируемого пешеходного перехода: формирование команды экстренного торможения (вечернее время):



Результаты моделирования сценария внезапного появления велосипедиста на регулируемом переходе: динамика изменения расстояния до перехода и TTC:



Выпускная квалификационная работа

Моделирование работы системы на тестовых видеоданных (лист 2 из 2)

Таблица - Итоговая сводка результатов:

Параметр	Значение	Целевое значение
mAP@0.5 (person, день)	0,83	≥ 0,75
mAP@0.5 (person, ночь)	0,60	≥ 0,50
MOTA	58,6 %	≥ 50 %
IDF1	60,7 %	≥ 55 %
Время обработки кадра	~25 мс	≤ 50 мс
FPS	~40	≥ 20
Время срабатывания	160–240 мс	≤ 500 мс
Ложные пропуски	3,8 %	≤ 15 %
Ложные срабатывания	5,2 %	≤ 10 %
Сокращение ост. пути	30–45 %	≥ 25 %

Все показатели соответствуют целевым значениям прототипа.

Таблица – Результаты по типовым сценариям:

Сценарий	Тестов	Верные срабатывания, %	Ложные срабатывания, %	Пропуски, %
Одиночный пешеход (день)	120	97,5	2,5	0,0
Одиночный пешеход (ночь)	80	88,7	5,0	6,3
Группа (3–5 чел.)	60	93,3	3,3	3,3
Велосипедист	40	90,0	5,0	5,0
Нерегулируемый переход	100	95,0	3,0	2,0
Регулируемый переход	80	96,3	2,5	1,3
Дождь (слабый)	40	92,5	5,0	2,5
Туман (средний)	30	83,3	6,7	10,0
Внезапное появление	50	94,0	2,0	4,0

					Выпускная квалификационная работа			
Изм.	Лист	№ документа	Подпись	Дата		Лит.	Масса	Масштаб